# COMP2012: Object-Oriented Programming and Data Structures

WU Jinjun

October 27, 2026

## Contents

**1 Review, Pointer, Constness** **6**
1.1 Review: Reference and Pointer . . . 6
1.1.1 Three Kinds of Variables . . . 6
1.1.2 Pointer vs. Reference (3 Big Differences) . . . 6
1.2 New Features Since C++11 . . . 6
1.2.1 Feature List . . . 6
1.2.2 {} Uniform Initialization . . . 7
1.2.3 Range-for . . . 7
1.3 Review Example: Person and Family . . . 8
1.3.1 Header: `person.h` . . . 8
1.3.2 Test Program: `family.cpp` . . . 8
1.3.3 Implementation: `person.cpp` . . . 9
1.4 General Remarks on C++ Classes . . . 10
1.4.1 In-class Member Initializers (C++11) . . . 10
1.4.2 Three Ways to Define Member Functions . . . 10
1.4.3 Class Scope and the `::` Operator . . . 11
1.4.4 this Pointer . . . 11
1.4.5 Parameter-Passing & Return Methods . . . 14
1.5 const-ness . . . 16
1.5.1 `const` Basics . . . 16
1.5.2 const Objects + const Member Functions . . . 17
1.5.3 const Pointers: Four Combinations . . . 19
1.5.4 const References as Function Parameters . . . 20
1.5.5 Const Summary — Good Practice . . . 21
1.5.6 Lambda Expressions (Brief Recap) . . . 22

**2 Constructors, Destructor** **23**
2.1 Object Initialization: Constructors . . . 23
2.1.1 Aggregate (public-member) Initialization . . . 23
2.1.2 Constructor Syntax . . . 23
2.1.3 Default Initializers for Non-static Data Members (C++17) . . . 24
2.1.4 Default Constructor . . . 24
2.1.5 Conversion Constructors . . . 25
2.1.6 Copy Constructor . . . 27

2.1.7 Default Copy Constructor & Default Assignment Operator . . . . . 29
2.1.8 Uniform Initialization Using {} Initializers Again . . . . . 32
2.1.9 Constructors and Function Overloading . . . . . 32
2.1.10 Default Arguments (Reminder) . . . . . 33
2.2 Member Initializer List (MIL) . . . . . 34
2.2.1 MIL Syntax . . . . . 34
2.2.2 MIL for User-Defined-Type Members . . . . . 35
2.2.3 `const` or Reference Members **Must** Use MIL . . . . . 35
2.2.4 Delegating Constructor vs. Private Utility Function . . . . . 37
2.2.5 Example: Delegating Constructor (C++11) . . . . . 37
2.3 Destructor . . . . . 42
2.3.1 Program Memory Layout . . . . . 42
2.3.2 Destructor Syntax . . . . . 42
2.3.3 Bug: Default Memberwise Assignment . . . . . 44
2.3.4 Summary: 6 Compiler-Generated Member Functions . . . . . 44
2.4 Order of Construction and Destruction . . . . . 46
2.4.1 "Has" Relationship . . . . . 46
2.4.2 Has-Relationship: Construction/Destruction Order (postoffice1.h) . 46
2.4.3 Owns-Relationship: Heap Sub-Object Needs Explicit delete! (postoffice2.h) . . . . . 47
2.4.4 Two Embedded Members: Room + Clock (postoffice4.h) . . . . . 48
2.4.5 Nested: Postoffice Has Room, Room Has Clock (postoffice5.h) . . . 49
2.4.6 Temporary Object in Constructor Body (postoffice6.h) . . . . . 49
2.4.7 Default Member Initialization and Order of Construction . . . . . 50
2.4.8 Order of Construction & Destruction of Array Elements . . . . . 51
2.4.9 Order Summary . . . . . 52

**3 Inheritance, Polymorphism 56**
3.1 Inheritance Basics . . . . . 56
3.1.1 Motivation: Duplicate Code is Bad . . . . . 56
3.1.2 Why Inheritance? Code Reuse . . . . . 57
3.1.3 UPerson Example: University Member System . . . . . 57
3.1.4 Inherited Member Access . . . . . 58
3.1.5 LSP in Action: Print Label Example . . . . . 59
3.1.6 Indirect Inheritance: PG_Student . . . . . 60
3.2 Inheritance: C/D order + Slicing + Name Hiding + Access Control . . . . 62
3.2.1 Constructor / Destructor Order with Inheritance . . . . . 62
3.2.2 Fatal Trap: Object Slicing . . . . . 66
3.2.3 Name Hiding (NOT Overloading!) . . . . . 67
3.2.4 Motivation for Access Control: Student::print( ) Problem . . . . . 68
3.2.5 One Solution: Protected Data Members . . . . . 69
3.2.6 Superior Solution: Use Public Member Functions . . . . . 69
3.2.7 Three Access Specifiers: public / protected / private . . . . . 70
3.3 Polymorphism & Virtual Functions . . . . . 71
3.3.1 Static Binding vs. Dynamic Binding . . . . . 72
3.3.2 Example: Polymorphism using Virtual Function . . . . . 74

3.3.3 C++11 `override` Keyword: Signature Safety Net . . . 75
3.3.4 Overloading vs. Overriding — never confuse again . . . 76
3.3.5 RTTI: Run-Time Type Information . . . 76
3.4 Virtual Destructor / ABC / final / Three Kinds of Inheritance . . . 83
3.4.1 Virtual Destructor (EXAM FOCUS!) . . . 83
3.4.2 Order of Construction and Destruction with Polymorphism . . . 84
3.4.3 Virtual Functions Called from Constructor / Destructor . . . 85
3.4.4 Abstract Base Classes (ABC) and Pure Virtual Functions . . . 87
3.4.5 Solution: Abstract Base Class (ABC) . . . 88
3.4.6 ABC as an Interface . . . 88
3.4.7 Final Remarks on ABC . . . 89
3.4.8 C++11 `final`: Two Separate Uses . . . 89
3.4.9 Three Kinds of Class-Wide Inheritance Access . . . 90

**4 Generic Programming 95**
4.1 Function Templates . . . 95
4.1.1 Motivation: Copy-and-Paste Does Not Scale . . . 95
4.1.2 Function Template Syntax . . . 96
4.1.3 Instantiation, Argument Matching, and Explicit Instantiation . . . 97
4.1.4 When the General Rule Is Wrong: Template Specialization . . . 98
4.1.5 Multiple Type Parameters and the Return-Type Problem . . . 100
4.1.6 Return by Reference vs. Return by Value: Why the Temporary Bites 102
4.1.7 Code Bloat . . . 104
4.2 Class Templates . . . 105
4.2.1 Defining a Class Template . . . 105
4.2.2 Using a Class Template . . . 106
4.2.3 Nontype Parameters . . . 106
4.2.4 Class Templates vs. Function Templates: Deduction and Separate Compilation . . . 109
4.3 Operator Overloading . . . 111
4.3.1 Motivation: Operators Are Just Functions . . . 111
4.3.2 Member Operator Functions: `+` and `+=` . . . 113
4.3.3 Commutativity, Implicit Conversion, and the Left-Operand Trap . . . 115
4.3.4 Non-member `operator<<` and Cascading Output . . . 116
4.3.5 Assignment: `operator=` . . . 117
4.3.6 Indexing and Increment: `operator[]` and `operator++` . . . 119
4.3.7 Summary: Member or Non-member? . . . 122
4.4 Friend Functions and Classes . . . 123
4.4.1 Why `operator<<` Cannot Be a Member . . . 123
4.4.2 Friend Functions and Classes . . . 124
4.4.3 Example: Student and Hacker: When Friendship Goes Wrong . . . 125

**5 Lambda Functions & Function Pointers 128**
5.1 Lambda Expressions . . . 128
5.1.1 Syntax and Rules . . . 128
5.1.2 Example: Simple Lambdas with No Captures . . . 128
5.1.3 Example: Lambdas with Captures . . . 129

5.1.4 Question: When Are Values Captured? . . . 130
5.1.5 Question: When Are References Captured? . . . 131
5.1.6 Capture Semantics: Value, Reference, `mutable` . . . 132
5.1.7 Mixing a Capture Default with Exceptions: `[&, a]` . . . 132
5.2 Function Pointers . . . 133
5.2.1 Example: Choosing a Function at Run Time . . . 133
5.2.2 Example: Array of Function Pointers — Calculator . . . 134
5.2.3 Example: A Function Pointer Pointing to a Lambda . . . 134
5.3 Nested Lambdas . . . 135

**6 Trees, Binary Trees, and Binary Search Trees 137**
6.1 Complexity Review: Big-O Notation . . . 137
6.2 Tree Terminology . . . 137
6.3 Binary Trees . . . 138
6.3.1 A Typical Implementation . . . 138
6.3.2 Traversals: `action` as a Function Pointer . . . 139
6.3.3 Example 2: Tree Creation and Traversal . . . 140
6.3.4 Example 4: Expression Trees . . . 141
6.4 Binary Search Trees . . . 142
6.4.1 BST Search . . . 142
6.4.2 Our BST Implementation (different from the textbook's) . . . 142
6.4.3 BST Code: Search . . . 143
6.4.4 BST Code: Print by Rotating the Tree 90 Degrees . . . 144
6.4.5 Finding the Minimum and Maximum . . . 144
6.4.6 Insertion . . . 145
6.4.7 Deletion: the Three Cases . . . 145
6.4.8 BST Code: Deletion . . . 146
6.4.9 BST Testing Program . . . 148

**7 Hashing 150**
7.1 Motivation . . . 150
7.2 The Hash Table ADT . . . 150
7.2.1 Unrealistic Solution: One Slot per Key . . . 150
7.2.2 Hash Functions . . . 150
7.2.3 Hash Function Design . . . 151
7.3 Collision Handling I: Separate Chaining . . . 152
7.4 Collision Handling II: Open Addressing . . . 152
7.4.1 Linear Probing . . . 152
7.4.2 Quadratic Probing . . . 153
7.4.3 Double Hashing . . . 153
7.5 Deletion in Open Addressing . . . 154
7.6 Load Factor and Re-hashing . . . 154

**8 The Standard Template Library (STL) 156**
8.1 Container Classes . . . 156
8.1.1 The Array Class Template . . . 156
8.1.2 `operator<<` as a Global or a Friend Template . . . 157
8.1.3 Example: Using the Class . . . 158

8.2 Containers in the STL . . . 160
8.2.1 Properties of Sequence Containers . . . 160
8.3 Container Adaptors: Stack and Queue . . . 160
8.3.1 Stack (LIFO) . . . 160
8.3.2 Queue (FIFO) . . . 163
8.4 STL Iterators: Generalized Pointers . . . 164
8.4.1 Pointers Traverse an Array . . . 164
8.4.2 Iterators Decouple Algorithms from Containers . . . 165
8.5 STL Algorithms . . . 167
8.5.1 Example: the 2nd Occurrence of a Value . . . 168
8.5.2 `find_if()`: Searching by Condition . . . 169
8.5.3 Function Objects (Functors) . . . 170
8.5.4 Other Algorithms in the STL . . . 173

# 1 Review, Pointer, Constness

## 1.1 Review: Reference and Pointer

### 1.1.1 Three Kinds of Variables

```
#include <iostream>
using namespace std;

int x = 5;                    // int variable
int& xref = x;                // reference: alias of x
int* xptr = &x;               // pointer: points to x

void xprint()
{
    cout << hex << endl;
    cout << "x= " << x << "\t\tx address = " << &x << endl;
    cout << "xref= " << xref << "\t\txref address = " << &xref
         << endl;
    cout << "xptr= " << xptr << "\txptr address = " << &xptr << endl;
    cout << "*xptr= " << *xptr << endl;
}

int main()
{
    x += 1;      xprint();
    xref += 1;   xprint();
    xptr = &xref; xprint();   // still points to x
    return 0;
}
```

**Key observation:** After `xptr = &xref`, `xptr` still points to `x` (since `xref` is just an alias of `x`). The addresses of `x` and `xref` are identical.

### 1.1.2 Pointer vs. Reference (3 Big Differences)

1. **Nullability:** A pointer can be `nullptr`; a reference is **always** bound to an object.
2. **Re-bindability:** A pointer can be reassigned to different objects. A reference is permanently bound to **the same object** — assigning to a reference only changes the *value* of the referenced object, not what it refers to.
3. **Access Syntax:** The pointer's name refers to the pointer itself; you need `*` or `->` to access the pointee. A reference's name always refers to the object directly.

## 1.2 New Features Since C++11

### 1.2.1 Feature List

- Uniform initialization with {} lists
- `auto` type deduction *(not allowed in COMP2011/2012)*
- Narrowing prevention (via {})
- `constexpr` — compile-time constants
- Range-for statement
- `nullptr` (type-safe null pointer)
- `enum class` (scoped, strongly-typed enums)

- Rvalue references & move semantics †
- Lambda expressions †
- `long long`
- Delegating constructors †
- In-class member initializers †
- Explicit conversion operators †
- `override` / `final` †

### 1.2.2 {} Uniform Initialization

Traditional = style vs. C++11 {} style:

```
/* =  style  */              /* {} style          */
int x = 5;                   int x = {5};
float y = 9.8;               float y {9.8};
int& xref = x;               int& xref {x};
int a[] = {1,2,3};           int a[] {1, 2, 3};      // = is optional
```

**Example: Narrowing Prevention**

```
int w = 3.4;              // OK: implicit narrowing, w == 3
int x1 {6};               // OK
int x2 = {8};             // OK (= optional)
const int y {'k'};        // OK (char->int promotion)
int z {6.4};              // COMPILE ERROR: narrowing double->int
```

**Why {} is preferred**

- Disallows **narrowing conversions** (implicit information loss)
- Works uniformly for scalars, arrays, aggregates, and class objects

### 1.2.3 Range-for

```
/* Traditional index loop */
for (int k = 0; k < 5; ++k)
    cout << k*k << endl;

/* C++11 range-for */
for (int k : { 0, 1, 2, 3, 4 })
    cout << k*k << endl;
for (int k : { 1, 19, 54 })          // list need not be consecutive
    cout << k*k << endl;

int range[] {2, 5, 27, 40};

for (int k : range)                  // read-only: copies each element
    cout << k*k << endl;

for (int& x : range)                 // in-place modification via ref
    x *= 2;
```

**Rule of thumb:** Use `for (auto& x : ...)` (or `const auto&`) when you don't need a copy.

## 1.3 Review Example: Person and Family

Each `Person` has at most one child; father/mother may be unknown. A "family print" shows the person, parents, and all 4 grandparents.

### 1.3.1 Header: `person.h`

```
#include <iostream>
using namespace std;

class Person
{
private:
    char*  _name;
    int    _age;
    Person* _father;
    Person* _mother;
    Person* _child;

public:
    Person(const char* my_name, int my_age,
           Person* my_father = nullptr,
           Person* my_mother = nullptr,
           Person* my_child  = nullptr);
    ~Person();

    Person* father() const;
    Person* mother() const;
    Person* child()  const;
    void print_age()    const;
    void print_name()   const;
    void print_family() const;

    void have_child(Person* baby);
};
```

### 1.3.2 Test Program: `family.cpp`

```
#include "person.h"

int main()
{
    Person arthur("Arthur", 65, nullptr, nullptr, nullptr);
    Person becky ("Becky",  63, nullptr, nullptr, nullptr);
    Person claire("Claire", 32, &arthur, &becky, nullptr);
    Person eddy  ("Eddy",    4, nullptr, &claire, nullptr);

    arthur.have_child(&claire);
    becky .have_child(&claire);
    claire.have_child(&eddy);

    arthur.print_family(); cout << endl;
    becky .print_family(); cout << endl;
    claire.print_family(); cout << endl;
    eddy  .print_family();
```

```
    return 0;
}
```

**Expected Output (Eddy only)**

```
Name: Eddy
Father: unknown
Mother: Claire
Grand Fathers: unknown, Arthur
Grand Mothers: unknown, Becky
```

### 1.3.3 Implementation: person.cpp

```
#include "person.h"
#include <cstring>

Person::Person(const char* my_name, int my_age,
               Person* my_father, Person* my_mother, Person* my_child)
{
    _name = new char[strlen(my_name) + 1];
    strcpy(_name, my_name);
    _age    = my_age;
    _father = my_father;
    _mother = my_mother;
    _child  = my_child;
}

Person::~Person() { delete [] _name; }

Person* Person::father() const { return _father; }
Person* Person::mother() const { return _mother; }
Person* Person::child()  const { return _child;  }
void Person::have_child(Person* baby) { _child = baby; }
void Person::print_age()  const { cout << _age; }

void Person::print_name() const
{ cout << (_name ? _name : "unknown"); }

/* ---- helper (not a member) ---- */
void print_parent(Person* parent)
{
    if (parent) parent->print_name();
    else        cout << "unknown";
}

void Person::print_family() const
{
    Person *fg=nullptr, *fm=nullptr, *mg=nullptr, *mm=nullptr;
    if (_father) { fg = _father->father(); fm = _father->mother(); }
    if (_mother) { mg = _mother->father(); mm = _mother->mother(); }

    cout << "Name: ";          print_name();   cout << endl;
    cout << "Father: ";        print_parent(_father); cout << endl;
    cout << "Mother: ";        print_parent(_mother); cout << endl;
```

```
    cout << "Grand Fathers: "; print_parent(fg);
    cout << ", ";               print_parent(mg); cout << endl;
    cout << "Grand Mothers: "; print_parent(fm);
    cout << ", ";               print_parent(mm); cout << endl;
}
```

## 1.4 General Remarks on C++ Classes

### 1.4.1 In-class Member Initializers (C++11)

```
class Complex
{
private:
    float real = 1.3;       // error before C++11
    float imag {0.5};       // either = or {} works
public:
    /* ... */
};
```

**Initialization priority (highest first):**

1. Constructor member-initializer list
2. In-class default member initializer (as above)
3. Otherwise undefined (for built-in types on stack)

### 1.4.2 Three Ways to Define Member Functions

```
class Person {
    ...
    Person* child() const { return _child; }
    void have_child(Person* baby) { _child = baby; }
};
```

```
/* person.h */
class Person {
    ...
    inline Person* child() const;
    inline void have_child(Person* baby);
};
inline Person* Person::child() const { return _child; }
inline void Person::have_child(Person* baby) { _child = baby; }
```

```
/* person.h  */   class Person { ... Person* child() const; ... };
/* person.cpp */   Person* Person::child() const { return _child; }
```

**Why separate into `.h` and `.cpp`?**

- **Compilation speed:** changing the impl only recompiles one .cpp.
- **Encapsulation:** clients see only declarations, not details.

- **Link-time reuse:** pre-compiled .obj/.a can be shipped.

### 1.4.3 Class Scope and the :: Operator

C++ uses **static (lexical) scope**: name binding is resolved at compile time. Names declared inside a class are in class scope. To define a member outside the class, qualify with `ClassName::`.

```
int height = 10;            // global
class Weird
{
    short height;           // member shadows global
    Weird() { height = 5; }
};
```

**Q1:** Which `height` is assigned in `Weird::Weird()`?

**A1:** The **member** `Weird::height`. Inside class scope the inner name shadows the outer one; the global `::height` is untouched.

**Q2:** Can we access the global `height` from inside the class?

**A2:** Yes, with the **unary scope operator**: `::height`.

```
Weird() { ::height = 5; }   // assigns to the GLOBAL int
```

### 1.4.4 this Pointer

Every non-static member function has an implicit first parameter `this`: a pointer of type `ClassName *const` pointing to the calling object.

```
/* source  */                   /* compiled (pseudo-code)  */
void Person::have_child(Person* baby)
{ _child = baby; }       =>     void Person::have_child(
                                   Person *const this, Person* baby)
                                 { this->_child = baby; }

becky.have_child(&eddy); =>     Person::have_child(&becky, &eddy);
```

#### Use #1: Resolving Name Ambiguity

```
class X
{
    int size;
public:
    X(int size) { size = size; }   // BUG: all "size" == parameter!
    void print() const { cout << size << endl; }
};
```

**Q1:** What does `X a(5); a.print();` output?

**A1: Undefined behaviour** (garbage). The parameter shadows the member; `size=size` is a useless self-assignment. `X::size` is never initialised.

**Q2:** Fix.

**A2:** Use an initialiser list (best), different name, or `this->`:

```
X(int size) : size(size) {}          // member-init list: preferred
// or
X(int s)      { size = s; }          // different param name
// or
X(int size)   { this->size = size; } // explicit this
```

**2025S Midterm, Problem 1(c) [2 pts] — which `data` is which?**

**Problem.** What is the output?

```
#include <iostream>
using namespace std;
int data = 0;
class Record {
private:
    int data = 10;
public:
    Record(int data = 100) { data = data; }
    int get() { return data; }
};
int main() {
    int data = 1000;
    Record rec(data);
    cout << data << " " << rec.get() << endl;
    return 0;
}
```

**Answer.** `1000 10`

**Solution.** Everywhere, the **innermost declaration wins**:

- In the constructor, the **parameter** `data` shadows the member, so `data = data;` is a useless **self-assignment of the parameter**. The member keeps its in-class initializer value `10` (same trap as `X(int size) { size = size; }` above).
- In `main`, the unqualified `data` is the local variable (`1000`); the member is reachable only via `rec.get()`.

Hence the output is `1000 10`: local first, member second. (The global `::data` is never touched; it would need `::data`.)

### Use #2: Method Chaining (return `*this` by reference)

```
class Complex
{
    float real, imag;
public:
    Complex(float r, float i) : real(r), imag(i) {}
    void print() const { cout << "(" << real << "," << imag << ")"; }

    Complex  add1(const Complex& x)  // return by VALUE
        { real+=x.real; imag+=x.imag; return *this; }
    Complex* add2(const Complex& x)  // return by RAW POINTER
```

```
        { real+=x.real; imag+=x.imag; return this; }
    Complex& add3(const Complex& x)  // return by REFERENCE (best)
        { real+=x.real; imag+=x.imag; return *this; }
};
```

**Chaining behaviour (given `Complex w(10,10)`):**

- `a.add1(w).add1(w).print(); a.print();`
  - `add1` returns a *copy*, so the 2nd call modifies a **temporary**.
  - Result: `a` is modified **once** only.
- `b.add2(w)->add2(w)->print(); b.print();`
  - `add2` returns `this`; chaining works but uses `->`.
  - Result: `b` is modified **twice**.
- `c.add3(w).add3(w).print(); c.print();`
  - `add3` returns a *reference to the current object*. Clean syntax.
  - Result: `c` is modified **twice**. **Idiom for operator overloading.**

**2024S Midterm, Problem 1(b) [2 pts] — chaining: return `D&` vs. return `D`**

**Problem.** What is the output?

```
#include <iostream>
using namespace std;
class D {
private:
    int d;
public:
    D(int n) : d(n) {}
    D& decrement() { d--; return (*this); }
    D fun1(D& other) {
        return (d > other.d ? decrement() : other.decrement());
    }
    D& fun2(D& other) {
        return (d < other.d ? decrement() : other.decrement());
    }
    void print() const { cout << d << endl; }
};
int main() {
    D n1(3), n2(1), n3(5);
    n1.fun1(n2).fun2(n3);
    n1.print();
    return 0;
}
```

**Answer.** 2

**Solution.** Follow the chain left to right:

- `n1.fun1(n2)`: `n1.d = 3 > other.d = 1`, so the ternary calls `decrement()` on `n1` ($3 \to 2$) and returns **by value** — the rest of the chain works on a temporary copy of `n1` (with `d = 2`).
- `.fun2(n3)` is then invoked **on that temporary**: `temp.d = 2 < n3.d = 5`, so it decrements the temporary and returns a reference to it.
- The temporary is destroyed at the end of the full expression. `n1` was last modified inside `fun1`, so `n1.print()` prints 2.

**Takeaway:** once one link of a chain returns by value, every later call in the chain mutates a temporary, not the original object (compare with the `add3` idiom above, which returns `*this` by reference).

### 1.4.5 Parameter-Passing & Return Methods

**Passing parameters:**

- **Pass-by-value:** a copy is made. Safe for small types, costly for big ones. No side effects on the original.
- **Pass-by-(lvalue)-reference:** no copy; the function can see and/or mutate the caller's object.
- **Pass-by-rvalue-reference (C++11):** for move semantics (later).

**Returning from a function:**

- **Return-by-value:** returns a new copy. Safe even for local variables.
- **Return-by-(lvalue)-reference:** returns the object itself. **Never** return a ref to a local variable (dangling)! Use only when the object outlives the call (e.g. `*this`, or a container element passed in).
- **Return-by-pointer:** similar caveats as ref; additionally may be `nullptr`.

**Doubly linked list (DLL) — the exam checklist**

A DLL node stores `prev` and `next`. Two rules dominate every DLL question:

- **Private members are invisible outside the node class.** Even a member function of `Playlist` (a different class) must use the accessors `getNext()`/`setNext()`/... instead of `temp->next`. Using `temp->next` is not just a syntax error — it is a design violation.
- **Traversal idiom:**

```
SongNode* temp = head;
while (temp != nullptr) { /* visit temp */
    temp = temp->getNext();
}
```

**Insert at head** touches three pointers (the new node's two, plus the old head's `prev`, guarded by `if (head)`):

```
// prev = nullptr, next = old head
SongNode* ptr = new SongNode(str, len, nullptr, head);
// old head's prev points back to the new node
if (head) head->setPrev(ptr);
head = ptr;                          // update head last
```

**Swap two nodes** = six pointer updates (never swap the data!). Order matters: after re-wiring the `prev` pointers you must fix the `next` of the two predecessors; after re-wiring the `next` pointers you must fix the `prev` of the two successors. Also guard: empty list, node(s) not found, and `s1 == s2`.

**2025S Midterm, Problem 4 [25 pts] — `Playlist`: traversal & search (a)(b)**

**Problem.** A song list is managed as a doubly linked list of `SongNodes` (`title`, `length`, `prev`, `next`; all private, with getters/setters and a constructor `SongNode(const char* str, int l, SongNode* p = nullptr, SongNode* n = nullptr)`). Implement the remaining member functions of

```
class Playlist {
public:
    void print() const;                 // given
    int  getTotalLength() const;        // part (a)
    SongNode* search(const char*) const;  // part (b)
```

```
    void insert(const char*, int);    // part (c)
    bool swapSongs(const char*, const char*); // part (d)
private:
    SongNode* head = nullptr;
};
```

**Part (a).** Total length of all songs:

```
int Playlist::getTotalLength() const {
    int total = 0;              // accumulator
    SongNode* temp = head;      // copy the head pointer
    while (temp != nullptr) {   // or: while (temp)
        total += temp->getLength();  // accessor, not temp->length!
        temp = temp->getNext();
    }
    return total;
}
```

**Part (b).** Find the first node whose title matches (case-sensitive), else `nullptr`:

```
SongNode* Playlist::search(const char* str) const {
    SongNode* temp = head;
    while ((temp != nullptr)
           && (strcmp(str, temp->getTitle()) != 0))
        temp = temp->getNext();     // stop early on match
    return temp;                    // nullptr if not found
}
```

Both are O(n) traversals; `search` simply stops at the first match.

## 2025S Midterm, Problem 4 — `insert()` (c)

**Problem.** Insert a new `SongNode` at the **front** of the list. **Solution.**

```
void Playlist::insert(const char* str, int len) {
    // Step 1: new node;
    // its prev = nullptr, its next = original head
    SongNode* ptr = new SongNode(str, len, nullptr, head);
    // Step 2: the original head's prev
    // must point back to the new node
    if (head)                   // or: if (head != nullptr)
        head->setPrev(ptr);
    head = ptr;                 // Step 3: update head
}
```

**Common mistakes:** forgetting the `if (head)` guard (crashes on an empty list — `head->setPrev` dereferences `nullptr`), writing `head->prev = ptr` (private!), or setting `head` before wiring the links. The order above keeps the list consistent at every step. (Checking whether the song already exists is optional.)

## 2025S Midterm, Problem 4 — `swapSongs()` (d)

**Problem.** Swap the nodes (not the data!) of the first two songs matching `str1` and `str2`; return `false` if the list is empty, a song is not found, or both names match the same node. You may define extra `SongNode` pointers but no `SongNode` objects. **Solution** (reusing part (b)):

```
bool Playlist::swapSongs(const char* str1, const char* str2) {
    if (head) {
        SongNode *s1 = nullptr, *s2 = nullptr;
```

```
        // Step 1: locate both songs
        s1 = search(str1);  s2 = search(str2);
        if ((s1 && s2) && (s1 != s2)) {   // found and distinct
            // Step 2: fix head if either node is the current head
            if (head == s1)      head = s2;
            else if (head == s2) head = s1;
            // Step 3: swap the two nodes' prev pointers
            SongNode* temp = s1->getPrev();
            s1->setPrev(s2->getPrev());
            s2->setPrev(temp);
            // Step 4: predecessors' next
            // must point to the new successor
            if (s1->getPrev()) s1->getPrev()->setNext(s1);
            if (s2->getPrev()) s2->getPrev()->setNext(s2);
            // Step 5: swap the two nodes' next pointers
            temp = s1->getNext();
            s1->setNext(s2->getNext());
            s2->setNext(temp);
            // Step 6: successors' prev
            // must point back to the new predecessor
            if (s1->getNext()) s1->getNext()->setPrev(s1);
            if (s2->getNext()) s2->getNext()->setPrev(s2);
            return true;
        }
    }
    return false;   // empty list / not found / same node
}
```

**Why the order matters.** Steps 3–4 and 5–6 form two pairs: once the `prev` pointers are swapped (Step 3), `s1->getPrev()` is the new predecessor, which is exactly the pointer Step 4 needs to fix; likewise Step 6 must run after Step 5. (Step 5 may run before Step 3.) Defining a new `SongNode` object, or swapping only the `title`/`length` data, earns zero marks here.

## 1.5 const-ness

### 1.5.1 `const` Basics

```
const float PI = 3.1416;   // C++ style: typed, scoped
#define PI 3.1416           // C style: textual substitution
```

**Q: Shortcomings of `#define` vs. `const`?**

**A:**
1. **No type:** raw text replacement; no type-checking.
2. **No scope:** global; ignores namespaces / blocks / class scope.
3. **Undebuggable:** debugger sees 3.1416, not the name `PI`.
4. **No address:** you can't take `&PI` (no storage location).

**Good-practice mantra:** *If something shouldn't change, make it* ***`const`****.* `const` catches mistakes at compile time.

### 1.5.2 const Objects + const Member Functions

```
class Date                    // BUGGY version
{
    int year, month, day;
public:
    Date(int y, int m, int d) { year=y; month=m; day=d; }
    void add_month() { month += 1; }
    int  difference(const Date& d)   { /* ... */ }
    void print()   { cout << year << "/" << month << "/" << day; }
};

int main() {
    const Date WW2(1945, 9, 2);
    WW2.print();              // ERROR: print isn't const
    WW2.add_month();          // ERROR: add_month isn't const
                              // (correct!)
}
```

**Q: What problems are in the code above?**

**A:**
1. `WW2` is `const` but calls non-const `print()` and `add_month()`. Only **const member functions** may be called on a const object.
2. `difference()` doesn't modify `*this` and should be `const`.
3. `add_month()` on `WW2` is correctly rejected (WW2 shouldn't change).

```
class Date
{
    int year, month, day;
public:
    Date(int y, int m, int d) : year(y), month(m), day(d) {}
    void add_month() { month += 1; }                // mutating
    int  difference(const Date& d) const { /*...*/ } // const!
    void print() const                               // const!
        { cout << year << "/" << month << "/" << day; }
};
```

**const Member Functions Under the Hood**

The `this` pointer becomes **pointer-to-const**:

```
/* source  */   void Date::print() const
/* compiled*/   void Date::print(const Date *const this)
```

Any assignment to a data member inside the function becomes a compile error.

**Call-ability rules (memorise these):**

| | call const MF | call non-const MF |
|---|---|---|
| const object | yes | no |
| non-const object | yes | yes |

**2024S Midterm, Problem 1(a) [2 pts] — const object, non-const member function**

**Problem.** What is the output?

```
#include <iostream>
using namespace std;
class A {
public:
    A(int n): x(n) {}
    void print() { cout << x << endl; }
private:
    int x;
};
void fun(const A obj) {
    obj.print();
}
int main() {
    A objA {10};
    fun(objA);
    return 0;
}
```

**Answer.** COMPILATION ERROR
**Solution.** The parameter is `const A obj`, so inside `fun` the object is **const**. Per the call-ability table above, a const object may call only const member functions — but `print()` is non-const (its implicit `this` is `A* const`, not `const A* const`). `obj.print()` is therefore a compile error. Fix: declare `void print() const`.

**2025S Midterm, Problem 1(b) [2 pts] — calling through a `const Object*`**

**Problem.** What is the output?

```
#include <iostream>
using namespace std;
class Object {
private:
    int data;
public:
    Object(int n) { data = n; }
    int get() { return data; }
};
const Object* fun(Object* obj) {
    return obj;
}
int main() {
    Object obj(20);
    fun(&obj)->get();
    return 0;
}
```

**Answer.** COMPILATION ERROR
**Solution.** Two `const` facts are being tested:

- `return obj;` is **fine**: converting `Object*` to `const Object*` only adds const, which is always allowed.
- `fun(&obj)->get();` is the error: `fun` returns a `const Object*`, and `get()` is non-const. Calling a non-const member function through a pointer-to-const is forbidden (same table as above).

Fix: make `get()` const (`int get() const`).

### 1.5.3 const Pointers: Four Combinations

**Reading pointer declarations**

**Rule:** `const` to the left of `*` ⇒ **pointee** is const.
`const` to the right of `*` ⇒ **pointer itself** is const.

**Trick:** Read the declaration from **right to left**.

```
char c = 'Y';
char *const cpc      = &c;      // const pointer TO       char
char const* pcc;                // pointer TO       const char
const char* pcc2;               // same as above
const char *const cpcc = &c;    // const pointer TO const char



      char s[] = "COMP2012";
const char* pcc {s};            // pointer to const char
pcc[5] = '5';                   // ERROR: can't write via pcc
pcc    = p;                     // OK: pointer itself is not const

char *const cpc = s;            // const pointer
cpc[5] = '5';                   // OK: pointee is writable
cpc    = p;                     // ERROR: pointer itself is fixed

const char *const cpcc = s;     // both const
cpcc[5] = '5';                  // ERROR
cpcc    = p;                    // ERROR
```

**Important subtlety:** A **pointer-to-const** on a non-const object is just a "read-only view":

```
int i = 151;
i += 20;                        // OK (i itself is non-const)
      int*  pi  = &i;
*pi += 20;                      // OK
const int*  pic = &i;
*pic += 20;                     // ERROR: can't write through pic
pi = pic;                       // ERROR: cannot drop const implicitly
pic = pi;                       // OK: adding const is always fine
```

**2025S Midterm, Problem 1(d) [2 pts] — pointer to a const pointer**

**Problem.** What is the output?

```
#include <iostream>
using namespace std;
class Thing {
private:
    int data = 10;
public:
    void set(int n) { data = n; }
    int get() const { return data; }
```

```
};
void mystery(Thing* const* p) {
    (*p)->set(100);
}
int main() {
    Thing* thing = new Thing;
    mystery(&thing);
    cout << thing->get() << endl;
    return 0;
}
```

**Answer.** 100
**Solution.** Read `Thing* const* p` from right to left: "pointer (`*`) to a `const` pointer (`Thing* const`) to a `Thing`". So:

- `p` is `&thing`: `*p` is the pointer `thing` itself, and that pointer is **const** — `*p = ...;` (re-pointing `thing`) would be a compile error.
- But the `Thing` object that `thing` points to is **not** const, so `(*p)->set(100);` is legal: it calls `set` on the heap object.

Hence `thing->get()` prints `100`. Constness of the pointer never makes the pointee const (contrast `const Thing**`).

### 1.5.4 const References as Function Parameters

**Q: Why pass by reference?**

**A:**
1. **Efficiency:** avoid copying big objects (arrays, classes with heap data).
2. **Out-parameters:** let the function mutate a caller-side object without (ugly) pointers.

If you use a reference but **don't** want mutation, always add `const`:

```
void cbr (int& x)       { x += 10; }   // OK: intended mutation
void cbcr(const int& x) { x += 10; }   // COMPILE ERROR: caught!
```

```
void cbr (int& a);
void cbcr(const int& a);

int x {50};
const int y {100};

cbr (x);          // OK
cbcr(x);          // OK

cbr (y);          // ERROR: const int -> int& not allowed
cbcr(y);          // OK

cbr (1234);       // ERROR: literal to non-const lvalue ref
cbcr(1234);       // OK: const ref extends the temporary
```

**Parameter-acceptance rules (memorise these):**

| | accepts const & / const * | accepts non-const & / non-const * |
|---|---|---|
| literal / rvalue | yes | no |
| const object | yes | no |
| non-const object | yes | yes |

### 1.5.5 Const Summary — Good Practice

✓ Objects that shouldn't change ⇒ **const objects**

```
const double PI = 3.1415927;
const Date handover(1, 7, 1997);
```

✓ Function params you won't mutate ⇒ **const references**

```
void print_height(const LargeObj& o) { o.print(); }
```

✓ Member functions that don't mutate `*this` ⇒ **const MFs**

```
int Date::get_day() const { return day; }
```

### 1.5.6 Lambda Expressions (Brief Recap)

A **lambda expression** creates an unnamed function object on the fly. The **capture list** `[...]` decides, for each outer variable used, whether the closure stores a copy or an alias:

| | |
|---|---|
| `[a]` | capture `a` **by value** (a copy stored inside the closure) |
| `[&a]` | capture `a` **by reference** (an alias to the caller's variable) |
| `[=]` / `[&]` | capture all used variables by value / by reference |
| `mutable` | allow the body to modify the by-value copies |

**Lambda capture rules for exams**

- By-value captures are **frozen when the closure object is created** — i.e. when the lambda expression is evaluated (the `auto f = [...]...;` line), not when the lambda is later called.
- Assigning to a by-value captured variable inside a `mutable` lambda only changes the **closure's own copy**; the outer variable is untouched.
- Assigning through a by-reference capture changes the **outer** variable.
- A nested lambda captures from its enclosing scope; if the enclosing lambda captured by value, the nested lambda sees the enclosing closure's **copies**.

**2024S Midterm, Problem 2 [9 pts] — nested lambdas and capture modes**

**Problem.** What is the output of the program below?

```
#include <iostream>
using namespace std;
int main()
{
    int a = 1, b = 1, c = 1;
    auto f1 = [a, &b, &c] () mutable {
        cout << a << " " << b << " " << c << '\n';
        auto f2 = [a, b, &c] () mutable {
            cout << a << " " << b << " " << c << '\n';
            a = b = c = 4;
        };
        a = b = c = 3;
        f2();
    };
    a = b = c = 2;
    f1();
    cout << a << " " << b << " " << c << '\n';
}
```

**Answer.**

```
1 2 2
1 2 3
2 3 4
```

**Solution.** Trace the captures carefully:

- `f1` captures `a` **by value** (frozen at the definition line, when `a == 1`) and `b`, `c` **by reference**. Just before the call, `main` sets `a=b=c=2`.
- **Line 1:** inside `f1`, `a` is still its frozen copy (`1`) while `b`, `c` read the outer variables (2, 2) ⇒ `1 2 2`.
- The closure `f2` is constructed at its own definition line, which runs before `a = b = c = 3;` executes. So `f2` freezes `a` (the enclosing `f1` copy, value `1`) and `b` (the outer `b`, value 2), and captures `c` **by reference**.

- Then `a = b = c = 3;` sets `f1`'s copy of `a` to 3 and the outer `b`, `c` to 3.
- **Line 2:** `f2` prints `a`=1 (frozen), `b`=2 (frozen), `c`=3 (by reference) ⇒ `1 2 3`. Its body `a = b = c = 4` modifies only `f2`'s own copies of `a` and `b`, but `c` is the outer `c` ⇒ outer `c` becomes `4`.
- **Line 3:** back in `main`: `a` was last set to 2, `b` was set to 3 by `f1`, `c` was set to 4 through `f2`'s reference ⇒ `2 3 4`.

# 2 Constructors, Destructor

## 2.1 Object Initialization: Constructors

### 2.1.1 Aggregate (public-member) Initialization

If all data members are `public` (so the class is effectively a `struct`), you can brace-initialize directly:

```
class Word                          /* File: public-member-init.cpp */
{
   public:
     int frequency;
     const char* str;
};

int main() { Word movie = {1, "Titanic"}; }
```

**Private members block brace-init**

If any data member is private, aggregate initialization is disabled:

```
class Word                          /* File: private-member-init.cpp */
{
   public:
     int frequency;
   private:
     char* str;
};

int main() { Word movie = {1, "Titanic"}; }   // compile error!
```

Solution: use a **class constructor** (see below).

### 2.1.2 Constructor Syntax

A **constructor** is a special member function with:

- the **same name as the class**;
- no return type (not even `void`);
- automatically called every time an object is created:
  - on object declaration
  - when an object is passed *by value* to a function
  - when an object is returned *by value* from a function

```
Word movie;                         // default constructor
Word director = "J. Cameron";       // implicit conversion constructor
Word sci_fi("Avatar");              // explicit conversion constructor
Word drama {"Titanic"};             // C++11 explicit conversion ctor
```

```
Word* p = new Word("action", 1);  // general constructor
```

**Four kinds of constructors (analogy: iPhone factory)**

| | | |
|---|---|---|
| Default constructor | `Word()` | blank iPhone |
| Conversion constructor | `Word(const char*) / Word(char)` | Android → iPhone |
| Copy constructor | `Word(const Word&)` | old iPhone → new iPhone |
| General / other constructor | `Word(const char*, int)` | custom iPhone |

### 2.1.3 Default Initializers for Non-static Data Members (C++17)

```
class Word               /* File: default-initializer.cpp */
{   // Implicitly private members
    int frequency {0};
    const char* str {nullptr};
};

int main() { Word movie; }
```

Nevertheless, C++ supports a more general mechanism for user-defined initialization of class objects through constructor member functions. During the construction of a non-global object, if its constructor does not initialize a non-static member, it will have the value of its default initializer if it exists, otherwise its value is undefined.

### 2.1.4 Default Constructor

The default constructor of class `X` is any constructor callable with 0 arguments.

```
class Word                /* File: default-constructor.cpp */
{
   private:
     int frequency;
     char* str;
   public:
     Word() { frequency = 0; str = nullptr; } // Default constructor
};

int main()
{
    Word movie; // No arguments => expect default constructor
}
```

Cf. Variable definition of basic data types: `int x; float y;`. It is used to create objects with user-defined default values.

#### Compiler-Generated Default Constructor

If you write no user-defined constructor at all, the compiler automatically generates a trivial one for you:

```
class Word    /* File: compiler-default-constructor.cpp */
{   // Implicitly private members
    int frequency;
    char* str;
};

int main() { Word movie; }
```

If there are no user-defined constructors in the definition of class `X`, the compiler will generate the following default constructor for it:

```
X::X() { }
```

`Word::Word() { }` only creates a `Word` object with enough space for its `int` component and `char*` component. The initial values of the data members cannot be trusted.

**Common default-constructor bug**

Only when there are NO user-defined constructors at all, does the compiler supply `X::X(){}`. If you add **any** constructor, you lose the auto-generated default constructor:

```
class Word                /* File: default-constructor-bug.cpp */
{
  private: int frequency; char* str;
  public: Word(const char* s, int k = 0);
};

int main() { Word movie; }                  // which constructor?
```

Compilation:

```
error: no matching function for call to Word::Word()
note: candidate: Word::Word(const char*, int)
  candidate expects 2 arguments, 0 provided
```

**Fix:** either write your own `Word()`, or use `= default` (see later).

### 2.1.5 Conversion Constructors

A constructor accepting a single argument specifies a conversion from its argument type to the type of its class.

```
#include <cstring>     /* File: implicit-conversion-constructor.cpp */
class Word
{
  private: int frequency; char* str;
  public:
    Word(char c)
        { frequency = 1; str = new char[2];
          str[0] = c; str[1] = '\0'; }
    Word(const char* s) // Assumption: s != nullptr
        { frequency = 1; str = new char [strlen(s)+1];
          strcpy(str, s); }
};

int main()
{
    Word movie("Titanic");                  // Explicit conversion
    Word movie2 {'A'};                      // Explicit conversion
    Word movie3 = 'B';                      // Implicit conversion
    Word director = "James Cameron";        // Implicit conversion
}
```

A constructor with **many arguments** still works as a conversion constructor if all but one have default values:

```
#include <cstring>      // conversion-constructor-default-arg.cpp
```

```
class Word
{
    int frequency; char* str;
  public:
    Word(const char* s, int k = 1)
    // Can work like a conversion constructor!
    {
        frequency = k;
        str = new char [strlen(s)+1]; strcpy(str, s);
    }
};

int main()
{
    Word *p = new Word {"action"};   // Explicit conversion
    Word movie("Titanic");           // Explicit conversion
    Word director = "James Cameron"; // Implicit conversion
}
```

A class may have more than one conversion constructor.

### Surprise Implicit Conversion

```
#include <iostream>     /* File: implicit-conversion-surprise.cpp */
#include <cstring>
using namespace std;
class Word
{
  private:
    int frequency; char* str;
  public:
    Word(char c)
        { frequency = 1; str = new char[2]; str[0] = c; str[1] = '\0';
          cout << "call char conversion\n"; }
    Word(const char* s)
        { frequency = 1; str = new char [strlen(s)+1];
          strcpy(str, s);
          cout << "call const char* conversion\n"; }
    void print() const { cout << str << " : " << frequency << endl; }
};

void print_word(Word x) { x.print(); }
int main() { print_word("Titanic"); print_word('A'); return 0; }
```

Sometimes this is undesirable! It compiles when you didn't intend a conversion.

**`explicit` keyword: disallow implicit conversion**

Prefix a conversion constructor with `explicit` to forbid the surprise:

```
#include <cstring> // explicit-conversion-constructor.cpp
class Word
{
  private:
    int frequency; char* str;
  public:
    explicit Word(const char* s)
```

```
        { frequency = 1; str = new char [strlen(s)+1];
          strcpy(str,s); }
};

int main()
{
    Word *p = new Word("action");    // Explicit conversion
    Word movie("Titanic");           // Explicit conversion
    Word director = "James Cameron"; // Bug: implicit conversion
}
```

Compilation:

```
error: conversion from const char [14] to non-scalar type Word
   requested
   Word director = "James Cameron"; // Bug: implicit conversion
                   ^~~~~~~~~~~~~~~
```

### 2.1.6 Copy Constructor

Signature for class `X`: `X(const X& other)`

**Exactly one argument:** an object of the same class, passed by const reference.

It is called in 3 situations:

1. object initialization using =: `Word y = x;`
2. object passed **by value** to a function
3. object returned **by value** from a function

```
#include <iostream>      /* File: copy-constructor.cpp */
#include <cstring>
using namespace std;

class Word
{
  private:
    int frequency; char* str;
    void set(int f, const char* s)
        { frequency = f; str = new char [strlen(s)+1];
          strcpy(str,s); }
  public:
    Word(const char* s, int k = 1)
        { set(k, s); cout << "conversion\n"; }
    Word(const Word& w)
        { set(w.frequency, w.str); cout << "copy\n"; }
};

int main()
{
    Word movie("Titanic");        // Conversion constructor
    Word song(movie);             // Copy constructor
    Word ship = movie;            // Copy constructor
}
```

**Return-by-Value Triggers Copy Constructor (in theory)**

```
#include <iostream>     /* File: return-by-value.cpp */
#include <cstring>
using namespace std;
class Word
{
  private:
    int frequency; char* str;
    void set(int f, const char* s)
        { frequency = f; str = new char [strlen(s)+1];
          strcpy(str, s); }
  public:
    Word(const char* s, int k = 1)
        { set(k, s); cout << "conversion\n"; }
    Word(const Word& w)
        { set(w.frequency, w.str); cout << "copy\n"; }
    void print() const { cout << str << " : " << frequency << endl; }
    Word to_upper_case() const
    {
        Word x(*this);
        for (char* p = x.str; *p != '\0'; p++) *p += 'A' - 'a';
        return x;
    }
};
int main()
{
    Word movie {"titanic"}; movie.print();
    Word song = movie.to_upper_case(); song.print();
}
```

**Question:** How many calls of the copy constructor do you expect?

```
conversion
titanic : 1
copy
TITANIC : 1
```

Only 1 copy! Return value optimization (RVO) is a compiler optimization technique which applies copy elision in a return statement. It omits copy/move operation by constructing the required object directly into the receiving variable. For the example, codes that are supposed to be run by the temporary are run directly on `song`.

**Which line actually called the copy constructor?**

**Answer:** the line `Word x(*this);` — the explicit local copy. RVO merged the return + the `song` initialization into zero copies.

### Named and Unnamed RVO

```
#include <iostream>      /* File: unrvo-nrvo.cpp */
using namespace std;
#include <cstring>

class Word
{
```

```
  private:
    int frequency; char* str;
    void set(int f, const char* s)
        { frequency = f; str = new char [strlen(s)+1];
          strcpy(str, s); }
  public:
    Word(const char* s, int k = 1)
        { set(k, s); cout << "conversion" << endl; }
    Word(const Word& w)
        { set(w.frequency, w.str); cout << "copy" << endl; }
};

Word unnamed_return_word(const char* s) { return Word {s}; }
Word named_return_word(const char* s) { Word w {s}; return w; }

int main()
{
    Word w1 = unnamed_return_word("titanic");
    cout << "------------" << endl;
    Word w2 = named_return_word("superman");
    return 0;
}
```

**C++11 vs. C++17 RVO differences**

- **Unnamed RVO (URVO):** when a temporary nameless object is returned.
  - In C++11, URVO is optional.
  - Since C++17, URVO is mandatory even if `-fno-elide-constructors` is turned on.
- **Named RVO (NRVO):** when a named local object is returned.
  - NRVO is optional depending on whether `-fno-elide-constructors` is turned on.
  - The result is further complicated if there is a move constructor.

Force-disable it with `-fno-elide-constructors` to see the naive behaviour.

### 2.1.7 Default Copy Constructor & Default Assignment Operator

```
class Word /* File: default-copy-constructor.cpp */
{
  private: ...
  public: Word(const char* s, int k = 0) { ... };
};
int main()
{
    Word movie {"Titanic"}; // which constructor?
    Word song {movie};       // which constructor?
    Word song = movie;       // which constructor?
}
```

If no copy constructor is defined for a class, the compiler will automatically supply it a default copy constructor:

```
X(const X&) { /* memberwise copy */ }
```

Memberwise copy by calling the copy constructor of each data member:

– copy `movie.frequency` to `song.frequency`

– copy `movie.str` to `song.str`

It works even for array members by copying each array element.

Unless you re-define the assignment operator `=` for a class, the compiler generates the default assignment operator function "memberwise assignment" for it. Different from the default copy constructor, the default assignment operator `=` will perform memberwise assignment by calling the assignment operator `=` of each data member. Again for array members, each array element is assigned. Memberwise assignment/copy is usually not what you want when memory allocation is required for the class members.

**Shallow copy (memberwise copy on pointer members) is FATAL**

If a class has **pointer members to heap memory**, the default copy constructor / default `=` only copies the **pointer value** (address), not the pointed-to data. This causes:

1. **Double-free bug:** two objects share the same `str`; both destructors try to `delete[]` it.
2. **Memory leak:** assignment `z = x` overwrites `z.str` without freeing its old buffer.
3. **Aliasing bug:** mutating one object silently mutates the other.

Fix: write your own copy constructor, copy-assignment operator, and destructor (see "Rule of Three/Big Five", later).

```
#include <iostream>     /* File: default-assign-problem1.cpp */
#include <cstring>
using namespace std;
class Word
{
  private:
    int frequency; char str[100];
    void set(int f, const char* s) { frequency = f; strcpy(str, s); }
  public:
    Word(const char* s, int k = 1)
        { set(k, s);
          cout << "\nMay work as const char* conversion\n"; }
    Word(const Word& w)
        { set(w.frequency, w.str); cout << "\nCopy\n"; }

     void print() const
     // Also prints the address of object's str array
         { cout << str << " : " << frequency << " ; "
                << reinterpret_cast<const void*>(str) << endl; }
};

int main()
{
    Word x("rat"); x.print();          // Conversion constructor
    Word y = x;    y.print();          // Copy constructor
    Word z("cat"); z.print();          // Conversion constructor
    z = x;         z.print();          // Default assignment operator
}
```

Here default copy/assign actually works correctly! (Copies the array contents element-by-element.) The downside is fixed 100-char limit.

```
#include <iostream>     /* File: default-assign-problem2.cpp */
#include <cstring>
using namespace std;
class Word
{
  private: int frequency; char* str;
    void set(int f, const char* s)
        { frequency = f; str = new char [strlen(s)+1];
          strcpy(str, s); }
  public:
    Word(const char* s, int k = 1)
        { set(k, s);
          cout << "\nMay work as const char* conversion\n"; }
    Word(const Word& w)
        { set(w.frequency, w.str); cout << "\nCopy\n"; }

     void print() const
     // Also prints the address of object's str array
         { cout << str << " : " << frequency << " ; "
                << reinterpret_cast<void*>(str) << endl; }
};

int main()
{
    Word x("rat");    x.print(); // Conversion constructor
    Word y = x;       y.print(); // Copy constructor
    Word z("cat", 2); z.print(); // Conversion constructor
    z = x;            z.print(); // Default assignment operator
}
```

After default `z = x`: `z.str == x.str` (same heap block!). Originally-allocated "cat" is leaked; program later double-frees "rat".

**Quiz: Constructors**

Which constructor is called in the following statements?

```
1  Word nothing;
2  Word dream_grade('A');
3  Word major {"COMP"};
4  Word hkust = "hkust";
5  Word exchange_to(hkust);
6  Word grade = dream_grade;
7  Word grade2 {dream_grade};
```

**Answers:**

1. Default constructor (if available; otherwise compile error)
2. Explicit `char` conversion constructor
3. Explicit `const char*` conversion constructor (C++11 {})
4. Implicit `const char*` conversion constructor (unless `explicit`)
5. Copy constructor (from `hkust`)
6. Copy constructor (`=` with same class is still copy constructor, NOT assignment operator!)
7. Copy constructor (C++11 {} syntax)

**Memorize:** `Type x = y;` when `x` and `y` are same class = copy constructor. The `=` here is **not** the copy-assignment operator!

### 2.1.8 Uniform Initialization Using {} Initializers Again

In general, initializations may be done using ( ), =, or {}:

```
int x(1); int y = 2; int z {3};
```

The braced initialization syntax helps avoid some misleading syntax from the other two kinds:
(1) When = doesn't really mean assignment:

```
Word word1 = word2; // What is this?
```

(2) When ( ) doesn't really mean calling the default constructor:

```
Word w(); // What is this?
```

In both cases, braced initialization works fine:

```
Word word1 { word2 }; Word w {};
```

When a class member of user-defined types is initialized, its corresponding constructor will be called. ( ) initializer cannot be used to do default initialization of non-static class data members.

### 2.1.9 Constructors and Function Overloading

Overloading allows programmers to use the same name for functions that do similar things but with different input arguments. Constructors are often overloaded.

```
class Word                          /* File: overload-constructor.cpp */
{
  private:
    int frequency;
    char* str;

     public:
       Word();                              // Default constructor
       Word(const char* s, int k = 1); // Maybe conversion constructor
       Word(const Word& w);                 // Copy constructor
};
```

**Review: Function Overloading**

In general, function names can be overloaded in C++. Actually, operators are often overloaded. E.g., what is the type of the operands for +?

```
#include <iostream>      /* File: overload-function.cpp */
#include <cstring>
using namespace std;
class Word
{
  private:
    int frequency; char* str;
  public:
    void set() const { cout << "Input the string: ";
                       cin >> str; } // Error!
    void set(int k) { frequency = k; }
```

```
    void set(char c)
        { str = new char [2]; str[0] = c; str[1] = '\0'; }
    void set(const char* s)
        { str = new char [strlen(s)+1]; strcpy(str, s); }
};

int main()
{
    Word movie;           // Which constructor?
    movie.set();          // Which set function?
}
```

### 2.1.10 Default Arguments (Reminder)

If a function shows some default behaviors most of the time, and some exceptional behaviors only once awhile, specifying default arguments is a better option than using overloading. There may be more than one default argument:

```
void upload(char* prog, char os = LINUX, char format = TEXT);
```

**2 default-argument rules**

1. Parameters **without** defaults must appear **to the left** of those with defaults. **Wrong:** `void upload(char os = LINUX, char* prog, ...);`
2. A default argument can only be written **once** per translation unit, and only in the declaration (header), not in the definition (`.cpp`). **Wrong:**

```
// word.h
class Word { public: Word(const char* s, int k = 1); };
// word.cpp
Word::Word(const char* s, int k = 1) { ... }
// ERROR: duplicate default
```

## 2.2 Member Initializer List (MIL)

### 2.2.1 MIL Syntax

Instead of assigning members inside the constructor body, prefer initialising them **before the body** using the **member initializer list** ("colon syntax"):

```
class Word                          /* File: mil-word.h */
{
  private:
    char lang;
    int freq;
    char* str;

     public:
       Word() : lang('E'), freq(0), str(nullptr) { };

       /* Or, using the braced initialization syntax as follows
          Word() : lang{'E'}, freq{0}, str{nullptr} { };
       */

       Word(const char* s, int f = 1, char g = 'E') : lang(g), freq(f)
           { str = new char [strlen(s)+1]; strcpy(str, s); }

       void print() const { cout << str << " : " << freq << endl; }
};
```

**Why MIL instead of body-assignment?**

Since the MIL calls the constructors of the data member, it works well for data members of user-defined types. Thus, it is better to perform initialization by MIL than by assignments inside constructors. Make sure that the corresponding member constructors exist!

**Important MIL gotcha**

The **order of members in the MIL does NOT matter**. Actual initialization order is always the order of member declaration in the class body.

**2024S Midterm, Problem 1(c) [2 pts] — constructor body vs. the member**

**Problem.** What is the output?

```
#include <iostream>
using namespace std;
class B {
public:
    int b = 0;
    B(int b): b(b) { b = 2*b; }
};
int main() {
    B obj(10);
    cout << obj.b << endl;
    return 0;
```

```
}
```

**Answer.** 10
**Solution.** In the MIL, `b(b)` initialises the **member** `b` with the **parameter** `b` (= 10). But inside the constructor body, the name `b` refers to the **parameter** (the innermost declaration wins — same shadowing trap as `X(int size) { size = size; }`). So `b = 2*b;` doubles the parameter (a value that is about to die), never the member. The member keeps 10.

### 2.2.2 MIL for User-Defined-Type Members

`Word_Pair` *contains* two `Word` objects as members. MIL allows passing arguments to `Word`'s constructor:

```
class Word_Pair           /* File: mil-word-pair.h */
{
   private:
     Word w1; Word w2;
   public:
     Word_Pair(const char* s1, const char* s2) : w1(s1,5), w2(s2) { }
     void print() const
     {
         cout << "word1 = "; w1.print();
         cout << "word2 = "; w2.print();
     }
};
```

**MIL-less alternative is error-prone**

```
class Word_Pair /* File: member-class-init-by-mil.h */
{
  private:
    Word w1; Word w2;
  public:
    Word_Pair(const char* s1, const char* s2) : w1(s1,5), w2(s2) { }
};
```

⇒ `w1` and `w2` are initialized using the conversion constructor, `Word(const char*, int = 1, char = 'E')`.
Compare with the MIL-less version:

```
Word_Pair(const char* x, const char* y) { w1 = x; w2 = y; }
```

⇒ error-prone because `w1` and `w2` are initialized by assignment. If the assignment operator function is not appropriately defined, the default memberwise assignment may not be good enough.

### 2.2.3 `const` or Reference Members Must Use MIL

`const` and reference members must be initialized using member initializer list if they don't have default initializers. Cf. `float y; float& z = y; const int x = 123;`

```
#include <iostream>       /* File: mil-const-ref.cpp */
using namespace std;
int a = 5;

class Example
```

```
{
    const int const_m = 3;
    int& ref_m = a;
  public:
    Example() { }
    Example(int c, int& r) : const_m(c), ref_m(r) { }
    void print() const { cout << const_m << "\t" << ref_m << endl; }
};

int main()
{
    Example x; x.print();
    int b = 55; Example y(10, b); y.print();
}
```

### Forgetting MIL for a const member

It cannot be done using default arguments:

```
#include <iostream>     /* File: mil-const-member-error.cpp */
using namespace std;
class Word
{
  private:
    const char lang; int freq; char* str;
  public:
    Word() : lang('E'), freq(0), str(nullptr) { };
    Word(const char* s, int f = 1, char g = 'E')
        { str = new char [strlen(s)+1]; strcpy(str, s); }
    void print() const
        { cout << str << " : " << freq << endl; }
};

int main() { Word x("hkust"); }
```

Compilation:

```
error: constructor for Word must explicitly initialize
  the const member 'lang'
   Word(const char* s, int f = 1, char g = 'E')
   ^
```

**Fix:** Add : `lang(g), freq(f)` to the MIL.

### 2025S Midterm, Problem 1(a) [2 pts] — const member must use the MIL

**Problem.** What is the output?

```
#include <iostream>
using namespace std;
class Item {
private:
    int value;
    const int cap;
public:
    Item(int x = 10, int y = 100) { value = x; cap = y; }
    void update(int x)
        { value = ((value+x) > max) ? max : (value+x); }
    void print() const
```

```
        { cout << value << " (" << cap << ")" << endl; }
};
int main() {
    Item item(50, 80);
    item.update(20);
    item.print();
    return 0;
}
```

**Answer.** COMPILATION ERROR
**Solution.** Two independent errors, either of which fails the compilation:

- `cap` is a `const` member. A const member cannot be assigned to in the constructor body — by the time the body runs, `cap` is already initialised (to garbage here) and frozen. The only way to set it is the MIL: `Item(int x = 10, int y = 100) : value(x), cap(y) {}`. The code above writes `cap = y;` in the body ⇒ error.
- `max` in `update` is undeclared (there is no member, global, or two-argument `std::max` call here) ⇒ error.

**Rule:** `const` members and reference members must be initialised in the MIL.

### 2.2.4 Delegating Constructor vs. Private Utility Function

```
#include <iostream>        /* File: copy-constructor2.cpp */
#include <cstring>
using namespace std;

class Word
{
  private:
    int frequency; char* str;
    void set(int f, const char* s) // Private utility function
        { frequency = f; str = new char [strlen(s)+1];
          strcpy(str,s); }
  public:
    Word(const char* s, int k = 1)
        { set(k, s); cout << "conversion\n"; }
    Word(const Word& w)
        { set(w.frequency, w.str); cout << "copy\n"; }
};
```

In this previous example, since most of the code of the conversion and copy constructors are similar, they are defined with a private utility function `set()`. May we achieve similar result without defining the latter?

### 2.2.5 Example: Delegating Constructor (C++11)

Instead of duplicating code between constructors with a private helper function, let one constructor delegate to another constructor using MIL syntax:

```
#include <iostream>      /* File: delegating-constructor.cpp */
#include <cstring>
using namespace std;

class Word              // Modified from copy-constructor.cpp
{
```

```
  private:
    int frequency; char* str;
  public:
    Word(const char* s, int f = 1)
    {
        frequency = f; str = new char [strlen(s)+1]; strcpy(str, s);
        cout << "conversion" << endl;
    }
    Word(const Word& w) : Word(w.str, w.frequency)
        { cout << "copy" << endl; }
    void print() const { cout << str << " : " << frequency << endl; }
};

int main()
{
    Word movie("Titanic"); movie.print(); // which constructor?
    Word song(movie); song.print();       // which constructor?
    Word ship = movie; ship.print();      // which constructor?
}
```

In this example, the copy constructor, using the member initializer list syntax, delegates the conversion constructor to create an object. The copy constructor is now a delegating constructor.

**Delegation restriction**

Restriction: the delegated constructor must be the only item in the MIL. You cannot mix delegation with other member initializers.

**2024S Midterm, Problem 4 [20 pts] — debugging (I): constructors & objects**

**Problem.** The program below contains **10 errors** after line 18 (compilation errors or runtime errors, including memory leaks). Identify each error by its line number and explain it. Errors are considered independently (assume the others are fixed).

```
1  #include <iostream>
2  #include <ctime>
3  #include <cstring>
4  using namespace std;
5
6  const int MAX_SIZE = 10;
7  const int MAX_STR_LEN = 100;
8
9  class Time {
10   private:
11     int hh, mm;
12   public:
13     Time() = delete;
14     explicit Time(int t): hh(t/60), mm(t%60) {}
15     void print() const {
16       cout << ((hh < 10) ? "0" : "") << hh << ":"
17            << ((mm < 10) ? "0" : "") << mm;
18     }
19 };
20
21 class Object {
22   private:
```

```
23      char name[MAX_STR_LEN];
24      int x, y;
25    public:
26      Object(const char* s, int n = 0, int m = 0): x{n}, y{m}
27          { strcpy(name, s); }
28      ~Object() = default;
29      const char* get_name() const { return name; }
30      int get_x() const { return x; }
31      int get_y() const { return y; }
32      virtual void display() const = 0;
33  };
34
35  class Button: public Object {
36    private:
37      char text[MAX_STR_LEN];
38      Time trigger_time = 0;
39    public:
40      Button(const char* s, int n, int m, const char* t):
41          trigger_time(), Object(s, n, m) {
42        strcpy(text, t);
43      }
44      void display() const {
45        cout << name << "[" << text << "]" << " at ("
46             << get_x() << ", " << get_y()
47             << ") is last clicked at ";
48        trigger_time.print();
49        cout << endl;
50      }
51      void trigger(const Time& t) {
52        cout << get_name() << " clicked" << endl;
53        trigger_time = t;
54      }
55
56  };
57
58  class Frame: public Object {
59    private:
60      Object* objs[MAX_SIZE] = {};
61      int num_objs = 0;
62    public:
63      Frame() {}
64      Frame(const char* s, int n, int m)
65          : Frame(), Object(s, n, m) {}
66      void add_objects(Object* obj) {
67        if (num_objs < MAX_SIZE)
68          objs[num_objs++] = obj;
69      }
70      ~Frame() {
71        for (int i=0; i<num_objs; ++i)
72          if (objs[i])
73            delete objs[i];
74      }
75      void display(int i=0) const override {
76        cout << "<--" << endl;
77        cout << get_name() << " at (" << get_x() << ", "
```

```
78              << get_y() << "):" << endl;
79        for (int i=0; i<num_objs; ++i)
80          if (objs[i])
81            objs[i]->display();
82        cout << "-->" << endl;
83      }
84      void trigger(const char* s, const Time& t) {
85        for (int i=0; i<num_objs; ++i)
86          if (objs[i]
87              && (strcmp(objs[i]->get_name(), s) == 0))  {
88            objs[i]->trigger(t);
89            break;
90          }
91      }
92  };
93
94  int main() {
95    int start_time = time(0);
96    // time(0) calls a library function
97    // to return the current time in seconds.
98
99    Frame* root = new Frame("Window", 0, 0);
100   root->add_objects(new Object("Obj1"));
101   root->add_objects(new Button("Btn1", 10, 20, "Confirm"));
102   root->add_objects(new Button("Btn2", 30, 20, "Cancel"));
103
104   Frame* child = new Frame("Frame1", 40, 50);
105   child->add_objects(
106       new Button("Welcome-btn", 10, 10, "Welcome!"));
107   root->add_objects(child);
108
109   char option;
110   cout << "Option: ";
111   while (cin >> option) {
112     if (option == '1')
113       root->trigger("Btn1", time(0)-start_time);
114     else if (option == '2')
115       root->trigger("Btn2", Time(time(0)-start_time));
116     else if (option == '3')
117       child->trigger("Welcome-btn",
118                      Time(time(0)-start_time));
119     else if (option == 'p')
120       root->display();
121     else
122       break;
123     cout << "Option: ";
124   }
125
126   delete root;
127   return 0;
128 }
```

**Solution (the five constructor/object-related errors; the five inheritance-related ones are analysed at the end of the inheritance section).**

#1 **Line 36:** `Time trigger_time = 0;` — a default member initializer. It needs the implicit conversion `int` → `Time` through the conversion constructor, but that

constructor is `explicit`, so the implicit conversion is forbidden.

#2 **Line 38:** the MIL item `trigger_time()` invokes the default constructor of `Time`, which was `= deleted` (line 13).

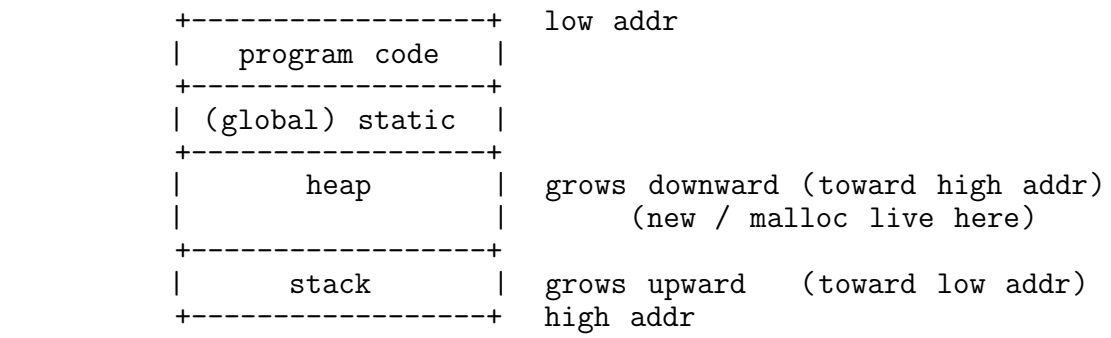
#4 **Line 56:** `Frame() {}` needs the default constructor of the base `Object`. But `Object` has user-defined constructors (with parameters), so the compiler generates no implicit default constructor ⇒ error.

#5 **Line 57:** `Frame(...): Frame(), Object(s, n, m) {}` — a delegating constructor, but the delegated constructor must be the **only** item in the MIL; mixing it with `Object(s,n,m)` is illegal.

#9 **Line 101:** `root->trigger("Btn1", time(0)-start_time);` — the second argument is an `int` and the parameter is `const Time&`, requiring the implicit conversion `int` → `Time`; forbidden because the conversion constructor is `explicit`. (Lines 103–105 are fine: they build `Time(...)` explicitly.)

## 2.3 Destructor

### 2.3.1 Program Memory Layout




```
void f()
{
    // x, y are local variables
    // on the runtime stack
    int x = 4;
    Word y {"Titanic"};
    // p is another local variable
    // on the runtime stack.
    // But the array of 100 int's
    // that p points to
    // is on the heap
    int* p = new int [100];
}
```

**Stack vs. Heap:**

- **Stack:** Local variables are constructed (created) when they are defined in a function/block on the run-time stack. When the function/block terminates, the local variables inside and the call-by-value (CBV) arguments will be destructed (and removed) from the run-time stack. Both construction and destruction of variables are done automatically by the compiler by calling the appropriate constructors and destructors.
- **Heap:** Dynamically allocated memory remains after function/block terminates, and it is the user's responsibility to return it back to the heap for recycling using `delete`; otherwise, it will stay until the program finishes.

**Definitions:**

- **Garbage:** a piece of storage that is part of a running program but there are no more references to it.
- **Memory leak:** program accumulates garbage over time.

### 2.3.2 Destructor Syntax

For class `X`, the destructor is: `X::~X()`

Properties:

- tilde + class name

- **no parameters**, **no return type** (not even `void`)
- therefore, **at most 1 destructor per class** (cannot overload)
- called automatically when object goes out of scope, or when you call `delete` / `delete[]` on a pointer
- if you don't write one, compiler generates empty default: `X::~X() {}`

The destructor itself does not actually release the object's memory. The destructor performs termination housekeeping before the object's memory is reclaimed by the system.

### Default destructor does NOT free heap memory!

```
void Example() /* File: default-destructor-problem.cpp */
{
    Word x("bug", 4);
    ...
}

int main() { Example(); .... }
```

On return from `Example()`, the local `Word` object `x` of `Example()` is destructed from the run-time stack. I.e., the storage of (int) `x.frequency` and (char*) `x.str` are released.
**Question:** How about the memory dynamically allocated for the string, `"bug"` that `x.str` points to?
Answer: It leaks! The default destructor only releases the storage of the data members themselves. It does **not** recursively follow pointers and `delete` heap memory! If a constructor did `new`, your destructor **must** do the matching `delete`.

**User-Defined Destructor:** Usually needed when there are pointer members pointing to memory dynamically allocated by constructor(s) of the class.

```
#include <cstring>      /* File: destructor.cpp */
class Word
{
  private:
    int frequency; char* str;
  public:
    Word() : frequency(0), str(nullptr) { };
    Word(const char* s, int k = 0): frequency(k)
        { str = new char [strlen(s)+1]; strcpy(str, s); }
    ~Word() { delete [] str; }
};
int main()
{
    Word* p = new Word {"Titanic"};
    Word* x = new Word [5];
    delete p;           // Destruct a single object
    delete [] x;        // Destruct an array of objects
}
```

### Matching `new` / `delete`

- `new T` pairs with `delete p`
- `new T[N]` pairs with `delete[] p`

Mix them and you get undefined behaviour.

### 2.3.3 Bug: Default Memberwise Assignment

**How many bugs? (`default-assign-bug.cpp`)**

```
#include <cstring>            /* File: default-assign-bug.cpp */

class Word
{
  private:
    int frequency; char* str;

     public:
       Word() : frequency(0), str(nullptr) { }
       Word(const char* s, int k = 0): frequency(k)
           { str = new char [strlen(s)+1]; strcpy(str, s); }
       ~Word() { delete [] str; }
};

void Bug(Word& x) { Word bug("bug", 4); x = bug; }

int main() { Word movie {"Titanic"}; Bug(movie); return 0; }
```

**Answer: 3 bugs total.**

**Bug (1) Memory leak in `x = bug`:** Default memberwise assign overwrites `x.str` (which pointed to `"Titanic"`) with `bug.str`'s address. The old `"Titanic"` buffer is lost forever (leaked).

**Bug (2) Double aliasing / double-free:** After `x = bug`, `x.str == bug.str` (same pointer). When `Bug()` returns, `bug`'s destructor runs first and `delete[]`s the block; then `main()` ends and `movie`'s destructor tries to `delete[]` the same already-freed block (undefined behaviour — crash or corruption).

**Bug (3) Dangling pointer inside `main`:** After `Bug()` returns, `movie.str` points to freed memory. Any use of it before `main()` ends is use-after-free.

Root cause of all 3: the compiler-generated default copy-assignment operator performs shallow copy. **Fix: write your own copy constructor and copy-assignment operator (Rule of Three).**

### 2.3.4 Summary: 6 Compiler-Generated Member Functions

Unless you define the following, they will be implicitly generated by the compiler for you:

1. Default constructor (but only if you don't define other constructors)
2. Default copy constructor
3. Default (copy) assignment operator function
4. Default move constructor (C++17)
5. Default move assignment operator function (C++17)
6. Default destructor

C++17 allows you to explicitly generate or not generate them:

- to generate: `= default;`
- not to generate: `= delete;`

```
#include <iostream>       /* File: default-delete.cpp */
#include <cstring>
using namespace std;
```

```
class Word
{
   private:
     int frequency {0}; char* str {nullptr};
   public:
     Word() = default; // Still want the simple default constructor
     Word(const Word& w) = delete; // Words can't be copied
     Word(const char* s, int k) : frequency(k)
         { str = new char [strlen(s)+1]; strcpy(str, s); }
     void print() const
         { cout << ((str == nullptr) ? "not-a-word" : str)
                << " : " << frequency << endl; }
};
int main()
{
     Word x; x.print();
     Word y("good", 3); y.print();
     Word z(y);      // Error: call to deleted constructor of Word
}
```

## 2.4 Order of Construction and Destruction

### 2.4.1 "Has" Relationship

When an object A has an object B as a data member, we say "A has a B". It is easy to see which objects have other objects. All you need to do is to look at the class definition.

```
/* File: example-has.h */
class B { ... };

class A
{
  private:
    B my_b;

     public:
     // Declaration of public members or functions
};
```

**Two object-composition relationships**

- **"A Has a B" (contains by value):** `A` has a data member of type `B` directly.
- **"A Owns a B" (points to):** `A` has a `B*` pointer; the actual `B` is allocated on the heap.

Look at the class definition to distinguish.

### 2.4.2 Has-Relationship: Construction/Destruction Order (postoffice1.h)

```
class Clock             /* File: postoffice1.h */
{
  public:
    Clock() { cout << "Clock Constructor\n"; }
    ~Clock() { cout << "Clock Destructor\n"; }
};

class Postoffice
{
    Clock clock;
  public:
    Postoffice() { cout << "Postoffice Constructor\n"; }
    ~Postoffice() { cout << "Postoffice Destructor\n"; }
};
#include <iostream> /* File postoffice.cpp */
using namespace std;

#include "postoffice.h"

int main()
{
    cout << "Beginning of main\n";
    Postoffice x;
    cout << "End of main\n";
}
```

Output:

```
Beginning of main
```

```
Clock Constructor
Postoffice Constructor
End of main
Postoffice Destructor
Clock Destructor
```

When an object is constructed, all its data members are constructed first. The order of destruction is the exact opposite of the order of construction: The Clock constructor is called before the Postoffice constructor code; but, the Clock destructor is called after the Postoffice destructor code. As always, construction of data member objects is done by calling their appropriate constructors. If you do not do this explicitly then their default constructors are assumed. Make sure they exist!

The default constructor `Postoffice::Postoffice() { }` is equivalent to:

```
Postoffice::Postoffice() : clock() { }
```

Or, you may do this explicitly by calling their appropriate constructors using the member initializer list syntax.

If you need a non-default constructor for `clock`, use MIL: `Postoffice() : clock(desired_args) { ... }`.

### 2.4.3 Owns-Relationship: Heap Sub-Object Needs Explicit delete! (postoffice2.h)

```
class Clock             /* File: postoffice2.h */
{
  public:
    Clock() { cout << "Clock Constructor\n"; }
    ~Clock() { cout << "Clock Destructor\n"; }
};

class Postoffice
{
    Clock* clock;
  public:
    Postoffice()
        { clock = new Clock; cout << "Postoffice Constructor\n"; }
    ~Postoffice() { cout << "Postoffice Destructor\n"; }
};
```

Output:

```
Beginning of main
Clock Constructor
Postoffice Constructor
End of main
Postoffice Destructor
```

**Clock Destructor is missing!** Memory leak!

Now the Postoffice "owns" a Clock. This is the terminology used in OOP. If A "owns" B, A only has a pointer pointing to B. The Clock object is constructed in the Postoffice constructor, but it is never destructed, since we have not implemented that. Remember that objects on the heap are never destructed automatically, so we have just created a memory leak. When object A owns object B, A is responsible for B's destruction.

**Correct version (postoffice3.h):**

```
class Clock             /* File: postoffice3.h */
{
```

```
  public:
    Clock() { cout << "Clock Constructor\n"; }
    ~Clock() { cout << "Clock Destructor\n"; }
};

class Postoffice
{
    Clock* clock;
  public:
    Postoffice()
        { clock = new Clock; cout << "Postoffice Constructor\n"; }
    ~Postoffice()
        { cout << "Postoffice Destructor\n"; delete clock; }
};
```

Output:

```
Beginning of main
Clock Constructor
Postoffice Constructor
End of main
Postoffice Destructor
Clock Destructor
```

Now output includes `Clock Destructor` *after* `Postoffice Destructor` line.

### 2.4.4 Two Embedded Members: Room + Clock (postoffice4.h)

```
class Clock         /* File: postoffice4.h */
{
  private: int HHMM;    // hour, minute
  public:
    Clock() : HHMM(0)
        { cout << "Clock Constructor\n"; }
    ~Clock() { cout << "Clock Destructor\n"; }
};
class Room
{
  public:
    Room() { cout << "Room Constructor\n"; }
    ~Room() { cout << "Room Destructor\n"; }
};
class Postoffice
{
  private:
    Room room; Clock clock;
  public:
    Postoffice()
        { cout << "Postoffice Constructor\n"; }
    ~Postoffice()
        { cout << "Postoffice Destructor\n"; }
};
```

⇒ Note that the 2 data members, `Clock` and `Room` are constructed first, in the order that they appear in the `Postoffice` class.

Output:

```
Beginning of main
```

```
Room Constructor
Clock Constructor
Postoffice Constructor
End of main
Postoffice Destructor
Clock Destructor
Room Destructor
```

Because `room` is declared *before* `clock`, it is constructed first. Destructor order is exactly reversed.

### 2.4.5 Nested: Postoffice Has Room, Room Has Clock (postoffice5.h)

```
class Clock         /* File: postoffice5.h */
{
  public:
    Clock() { cout << "Clock Constructor\n"; }
    ~Clock() { cout << "Clock Destructor\n"; }
};
class Room
{
  private:
    Clock clock;
  public:
    Room() { cout << "Room Constructor\n"; }
    ~Room() { cout << "Room Destructor\n"; }
};
class Postoffice
{
  private:
    Room room;
  public:
    Postoffice()
        { cout << "Postoffice Constructor\n"; }
    ~Postoffice()
        { cout << "Postoffice Destructor\n"; }
};
```

Output:

```
Beginning of main
Clock Constructor
Room Constructor
Postoffice Constructor
End of main
Postoffice Destructor
Room Destructor
Clock Destructor
```

Think of it as a depth-first, declaration-order traversal for construction, reversed for destruction.

### 2.4.6 Temporary Object in Constructor Body (postoffice6.h)

```
class Clock             /* File: postoffice6.h */
{
  private:
```

```
    int HHMM;
  public:
    Clock() : HHMM(0) { cout << "Clock Constructor\n"; }
    Clock(int hhmm) : HHMM(hhmm)
        { cout << "Clock Constructor at " << HHMM << endl; }
    ~Clock() { cout << "Clock Destructor at " << HHMM << endl; }
};

class Postoffice
{
  private:
    Clock clock;
  public:
    Postoffice()
        { cout << "Postoffice Constructor\n"; clock = Clock(800); }
    ~Postoffice() { cout << "Postoffice Destructor\n"; }
};
```

Output:

```
Clock Constructor
Postoffice Constructor
Clock Constructor at 800
Clock Destructor at 800
End of main
Postoffice Destructor
Clock Destructor at 800
```

Here a temporary clock object is created by `Clock(800)`. Like a ghost, it is created and destroyed behind the scene.

**Temporary object rule**

A temporary object (like `Clock(800)`) is a ghost: created at its expression, destroyed at the semicolon at the end of the statement (unless bound to a `const T&`, which extends its life).

### 2.4.7 Default Member Initialization and Order of Construction

```
#include <iostream>      /* file: default-member-init.cpp */
using namespace std;

class A
{
    int a;
  public:
    A(int z) : a(z) { cout << "call A's constructor: " << a << endl; }
    ~A() { cout << "call A's destructor: " << a << endl; }
    int get() const { return a; }
};

class B
{
    int b1 = 999;        // Remember: can't initialize by ( )
    A b2 = 10;           // Call A's conversion constructor
    A b3 {100};          // Call A's conversion constructor
  public:
```

```
    B() { cout << "call B's default constructor" << endl << endl; }
    ~B() { cout << "call B's destructor: " << b1 << "\t"
                << b2.get() << "\t" << b3.get() << endl; }
};

int main() { B x; return 0; }
```

Output:

```
call A's constructor: 10
call A's constructor: 100
call B's default constructor

call B's destructor: 999    10    100
call A's destructor: 100
call A's destructor: 10
```

Members are initialized in declaration order (`b1, b2, b3`); `b2`'s constructor runs before `b3`'s. Destructors for `b3, b2` reverse that.

**Note:** non-static data members cannot be default-initialized with `()`, only `=` or {}.

### 2.4.8 Order of Construction & Destruction of Array Elements

```
#include <iostream>      /* File: word-array.cpp */
#include <cstring>
using namespace std;

class Word
{
  private:
    int frequency;
    char* str;
    void set(int f, const char* s)
        { frequency = f; str = new char [strlen(s)+1];
          strcpy(str,s); }
  public:
    Word(const char* s, int k = 1)
        { set(k, s); cout << "conversion: " << s << endl; }
    ~Word()
        { cout << "~Word(): " << str << endl; delete [] str; }

};

int main()
{
    Word weather[] { "sunny", "cloudy", "rainy" };
    return 0;
}
```

Output:

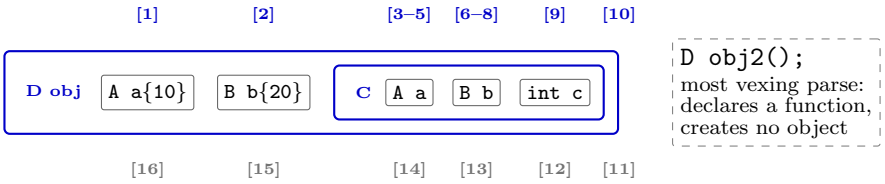
```
conversion: sunny
conversion: cloudy
conversion: rainy
~Word(): rainy
~Word(): cloudy
~Word(): sunny
```

Constructs in **index 0..N-1 order**; destructs in **index N-1..0 order** (reverse).

### 2.4.9 Order Summary

⇒ When an object is constructed, its data members are constructed first.
⇒ When the object is destructed, the data members are destructed after the destructor code of the object has been executed.
⇒ When object A owns other objects, remember to destruct them as well in A's destructor.
⇒ By default, the default constructor is used for the data members.
⇒ We can use a different constructor for the data members by using member initializer list — the "colon syntax".

**Worked Exam Problem: the Full Construction/Destruction Order**




*Construction [1]→[10] follows member **declaration** order (never the MIL order); destruction [11]→[16] is the exact mirror image. Blue numbers = construction, grey = destruction.*

**2024S Midterm, Problem 3 [9 pts] — order of construction/destruction**

**Problem.** What is the output (compiled with `g++ -std=c++11 -fno-elide-constructors`)?

```
#include <iostream>    /* File: SQ-order-ctor-dtor.cpp */
using namespace std;

class A {
  int a;
  public:
    A(int a = 0) : a(a) { cout << "AC1" << endl; }
    A(const A& a) : a(a.a) { cout << "AC2" << endl; }
    ~A() { cout << "AD" << endl; }
};

class B {
  int b;
  public:
    B(int b) : b(b) { cout << "BC1" << endl; }
    B(const B& b = 0) : b(b.b) { cout << "BC2" << endl; }
    ~B() { cout << "BD" << endl; }
};

class C {
  A a;
  B b;
  int c;
  public:
    C() : c(10), a(A()) { cout << "CC" << endl; }
    ~C() { cout << "CD" << endl; }
```

```
};

class D {
  A a{10};
  B b{20};
  C c;
  public:
    D() { cout << "DC" << endl; }
    ~D() { cout << "DD" << endl; }
};

int main() {
  cout << "Beginning of main" << endl;
  D obj;
  D obj2();
  cout << "End of main" << endl;
}
```

**Answer.** The program compiles and prints:

```
Beginning of main
AC1       [1]  D.a{10}   : A(int)
BC1       [2]  D.b{20}   : B(int)
AC1       [3]  c's MIL a(A()): temporary A() constructed
AC2       [4]  copy the temporary into member c.a
AD        [5]  the temporary dies at the end of that initializer
BC1       [6]  c.b default-constructed: 0 -> temp B(0), then copied
BC2       [7]  copy the temporary into member c.b
BD        [8]  the temporary dies
CC        [9]  C's constructor body
DC        [10] D's constructor body
End of main
DD        [11] ~D body first
CD        [12] then members in reverse: ~C body
BD        [13]     c.b
AD        [14]     c.a
BD        [15] D.b
AD        [16] D.a
```

**Solution.** Five ideas are being tested:

- **Declaration order rules.** `C() : c(10), a(A())` lists `c` first, but members are initialized in declaration order: `a`, then `b`, then `c` — exactly the order in the figure above.
- **A temporary in the MIL.** `a(A())` first default-constructs a temporary `A` (`AC1`), copy-constructs the member from it (`AC2`), then destroys the temporary immediately (`AD`). With `-fno-elide-constructors` nothing is optimized away.
- **A copy constructor as a default constructor.** `B(const B& b = 0)` has a default argument, so `B()` is legal: the default argument `0` is converted to `B` by `B(int)` (temporary, `BC1`), which is then copied into `c.b` (`BC2`); the temporary dies (`BD`).
- **Most vexing parse.** `D obj2();` does not create an object — it declares a function named `obj2` returning `D`. Hence nothing between `DC` and `End of main`.
- **Destruction is the exact reverse.** `DD` (body) first, then `D`'s members in reverse declaration order: `c` (whose destructor runs `CD`, then its members in reverse: `BD`, `AD`), then `b` (`BD`), then `a` (`AD`).

**2024S Midterm, Problem 5(a) [12 pts] — `shapes.h`: `Rectangle` & `Cuboid`**

**Problem.** Complete `shapes.h` for

```
class Rectangle{
  private:
    int coords[4];   // top-left X, top-left Y,
                     // bottom-right X, bottom-right Y
  protected:
    void setCoords(const int newCoords[4]);
  public:
    Rectangle(int length, int height, int x = 0, int y = 0);
    Rectangle(const Rectangle& r);
    ~Rectangle();
    int getX() const;  int getY() const;
    int getLength() const;  int getHeight() const;
    void print() const;
};
class Cuboid{
  private:
    Rectangle side;  int width;
  public:
    Cuboid(int length, int height, int width,
           int x = 0, int y = 0);
    Cuboid(const Rectangle& r, int w = 3);
    Cuboid(const Cuboid& c);
    ~Cuboid();
    int getVolume();
};
```

so that the test program (which constructs `Rectangle r1{4,2,2,3}`, copies it, prints it, then builds `Cuboid c1(3,4,5)`) prints `Rectangle constructor`, `Rectangle constructor`, `Rectangle copy constructor`, the vertices, `Rectangle constructor`, `Cuboid constructor`, `Volume: 60`, then three `Rectangle destructors` (the `Cuboid`'s member first) with `Cuboid destructor` first. **Solution.**

```
Rectangle::Rectangle(int length, int height, int x, int y)
    : coords{x,y,x+length,y-height}
{ cout << "Rectangle constructor" << endl; }
Rectangle::Rectangle(const Rectangle& r)
: Rectangle(r.getLength(), r.getHeight(), r.coords[0], r.coords[1])
{ cout << "Rectangle copy constructor" << endl; }
Rectangle::~Rectangle() { cout << "Rectangle destructor" << endl; }
void Rectangle::setCoords(const int newCoords[4])
{ for (int i : {0,1,2,3}) coords[i] = newCoords[i]; }
int Rectangle::getX() const { return coords[0]; }
int Rectangle::getY() const { return coords[1]; }
int Rectangle::getLength() const { return coords[2] - coords[0]; }
int Rectangle::getHeight() const { return coords[1] - coords[3]; }
void Rectangle::print() const {
  cout << "The vertices are:" << endl;
  cout << "(" << coords[0] << "," << coords[1] << ") ";
  cout << "(" << coords[2] << "," << coords[1] << ")" << endl;
  cout << "(" << coords[0] << "," << coords[3] << ") ";
  cout << "(" << coords[2] << "," << coords[3] << ")" << endl;
}
Cuboid::Cuboid(int length, int height, int width, int x, int y)
```

```
: side(length,height,x,y), width(width)
{ cout << "Cuboid constructor" << endl; }
Cuboid::Cuboid(const Rectangle& r, int w) : side(r), width(w)
{ cout << "Cuboid conversion constructor" << endl; }
Cuboid::Cuboid(const Cuboid& c) : side(c.side), width(c.width)
{ cout << "Cuboid copy constructor" << endl; }
Cuboid::~Cuboid() { cout << "Cuboid destructor" << endl; }
int Cuboid::getVolume()
    { return side.getLength() * side.getHeight() * width; }
```

**Key points.**

- `coords` is a plain array; array members cannot be set by assignment — use the MIL brace-list `coords{x,y,x+length,y-height}`.
- The copy constructor delegates to `Rectangle(int,int,int,int)` — that is why a `Rectangle copy constructor` printout is always preceded by a `Rectangle constructor` printout (recall: a delegating constructor must be the only MIL item).
- `Cuboid`'s member `side` is an `Rectangle` object: it is initialized via the MIL before the `Cuboid` body runs, and destroyed after the `Cuboid` destructor body — hence `Cuboid destructor` then `Rectangle destructor`.

**2024S Midterm, Problem 5(d) [3 pts] — array `new` needs a default constructor**

**Problem.** What is the output?

```
#include "shapes.h"
int main(){
  Rectangle* arr = new Rectangle[2];
  arr[0] = {4, 2, 2, 3};
  arr[1] = arr[0];
  arr[1].print();
  delete [] arr;
}
```

**Answer.** COMPILATION ERROR.
**Solution.** `new Rectangle[2]` must default-construct both array elements, but `Rectangle` has user-defined constructors (the 4-parameter one and the copy constructor), so the compiler generates no implicit default constructor ⇒ compilation error, regardless of the rest of the code.

# 3 Inheritance, Polymorphism

## 3.1 Inheritance Basics

### 3.1.1 Motivation: Duplicate Code is Bad

Let's implement a system for maintaining university administrative information. First, a naive approach where `Teacher` and `Student` are two completely separate classes with duplicated code:

```
/* File: student1.h */
enum Department { CBME, CIVL, CSE, ECE, IELM, MAE };

class Student
{
  private:
    string name;
    Department dept;
    float GPA;
    Course* enrolled;
    int num_courses;

     public:
       Student(string n, Department d, float x) :
           name(n), dept(d), GPA(x),
           enrolled(nullptr), num_courses(0) { }
       string get_name() const;
       Department get_department() const;
       float get_GPA() const;
       bool add_course(const Course& c);
       bool drop_course(const Course& c);
};

/* File: teacher1.h */
enum Department { CBME, CIVL, CSE, ECE, IELM, MAE };
enum Rank { PROFESSOR, DEAN, PRESIDENT };

class Teacher
{
  private:
    string name;
    Department dept;
    Rank rank;
    string research_area;

     public:
       Teacher(string n, Department d, Rank r, string a) :
         name(n), dept(d), rank(r), research_area(a) { }
       string get_name() const;
       Department get_department() const;
       Rank get_rank() const;
       string get_research_area() const;
};
```

### 3.1.2 Why Inheritance? Code Reuse

In the naive approach above, we notice large amounts of duplicated code: `name`, `dept`, `get_name()` and `get_department()` are written identically in both classes. This violates the DRY (Don't Repeat Yourself) principle: every bug fix or design change (e.g., add a new `address` member) must be applied in two places and kept perfectly in sync.

**Core idea of Inheritance:** Extract everything shared by several classes into a Base Class (Parent Class). Each derived subclass then only adds its own unique members.

**Inheritance analogy (biological)**

Just like you inherit eye colour and height from your parents, yet still have your own unique hobbies and skills, in code: the **base class = shared traits**, **derived classes = unique traits + inherited shared traits**.

### 3.1.3 UPerson Example: University Member System

**Shared traits (every university member has):**

- `name`
- `dept`
- `get_name()`, `get_department()`

**Student only:**

- `GPA`, `enrolled` (course array), `num_courses`
- `get_GPA()`, `add_course()`, `drop_course()`

**Teacher only:**

- `rank` (PROFESSOR / DEAN / PRESIDENT)
- `teaching` (taught-course array), `num_courses`
- `get_rank()`, `assign_course()`

**Inheritance Syntax**

```
#ifndef UPERSON_H              /* File: uperson.h */
#define UPERSON_H

enum Department { CBME, CIVL, CSE, ECE, IELM, MAE };
class UPerson
{
private:
    string name;
    Department dept;
public:
    UPerson(string n, Department d) : name(n), dept(d) { }
    string get_name() const { return name; }
    Department get_department() const { return dept; }
};

#endif

#ifndef STUDENT_H              /* File: student.h */
#define STUDENT_H

#include "uperson.h"    // Don't forget your parents!!
```

```
class Course { /* incomplete */ };

class Student : public UPerson                // Public inheritance
{
private:
    float GPA;
    Course* enrolled;
    int num_courses;
public:
    Student(string n, Department d, float x) :
        UPerson(n, d), GPA(x), enrolled(nullptr), num_courses(0) { }
    float get_GPA() const { return GPA; }
    bool add_course(const string& c) { /* incomplete */ }
    bool drop_course(const Course& c) { /* incomplete */ }
};
#endif

#ifndef TEACHER_H             /* File: teacher.h */
#define TEACHER_H

#include "uperson.h"    // Don't forget your parents!!
enum Rank { PROFESSOR, DEAN, PRESIDENT };

class Teacher : public UPerson                // Public inheritance
{
private:
    Rank rank;
    string research_area;
public:
    Teacher(string n, Department d, Rank r, string a) :
        UPerson(n, d), rank(r), research_area(a) { }
    Rank get_rank() const { return rank; }
    string get_research_area() const { return research_area; }
};

#endif
```

**is-a relationship + Liskov Substitution Principle**

Inheritance encodes an **is-a** relationship:

- `Student` **is a** `UPerson`
- `Teacher` **is a** `UPerson`

**Liskov Substitution Principle (LSP):** Any context that expects a Base object must also correctly accept a Derived object.
In code: any function taking `UPerson&` or `UPerson*` can accept `Student` or `Teacher`. The reverse is NOT true (a random `UPerson` may not be a `Student`).

### 3.1.4 Inherited Member Access

Derived class objects may call base class functions, but base class objects cannot call derived-only functions:

```
#include <iostream>       /* File: inherited-fcn.cpp */
using namespace std;
#include "student.h"
```

```
void some_func(UPerson& uperson, Student& student) {
    cout << uperson.get_name() << endl;
    Department dept = uperson.get_department();
    // Error! Base class object can't call derived class's function
    uperson.add_course("COMP1001");

    // Derived class object may call base class's member function
    cout << student.get_name() << endl;
    // Derived class object calls its own member functions
    cout << student.get_GPA() << endl;
    student.add_course("COMP2012");
}

int main() {
    UPerson abby("Abby", CBME);
    Student bob("Bob", CIVL, 3.0);
    some_func(abby, bob);
}
```

### 3.1.5 LSP in Action: Print Label Example

By deriving `Student` and `Teacher` from `UPerson`, functions written for `UPerson` automatically work for all derived classes:

```
#include <iostream>       /* File: print-label.cpp */
using namespace std;
#include "student.h"
#include "teacher.h"

void print_label(const UPerson& uperson)
{
    cout << "Name: " << uperson.get_name() << endl;
    cout << "Dept: " << uperson.get_department() << endl;
}

int main()
{
    Student tom("Tom", CIVL, 3.9);
    print_label(tom);   // Tom is also a UPerson

    Teacher alan("Alan Turing", CSE, PROFESSOR, "AI");
    print_label(alan); // Alan is also a UPerson
    return 0;
}
```

When multiple overloads exist, the compiler chooses based on the static (declared) type, not the actual runtime type:

```
#include <iostream>       /* File: print-label2.cpp */
using namespace std;
#include "student.h"

void print_label(const UPerson* uperson) {
    cout << "Name: " << uperson->get_name() << endl;
    cout << "Dept: " << uperson->get_department() << endl;
}
void print_label(const UPerson& uperson) {
```

```
    cout << "Name: " << uperson.get_name() << endl;
    cout << "Dept: " << uperson.get_department() << endl;
}
void print_label(const Student& student) {
    cout << "Name: " << student.get_name() << endl;
    cout << "Dept: " << student.get_department() << endl;
    cout << "GPA: " << student.get_GPA() << endl;
}
int main() { // Which print_label()?
    Student tom("Tom", CIVL, 3.9); print_label(tom);
    UPerson& tom2 = tom; print_label(tom2);
    UPerson* p = &tom; print_label(p);
}
```

**Quiz: Substitute dance / study**

```
#include <iostream>       /* File: substitute.cpp */
using namespace std;
#include "student.h"

int main() {
    void dance(const UPerson& p); // Anyone can dance
    void dance(const UPerson* p); // Anyone can dance
    void study(const Student& s); // Only students study
    void study(const Student* s); // Only students study
    UPerson p("P", IELM); Student s("S", MAE, 3.3);

    // Which of the following statements can compile?
    dance(p);
    dance(s);
    dance(&p);
    dance(&s);
    study(s);
    study(p);
    study(&s);
    study(&p);
}
```

**Answers:**
- `dance(p)` ✓; `dance(s)` ✓ (upcast reference)
- `dance(&p)` ✓; `dance(&s)` ✓ (upcast pointer)
- `study(s)` ✓; `study(p)` × (cannot downcast UPerson to Student)
- `study(&s)` ✓; `study(&p)` × (cannot downcast UPerson* to Student*)

**Memory mnemonic:** Derived → Base ("upcast") is always safe; Base → Derived ("downcast") is unsafe and needs RTTI (see later). Just like "every cat is an animal, but not every animal is a cat".

### 3.1.6 Indirect Inheritance: PG_Student

Inheritance chains naturally (grandparent / parent / grandchild). No code duplication in the middle layer.

```
#ifndef PG_STUDENT_H          /* File: pg-student.h */
#define PG_STUDENT_H
```

```
#include "student.h"

class PG_Student : public Student
{
private:
    string research_topic;
public:
    PG_Student(string n, Department d, float x) :
        Student(n, d, x), research_topic("") { }

    string get_topic() const { return research_topic; }
    void set_topic(const string& x) { research_topic = x; }
};

#endif
```

**Call only the DIRECT parent's constructor from MIL**

`PG_Student` **must NOT call** `UPerson` **directly in its MIL.** It may only call its immediate parent `Student`, which in turn calls `UPerson`.
Analogy: you cannot skip your parents and ask your grandparents to give birth to you; the chain must be followed one generation at a time.

We can easily add classes to our existing class hierarchy. New classes can immediately benefit from all functions that are available to their base classes. For example, `print_label(const UPerson&)` will work immediately for a new class called `PG_Student`, even though this type of objects was unknown when `print_label()` was designed and written:

```
#include <iostream>     /* File: pg-print-label.cpp */
using namespace std;
#include "pg-student.h" // Change student.h to pg-student.h

void print_label(const UPerson& uperson)
{
    cout << "Name: " << uperson.get_name() << endl;
    cout << "Dept: " << uperson.get_department() << endl;
}

int main()
{
    PG_Student tom("Tom", CIVL, 3.9); // Tom is now a PG Student
    print_label(tom);                 // Tom is also a UPerson
    return 0;
}
```

## 3.2 Inheritance: C/D order + Slicing + Name Hiding + Access Control

### 3.2.1 Constructor / Destructor Order with Inheritance

Previously we learned the Has-A (embedded member) order: members constructed first, enclosing constructor body last. With inheritance we now have **three layers**:

```
#include <iostream>          /* File: init-order.cpp */
using namespace std;

class Address {
public:
    Address() { cout << "Address's constructor" << endl; }
    ~Address() { cout << "Address's destructor" << endl; }
};

class UPerson {
public:
    UPerson() { cout << "UPerson's constructor" << endl; }
    ~UPerson() { cout << "UPerson's destructor" << endl; }
};

class Student : public UPerson {
public:
    Student() { cout << "Student's constructor" << endl; }
    ~Student() { cout << "Student's destructor" << endl; }

private:
    Address address;
};

int main() { Student x; return 0; }
```

**3 Golden Rules of Constructor/Destructor Order (with Inheritance)**

1. **Construction order:** (1) Base class constructor → (2) Derived class's embedded members (in declaration order) → (3) Derived class's own constructor body.
2. **Destruction order:** exact reverse: derived destructor body first → derived members (reverse declared) → base class destructor last.
3. Within any layer (members / bases) the declared order wins; MIL ordering is ignored.

**House-building analogy:** Lay the foundation (base class) → install plumbing/windows (embedded members) → decorate and move in (your constructor body). Demolition proceeds in reverse order: move your stuff out → strip fixtures → excavate the foundation.

The code above produces:

```
UPerson's constructor
Address's constructor
Student's constructor
Student's destructor
Address's destructor
UPerson's destructor
```

### Quiz: Move `Address` to `UPerson`

```
#include <iostream>      /* File: init-order2.cpp */
using namespace std;

class Address {
public:
    Address() { cout << "Address's constructor" << endl; }
    ~Address() { cout << "Address's destructor" << endl; }
};

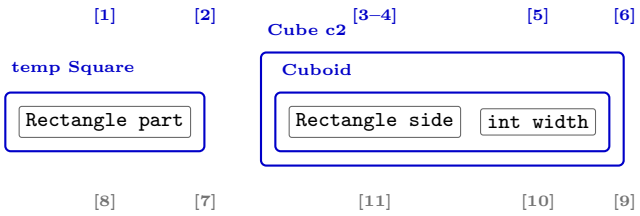
class UPerson {
public:
    UPerson() { cout << "UPerson's constructor" << endl; }
    ~UPerson() { cout << "UPerson's destructor" << endl; }
private:
    Address address;
};

class Student : public UPerson {
public:
    Student() { cout << "Student's constructor" << endl; }
    ~Student() { cout << "Student's destructor" << endl; }
};
```

**Answer:**

```
Address's constructor
UPerson's constructor
Student's constructor
Student's destructor
UPerson's destructor
Address's destructor
```

### Worked Exam Problem: Construction Chain with Inheritance




*Base/member construction [1]→[6] top-down; destruction [9]→[11] reverses it. The temporary Square — created by converting `3` for `Cube(const Square&)` — lives only inside steps [1]–[2] and dies at [7]–[8].*

### 2024S Midterm, Problem 5(b) [7 pts] — `square.h`: a derived class

**Problem.** Write the entire header `square.h` for `Square`, derived from `Rectangle` with no extra data members: a constructor `Square(int len, int x = 0, int y = 0)`, a destructor, `print()`, and `flip()` (move the rightmost edge to the left). Test: `Square s(3,4,4); s.flip(); s.print();` must print `Rectangle constructor`, `Square constructor`, the vertices `(7,4) (10,4) / (7,1) (10,1)`, `This is a square!`, `Square destructor`, `Rectangle destructor`. **Solution.**

```
#include "shapes.h"
class Square : public Rectangle {
  public:
    Square(int l, int xcord = 0, int ycord = 0) :
      Rectangle(l, l, xcord, ycord)
          { cout << "Square constructor" << endl; }
    ~Square() { cout << "Square destructor" << endl; }
    void print() const {
        Rectangle::print();
        cout << "This is a square!" << endl;
    }
    void flip() { int arr[4] {getX()+getLength(), getY(),
                              getX()+2*getLength(),
                              getY()-getHeight()};
                  setCoords(arr); }
};
```

**Key points.**

- The base is initialized through the MIL: `Rectangle(l, l, ...)` makes length = height = `l`.
- `print()` reuses the base version with the scope operator `Rectangle::print()`.
- `setCoords(...)` is `protected` in `Rectangle` — a derived class may call it on itself, even though `main` could not.
- Destruction order: `Square` body first, then the base `Rectangle` — always the mirror image of construction.

## 2024S Midterm, Problem 5(c) [5 pts] — output of `Cube c2(3);` (no copy elision)

**Problem.** Given

```
class Cube : public Cuboid {
  public:
    Cube(const Square& s) : Cuboid(s,s.getLength())
      { cout << "Cube conversion constructor" << endl; }
    ~Cube() { cout << "Cube destructor" << endl; }
};
int main(){ Cube c2(3); }
```

what is the output (no optimizations or copy elisions)? **Answer.**

```
Rectangle constructor                [1]  3 -> temp Square: Rect(3,3,0,0)
Square constructor                   [2]  temp Square's body
Rectangle constructor                [3]  c2's Cuboid part; its member
Rectangle copy constructor           [4]    side: copy-ctor delegation
Cuboid conversion constructor        [5]  Cuboid body
Cube conversion constructor          [6]  Cube body
Square destructor                    [7]  the temporary Square dies HERE,
Rectangle destructor                 [8]    at the ; that ends the decl
Cube destructor                      [9]  end of main: ~Cube body
Cuboid destructor                    [10] base Cuboid (body, then
   members)
Rectangle destructor                 [11] member side
```

**Solution.**

- `Cube c2(3);` needs a `Square`; `3` is converted by the `Square` conversion constructor — a temporary object ([1][2]).
- A derived object constructs its base subobject first: `Cuboid(s, s.getLength())` — note `s` binds as `const Rectangle&` (upcast, no copy). The member `side`

is copy-constructed from it; the copy constructor delegates, hence the extra `Rectangle constructor` at `[3]`.

- **The trap:** lines [7]–[8] appear before `Cube destructor`. The temporary `Square` is destroyed at the end of the full expression (the `;` of `Cube c2(3);`), while `c2` lives until the closing brace of `main`.
- Destruction of `c2` is the exact reverse: `Cube` body → `Cuboid` body → member `side`.

**2025S Midterm, Problem 2 [8 pts] — debugging an inheritance hierarchy**

**Problem.** The two classes below contain 4 errors (syntactic or semantic). Modify 4 lines so the program compiles and produces the expected output. Do not modify `main()`.

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  //base vehicle class
6  class Vehicle {
7  protected:
8    string model;
9    int speed;
10 public:
11   Vehicle(string m, int s) : model(m), speed(s) {}
12   void display() { cout << model << ": "
13                         << speed << " km/h" << endl; }
14   bool equals(const Vehicle& v) { return speed == v.speed; }
15   string getName() const { return model; }
16 };
17
18 //derived car class that publicly inherits vehicle
19 class Car : Vehicle {
20   int seats;
21 public:
22   Car(string m, int s, int st) : seats(st) {}
23   void display() { cout<<model<<": "<<speed<<" km/h, "
24                         <<seats<<" seats"<<endl; }
25   bool equals(const Car& c) { return seats == c.seats; }
26 };
27
28 int main() {
29   Car a("Sedan1", 120, 5);
30   Car b("Sedan2", 100, 5);
31   Car c("SUV", 120, 7);
32   Vehicle* vp = &c;
33
34   //output whether cars a and b are identical
35   //except for their model names
36   cout << boolalpha << "Equality between "
37        << a.getName() << " and " << b.getName()
38        << ": " << a.equals(b) << endl;
39
40   //display all the information
41   //about the car pointed to by vp
42   vp->display();
```

```
43 }
```

**Expected output:**

```
Equality between Sedan1 and Sedan2: false
SUV: 120 km/h, 7 seats
```

**Solution** (line number → corrected code):

```
// Line 12:
virtual void display()
    { cout << model << ": " << speed << " km/h" << endl; }
// Line 18:
class Car : public Vehicle {
// Line 21:
Car(string m, int s, int st) : Vehicle(m, s), seats(st) {}
// Line 23:
bool equals(const Car& c)
    { return seats == c.seats && speed == c.speed; }
```

(equivalently for Line 23: `return seats == c.seats && Vehicle::equals(c);`)

**Why each fix is needed:**

- **Line 12 — polymorphism needs `virtual`.** `vp` is a `Vehicle*` pointing to a `Car`; the expected output `SUV: 120 km/h, 7 seats` requires dynamic binding to `Car::display()`, so the base version must be `virtual`.
- **Line 18 — default inheritance is `private`.** `class Car : Vehicle` means private inheritance, which blocks the external upcast `Vehicle* vp = &c;` in `main`. Must write `public Vehicle`.
- **Line 21 — the base has no default constructor.** `Car` only initialises `seats`; the compiler would need `Vehicle`'s default constructor, which does not exist — and `model`/`speed` would be left uninitialised anyway. Initialise the base in the MIL: `Vehicle(m, s)`.
- **Line 23 — semantic bug.** "Identical except for their model names" must also compare speeds: `a` (120) vs `b` (100) differ, so the output is `false`. Comparing only `seats` (5 == 5) would wrongly print `true`.

### 3.2.2 Fatal Trap: Object Slicing

**Only occurs with public inheritance + value assignment / value passing.**

```
#include <iostream>     /* File: slice.cpp */
#include <string>
using namespace std;
#include "../basics/uperson.h"
#include "../basics/student.h"

int main()
{
    Student student("Snoopy", CSE, 3.5);
    UPerson* pp = &student;
    UPerson* pp2 = new Student("Mickey", ECE, 3.4);

    UPerson uperson("Unknown", CIVL);
    uperson = student; // What does "uperson" have?
    delete static_cast<Student*>(pp2);
    return 0;
}
```

**Slicing = Permanent, Silent Data Loss**

When a Derived object is copied (by **value**) into a Base variable, the compiler only copies the Base sub-object portion of memory. All derived-extra members are discarded silently (no warning, no exception, no trace).

**Pointers and references NEVER slice.** They only store/carry the address of the complete original Derived object sitting in memory.

```
// Safe alternatives:
UPerson* p = &st;   // NO slicing; points to the full Student object
UPerson& r = st;    // NO slicing; alias for the full Student
```

**Quiz: Will it slice?**

```
void f1(UPerson x);   void f2(UPerson& x);   void f3(UPerson* x);
Student s;

f1(s);          f2(s);          f3(&s);
```

**Answers:**
- `f1(s)` — **YES, slices** (pass-by-value truncates Derived)
- `f2(s)` — **No slice** (reference)
- `f3(&s)` — **No slice** (pointer)

### 3.2.3 Name Hiding (NOT Overloading!)

**Bug pattern: same-name function in Base + Derived = Name Hiding**

```
/* File: name-conflict.h */
void print(int x, int y) { cout << x << " , " << y << endl; }

class B
{
private:
    int x, y;
public:
    B(int p = 1, int q = 2) : x(p), y(q)
        { cout << "Base class constructor: "; print(x, y); }
    void f() const { cout << "Base class: "; print(x, y); }
};

class D : public B
{
private:
    float x, y;
public:
    D() : x(10.2), y(20.6)
        { cout << "Derived class constructor\n"; }
    void f() const
        { cout << "Derived class: "; print(x, y); B::f(); }
}; // D::f() hides away B::f().
   // This case is NOT function overloading
```

**Root cause:** As soon as `Derived` declares *any* function called `f`, ALL same-named

functions inherited from `Base` become hidden. The compiler does **not** look across different class scopes for overload resolution. Overloading happens only within **the same single scope** (one class).

**PDF follow-up example:**

```
#include <iostream>     /* File: name-conflict.cpp */
using namespace std;
#include "name-conflict.h"

void smart(const B* p) { cout << "Inside smart(): "; p->f(); }

int main()
{
    B base(5, 6); cout << endl;
    D derive; cout << endl;

    B* bp = &base; bp->f(); cout << endl;
    D* dp = &derive; dp->f(); cout << endl;

    bp = &derive; bp->f(); cout << endl;
}
```

### 3.2.4 Motivation for Access Control: Student::print( ) Problem

First, let's add a `print()` function to both `UPerson` and `Student`. The naive approach of accessing `UPerson`'s private members directly from `Student::print()` does **not** compile:

```
/* File: print1.cpp */

class UPerson { public: void print() const; ...              };

class Student: public UPerson { public: void print() const; ...   };

void UPerson::print() const
{
    cout << "--- UPerson details ---" << endl;
    cout << "Name: " << name << endl << "\nDept: " << dept << endl;
}

void Student::print() const
{
    cout << "--- Student details ---" << endl
         << "Name: " << name << endl
         << "\nDept: " << dept << endl << "Enrolled in:" << endl;
    for (int i = 0; i < num_courses; i++)
        enrolled[i].print(); // Assume a print function in Course
}
```

The implementation of `Student::print()` given above doesn't work. It will raise an error during compilation: `Student::print(): name and dept are declared private`. `name` is a private data member of the base class `UPerson`. Public inheritance does not change the access control of the data members of the base class. Private members are still only available to base class' own member functions (methods), and not to any other classes including derived classes (except friends) or global functions.

### 3.2.5 One Solution: Protected Data Members

By making `name` and `dept` protected, they are accessible to methods in the base class as well as methods in the derived classes. They should not be public though! (Principle of information hiding.)

```
class UPerson                    /* File: protected-uperson.h */
{
  protected:
    string name;
    Department dept;

     public:
       UPerson(string n, Department d) : name(n), dept(d) { }
       void print() const;
       ...
};
```

### 3.2.6 Superior Solution: Use Public Member Functions

In general, it is preferable to have private members instead of protected members. Use protected only where it is really necessary. Private is the only category ensuring full data encapsulation. In our example, there is no reason at all to make `name` and `dept` protected, as we can access them through appropriate public member functions:

```
void Student::print() const /* correct-student-print.cpp */
{
    cout << "--- Student details ---" << endl
         << "Name: " << get_name() << endl // Use UPerson's public fcn
         << "Dept: " << get_dept() << endl // Use UPerson's public fcn
         << "Enrolled in:" << endl;

    for (int i = 0; i < num_courses; i++)
        enrolled[i].print();                    // Use Course's
                                                // public fcn
}
void Teacher::print() const /* correct-teacher-print.cpp */
{
    cout << "--- Teacher details ---" << endl
         << "Name: " << get_name() << endl // Use UPerson's public fcn
         << "Dept: " << get_dept() << endl // Use UPerson's public fcn
         << "Rank: " << get_rank() << endl; // Use its own fcn
}
```

Now let's use the new `print()` functions:

```
/* File: print-example.cpp (incomplete) */
UPerson newton("Isaac Newton", MAE);
Teacher turing("Alan Turing", CSE, DEAN);
Student edison("Thomas Edison", ECE, 2.5);
edison.add_course("COMP2012");

newton.print();
turing.print();
edison.print();
```

### 3.2.7 Three Access Specifiers: public / protected / private

| **Member mod. ↓** | Inside class | Derived code | External code |
|---|---|---|---|
| `public` | yes | yes | yes |
| `protected` | yes | yes | no |
| `private` | yes | no | no |

**`protected` exists only to grant a back-door to derived classes**

- Without inheritance, `protected` behaves identically to `private`. External code touches neither.
- With inheritance, `protected` additionally allows member functions of **any descendant class** direct access.

**Protected data members break encapsulation**

While exposing protected data seems convenient for derived classes, it couples all descendants to the base implementation. If the base class ever re-implements a data member (e.g. swaps `string name;` for `char* name;`), EVERY derived class that read or wrote the protected data member must also be edited.
**Superior pattern:** always make data `private`, and expose controlled `protected` getter/setter member functions. Now changing the base's internals does *not* force any descendant edit.

```
class UPerson
{
private:
    string name;       // data: always private
    Department dept;
protected:
    // controlled back-door for descendants only
    void set_dept(Department d) { dept = d; }
public:
    string get_name() const { return name; }     // public-facing API
    Department get_dept() const { return dept; }
};

class Student : public UPerson
{
public:
    void transfer(Department new_dept)
    {
        set_dept(new_dept);     // OK: uses protected setter
        // dept = new_dept;
        // ERROR: private access to UPerson::dept
    }
};
```

## 3.3 Polymorphism & Virtual Functions

First, let's revisit the print-label scenario with three passing styles. Even though we pass `Student` and `Teacher` objects, the default C++ binding calls **only the UPerson::print()** version (slicing for value, static dispatch for ptr/ref):

```
#include <iostream>        /* File: print-label.cpp */
using namespace std;
#include "student.h"
#include "teacher.h"

void print_label_v(UPerson uperson) { uperson.print(); }
void print_label_r(const UPerson& uperson) { uperson.print(); }
void print_label_p(const UPerson* uperson) { uperson->print(); }

int main() {
    UPerson uperson("Charlie Brown", CBME);
    Student student("Edison", ECE, 3.5);
    Teacher teacher("Alan Turing", CSE, PROFESSOR, "CS Theory");
    student.add_course("COMP2012"); student.add_course("MATH1003");

    cout << "\n##### PASS BY VALUE #####\n";
    print_label_v(uperson); print_label_v(student);
    print_label_v(teacher);

    cout << "\n##### PASS BY REFERENCE #####\n";
    print_label_r(uperson); print_label_r(student);
    print_label_r(teacher);

    cout << "\n##### PASS BY POINTER #####\n";
    print_label_p(&uperson); print_label_p(&student);
    print_label_p(&teacher);
}
```

All 9 calls above print `--- UPerson Details ---` only. The Student's GPA/Courses and Teacher's Rank are invisible! This is **static binding** (the C++ default): dispatch is chosen at compile time from the declared type (`UPerson`), never from the actual run-time object type. What we actually want is the derived-class `print()` to run whenever we have a real Student or Teacher — that requires **dynamic (late) binding via virtual functions**.

Here is who may call whose `print()` under static binding:

```
#include <iostream>      /* File: static-example.cpp */
using namespace std;
#include "teacher.h"

int main()
{
    UPerson uperson("Charlie Brown", CBME);
    Teacher teacher("Alan Turing", CSE, PROFESSOR, "CS Theory");
    UPerson* u; Teacher* t;

    cout << "\nUPerson object pointed by UPerson pointer:\n";
    u = &uperson; u->print();

    cout << "\nTeacher object pointed by Teacher pointer:\n";
    t = &teacher; t->print();
```

```
    cout << "\nTeacher object pointed by UPerson pointer:\n";
    u = &teacher; u->print();

    cout << "\nUPerson object pointed by Teacher pointer:\n";
    t = &uperson; t->print(); // Error: convert base-class ptr
                              //        to derived-class ptr
    t = static_cast<Teacher*>(&uperson); t->print(); // Ok, but ...
}
```

### 3.3.1 Static Binding vs. Dynamic Binding

**Binding** = the process of resolving **which concrete version** of a named function should execute.

- **Static Binding (compile-time):** the compiler chooses, based only on the declared type of the pointer / reference. This is **the C++ default** unless you opt-in (Pascal / C / Fortran style).
- **Dynamic Binding (run-time):** the program waits until execution, inspects the actual object type pointed to or referred to, then dispatches to the implementation of that specific class. Requires the keyword `virtual` on the base-class declaration.

By default, C++ uses static binding. In static binding, what a pointer really points to, or what a reference actually refers to is not considered; only the pointer/reference type is. But C++ also allows dynamic binding which is supported through virtual functions.

When dynamic binding is used, the actual method to be called is selected using the actual type of the object in the call, but only if the object is passed by reference or pointer. i.e., `print_label_r(a UPerson object)` calls `UPerson::print()`; `print_label_r(a Teacher object)` calls `Teacher::print()`; `print_label_r(a Student object)` calls `Student::print()`.

Here is the full UPerson family of classes with `virtual print()`:

```
#ifndef V_UPERSON_H       /* File: v-uperson.h */
#define V_UPERSON_H

enum Department { CBME, CIVL, CSE, ECE, IELM, MAE };
class UPerson
{
  private:
    string name;
    Department dept;

  public:
    UPerson(string n, Department d) : name(n), dept(d) { }
    string get_name() const { return name; }
    Department get_department() const { return dept; }

   virtual void print() const
   {
       cout << "--- UPerson Details --- \n"
            << "Name: " << name << "\nDept: " << dept << "\n";
   }
};
#endif

#ifndef COURSE_H             /* File: course.h */
#define COURSE_H

class Course
```

```
{
  private:
    string code;

     public:
       Course(const string& s) : code(s) { }
       ~Course() { cout << "destruct course: " << code << endl; }
       void print() const { cout << code; }
};

#endif

#ifndef V_STUDENT_H      /* File: v-student.h */
#define V_STUDENT_H
#include "course.h"
#include "v-uperson.h"

class Student : public UPerson { // Public inheritance
  private:
    float GPA; Course* enrolled[50]; int num_courses;

  public:
    Student(string n, Department d, float x) :
        UPerson(n, d), GPA(x), num_courses(0) { }
    ~Student() {
        for (int j = 0; j < num_courses; ++j)
            delete enrolled[j];
    }
    float get_GPA() const { return GPA; }
    bool add_course(const string& s)
        { enrolled[num_courses++] = new Course(s); return true; }
    virtual void print() const {
        cout << "--- Student Details --- \n"
             << "Name: " << get_name()
             << "\nDept: " << get_department()
             << "\n" << num_courses << " Enrolled courses: ";
        for (int j = 0; j < num_courses; ++j)
           { enrolled[j]->print(); cout << " "; } cout << endl;
    }
};
#endif

#ifndef V_TEACHER_H      /* File: v-teacher.h */
#define V_TEACHER_H
#include "v-uperson.h"

enum Rank { PROFESSOR, DEAN, PRESIDENT };
class Teacher : public UPerson // Public inheritance
{
  private:
    Rank rank;
    string research_area;

  public:
    Teacher(string n, Department d, Rank r, string a) :
        UPerson(n, d), rank(r), research_area(a) { }
    Rank get_rank() const { return rank; }
```

```
    string get_research_area() const { return research_area; }
    virtual void print() const {
        cout << "--- Teacher Details --- \n"
             << "Name: " << get_name()
             << "\nDept: " << get_department()
             << "\nRank: " << rank
             << "\nResearch area: " << research_area << endl;
    }
};
#endif
```

**Polymorphism = Many Forms**

Polymorphism in C++ means that we can work with objects without knowing their precise type at compile time.

```
void print_label_p(const UPerson* uperson) { uperson->print(); }
```

The type of the object pointed to by `uperson` is not known to the programmer writing this code, nor to the compiler. We say that `uperson` exhibits polymorphism, because the object can take on multiple "shapes" (Student, Teacher, PG Student, etc.). Polymorphism allows us to write programs that behave correctly even when used with objects of derived classes. Again a pointer or reference must be used to take advantage of polymorphism.

**3 prerequisites for virtual dispatch (ALL must hold):**

1. Base class declares the function `virtual`.
2. You call the function through a **pointer** or a **reference**. Call-by-value slices + reverts to static.
3. Every derived override has a **100% identical signature** (parameter list, cv-qualifiers including `const` must all match).

### 3.3.2 Example: Polymorphism using Virtual Function

```
#include <iostream>      /* File: v-example.cpp */
using namespace std;
#include "v-student.h"
#include "v-teacher.h"

int main()
{
    char person_type; string name; UPerson* uperson[3];

    for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
    {
        cout << "Input the uperson type (u/s/t) and his name : ";
        cin >> person_type >> name;
        switch (person_type)
        {
            case 'u': uperson[j] = new UPerson(name, MAE); break;
            case 's': uperson[j] =
                      new Student(name, CIVL, 4.0); break;
            case 't': uperson[j] =
                      new Teacher(name, CSE, DEAN, "AI"); break;
        }
    }
```

```
    for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
        uperson[j]->print();
    return 0;
} // The example doesn't destruct the dynamically allocated objects
```

### 3.3.3 C++11 `override` Keyword: Signature Safety Net

It is trivially easy to typo a derived override signature (e.g. drop a `const`, tweak a parameter type). The compiler silently treats it as a brand-new unrelated overload instead of an override, so at run-time the base version executes unexpectedly — nightmare to debug.

C++11 solution: append `override` to the declaration inside the class. The compiler **stops compilation** if the signature does **not** really match any base virtual function:

```
#include <iostream>          /* File: override.cpp */
using namespace std;

class Base
{
public:
    virtual void f(int a) const { cout << a << endl; }
};

class Derived: public Base
{
    int x {25};
public:
    void f(int) const override;
};

// Don't repeat the keyword override here
void Derived::f(int b) const { cout << x+b << endl; }

int main() { Derived d; Base& b = d; b.f(5); return 0; }
```

**Where to write `override`**

Only inside the class-body **declaration**. When you implement the function outside the class (in a `.cpp`), do **not** repeat `override`:

```
// student.h
class Student : public UPerson {
    void print() const override;   // declare with override
};
// student.cpp
void Student::print() const        // define WITHOUT override
    { cout << "Student print\n"; }
```

### 3.3.4 Overloading vs. Overriding — never confuse again

| | Overloading | Overriding |
|---|---|---|
| Scope | **one single class** | across **Base + Derived** (inheritance) |
| Name | identical | identical |
| Parameter list | **must differ** | **must match exactly** |
| `virtual` | irrelevant | **Base must be virtual** |
| Binding time | compile-time (static) | run-time (dynamic) |
| Purpose | same op on different types | same API, different per-class impl |

**2024S Midterm, Problem 1(e) [2 pts] — virtual is decided by the static type**

**Problem.** What is the output?

```
#include <iostream>
using namespace std;
class X {
public:
    void f() const { cout << "X::f" << endl; }
};
class Y: public X {
public:
    virtual void f() const { cout << "Y::f" << endl; }
};
class Z: public Y {
public:
    void f() const { cout << "Z::f" << endl; }
};
int main() {
    Z obj;
    X& ref = obj;
    ref.f();
    return 0;
}
```

**Answer.** `X::f`

**Solution.** Whether a call is dynamically bound is decided by the **static type** of the expression used to call: here `ref` is an `X&`, and `X::f` is not declared `virtual`. Adding `virtual` in the derived class (`Y`) does not retroactively make `X::f` virtual. So `ref.f()` is statically bound to `X::f` — even though the dynamic type is `Z`. (Had the reference been `Y&` or `Z&`, the call would dispatch to `Z::f`.)

### 3.3.5 RTTI: Run-Time Type Information

RTTI is a run-time facility that keeps track of dynamic types ⇒ program can determine an object's type at run-time.

- `typeid(<expression>)` returns an object of the type `type_info`. It has the member function `const char* name() const` that returns the type name of the expression.
- `typeid()` can be used to find out the type name of any expression (which can be an object, pointer, reference, etc.)
- `static_cast()` may be used to perform type conversions, including conversions between pointers to classes in an inheritance hierarchy; it doesn't consult RTTI to ensure the conversion is safe; thus, it runs faster.

- `dynamic_cast()`, on the other hand, only works on pointers and references of polymorphic class (with virtual functions) types; it consults RTTI to make sure the conversion result refers to a valid complete object of the target type.
  - If the input is a pointer: it returns a pointer to a valid complete object of the target type, otherwise, a `nullptr`.
  - If the input is a reference: it returns a reference to a valid complete object of the target type, otherwise, it is a runtime error!

**Example: RTTI** `typeid(<expression>).name()`

```
#include <iostream>     /* File: rtti.cpp */
#include <typeinfo>     // In order to use typeid
using namespace std;
#include "v-student.h"
#include "v-teacher.h"
int main()
{
    char person_type; string name; UPerson* uperson[3];

    for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
    {
        cout << "Input the uperson type (u/s/t) and his name : ";
        cin >> person_type >> name;
        switch (person_type)
        {
            case 'u': uperson[j] = new UPerson(name, MAE); break;
            case 's': uperson[j] =
                      new Student(name, CIVL, 4.0); break;
            case 't': uperson[j] =
                      new Teacher(name, CSE, DEAN, "AI"); break;
        }
    }

    for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
        cout << "The uperson #" << j << " is a "
             << typeid(*uperson[j]).name() << endl; // RTTI
} // The example doesn't destruct the dynamically allocated objects
```

Sample output (g++ decorates type names):

```
Input the uperson type (u/s/t) and his name : s Abby
Input the uperson type (u/s/t) and his name : t Brian
Input the uperson type (u/s/t) and his name : s Chris
The uperson #0 is a 7Student
The uperson #1 is a 7Teacher
The uperson #2 is a 7Student
```

The returned type name is implementation dependent (different compilers may give different printout). In this course, we assume the above printout from the g++ compiler.

**Example: Compare** `typeid(<expression>)`

```
#include <iostream>     /* File: rtti2.cpp */
#include <typeinfo>     // In order to use typeid
using namespace std;
#include "v-student.h"
```

```
#include "v-teacher.h"
int main()
{
    char person_type; string name; UPerson* uperson[3];

    for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j) {
        cout << "Input the uperson type (u/s/t) and his name : ";
        cin >> person_type >> name;
        switch (person_type) {
            case 'u': uperson[j] = new UPerson(name, MAE); break;
            case 's': uperson[j] =
                      new Student(name, CIVL, 4.0); break;
            case 't': uperson[j] =
                      new Teacher(name, CSE, DEAN, "AI"); break;
        }
    }
    cout << boolalpha;
    for (int i = 0; i < sizeof(uperson)/sizeof(UPerson*); ++i) {
        for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
            cout << (typeid(*uperson[i])
                     == typeid(*uperson[j])) << "\t";
        cout << endl;
    }
} // The example doesn't destruct the dynamically allocated objects
```

**`typeid` pitfalls**

- **Dereference the pointer!** `typeid(p)` asks about the static type of the variable `p` itself (always `UPerson*`). You must write `typeid(*p)` to inspect the run-time object.
- **Base class must contain at least one `virtual` function** (i.e. be a polymorphic type). Otherwise RTTI is not recorded and `typeid` falls back to static type.

### Different Ways to Check Real Identity of Objects

1. Compare directly the returned `typeid()` objects:

   ```
   typeid(*uperson[i]) == typeid(*uperson[j])
   ```

   `typeid()` can be used to find the real identity from a base pointer only if, the base class is polymorphic (having virtual function(s)) and the expression represents an object (not pointer).
2. Compare the names of the returned `typeid()` objects:

   ```
   strcmp(typeid(*uperson[i]).name(), typeid(*uperson[j]).name()) == 0
   ```

3. Try dynamic-casting the object pointer to the desired type and check if it succeeds, though it also succeeds when the cast type is a pointer to an ancestor of the actual type:

   ```
   dynamic_cast<Student*>(uperson[i]) != nullptr
   ```

### (2) `dynamic_cast<Derived*>(base_ptr)` — checked cast (pointer flavour)

- Consults RTTI at run-time. Succeeds only if the actual target type (or a descendant) matches.
- Returns `nullptr` on failure.

- Works only on polymorphic base classes.

```
Student* sp = dynamic_cast<Student*>(base_ptr);
if (sp != nullptr)
    cout << "GPA = " << sp->get_GPA() << endl;   // safe!
else
    cout << "Not a Student (no GPA)\n";
```

**(3)** `dynamic_cast<Derived&>(base_ref)` — **checked cast (reference flavour)** References have no "null value", so failure instead **throws** the standard exception `std::bad_cast`:

```
try {
    Student& sr = dynamic_cast<Student&>(*base_ptr);
    cout << "GPA = " << sr.get_GPA() << endl;
} catch (const std::bad_cast& e) {
    cout << "Cast failed: not a Student\n";
}
```

**2024S Midterm, Problem 6 [25 pts] — `WindBand`: polymorphic container & deep copy**

**Problem.** Six classes form two hierarchies (`Instrument` ← `BrassInstrument`/`WoodwindInstrument`; `BandMember` ← `WindBandMember`) plus a container class `WindBand` holding an array of `WindBandMember` pointers. The given headers:

```
/* File: Instrument.h */
class Instrument {
  private:
    const string name;
    const int year;
  public:
    Instrument(const string& name, int year)
    : name(name), year(year) {}
    virtual ~Instrument() {}
    string getName() const { return name; }
    int getYear() const { return year; }
    virtual void play() const = 0;          // ABC!
};
class BrassInstrument : public Instrument {
  private:
    double boreDiameter;
  public:
    BrassInstrument(const string& name, int year,
                    double boreDiameter)
    : Instrument(name, year), boreDiameter(boreDiameter) {}
    double getBoreDiameter() const { return boreDiameter; }
    void play() const override { cout << "brass instrument: "
                                      << getName() << endl; }
};
class WoodwindInstrument : public Instrument {
  private:
    int numberOfKeys;
  public:
    WoodwindInstrument(const string& name, int year,
                       int numberOfKeys)
    : Instrument(name, year), numberOfKeys(numberOfKeys) {}
```

```
    int getNumberOfKeys() const { return numberOfKeys; }
    void play() const override { cout << "woodwind instrument: "
                                      << getName() << endl; }
};

/* File: BandMember.h */
class BandMember {
  private:
    const string name;
    const int age;
  public:
    BandMember(const string& name, int age)
    : name(name), age(age) {}
    virtual ~BandMember() = default;
    string getName() const { return name; }
    int getAge() const { return age; }
    virtual void perform() const {}
};
class WindBandMember : public BandMember {
  private:
    Instrument* instrument;                  // owned pointer!
  public:
    WindBandMember(const string& name, int age,
                   Instrument* instrument)
    : BandMember(name, age), instrument(instrument) {}
    ~WindBandMember() { delete instrument; }
    Instrument* getInstrument() const { return instrument; }
    void perform() const override {
      cout << "Wind band member " << getName();
      if(instrument) { cout << " plays a "; instrument->play(); }
      else cout << " plays nothing" << endl;
    }
};

/* File: WindBand.h */
class WindBand {
  private:
    WindBandMember** members;   // pointer to an array of
                                // WindBandMember pointers
    const int max_nums;         // maximum number of members
    int num_members;            // current number of members
  public:
    WindBand(int max_nums);
    // (a) conversion constructor
    WindBand(const WindBand& other);
    // (b) copy constructor: DEEP copy
    ~WindBand();                                // (c) destructor
    void addMember(WindBandMember* member);  // (d)
    void performAllMembers() const;          // (e)
};
```

The test program creates three members (John/Clarinet, Jane/Trumpet, Peter/`nullptr`), adds them to `windBand(5)`, calls `performAllMembers()`, makes `WindBand windBandCopy(windBand);`, then adds a 4th member (Ernest/Trombone) to the original only and performs both bands. **Solution** (in `WindBand.cpp`):

```
// (a): allocate the pointer array, zero it, reset the counter
WindBand::WindBand(int max_nums)
    : max_nums(max_nums), num_members(0) {
  members = new WindBandMember*[max_nums] {};
  // {} zero-initialises all pointers
}

// (b): DEEP copy --- every member AND
// its instrument must be cloned
WindBand::WindBand(const WindBand& other)
: max_nums(other.max_nums), num_members(other.num_members) {
  members = new WindBandMember*[max_nums] {};
  for (int i = 0; i < num_members; ++i) {
    // clone the instrument by discovering
    // its actual (dynamic) type:
    Instrument* instrumentCopy;
    const BrassInstrument* brassInstrument =
      dynamic_cast<BrassInstrument*>
      (other.members[i]->getInstrument());
    const WoodwindInstrument* woodwindInstrument =
      dynamic_cast<WoodwindInstrument*>
      (other.members[i]->getInstrument());
    if (brassInstrument)
      instrumentCopy = new BrassInstrument(*brassInstrument);
    else if (woodwindInstrument)
      instrumentCopy = new WoodwindInstrument(
        woodwindInstrument->getName(),
        woodwindInstrument->getYear(),
        woodwindInstrument->getNumberOfKeys());
    else
      instrumentCopy = nullptr;
    // then clone the member itself
    members[i] = new WindBandMember(
      other.members[i]->getName(),
      other.members[i]->getAge(), instrumentCopy);
  }
}

// (c): delete each member object, then the array of pointers
WindBand::~WindBand() {
  for (int i = 0; i < num_members; ++i)
    delete members[i];          // NOT delete [] members[i]
  delete [] members;            // NOT delete members
}

// (d): store the pointer, bump the counter
void WindBand::addMember(WindBandMember* member) {
  if (num_members == max_nums)                  // optional guard
    { delete member; return; }
  members[num_members] = member;
  num_members++;
}

// (e): dynamic binding does the printing
void WindBand::performAllMembers() const {
```

```
  for (int i = 0; i < num_members; ++i)
    members[i]->perform();
}
```

**Why part (b) needs RTTI (the 10-point core).**

- The band stores `Instrument*`, but `Instrument` is an abstract class: `new Instrument(*ptr)` cannot compile, and `new BrassInstrument(*ptr)` would slice if the object were actually a woodwind. So the copy constructor must discover the dynamic type of each stored instrument and clone through the right derived class.
- `dynamic_cast<BrassInstrument*>(...)` performs a checked downcast: it returns a valid pointer if the dynamic type really is (or derives from) `BrassInstrument`, and `nullptr` otherwise — so the `if/else if` chain is a type dispatch. (`typeid(*ptr) == typeid(BrassInstrument)` is an acceptable alternative test.)
- **Shallow copies are wrong and leak/double-delete:** `members[i] = other.members[i];` or copying the member while sharing the `instrument` pointer leaves two `WindBandMembers` owning one `Instrument` — both destructors would `delete` it.
- Because each `WindBandMember` destructor does `delete instrument`, the deep-copied band owns its own instruments; adding Ernest to the original afterwards must not affect the copy (that is exactly what the sample run verifies).

## 3.4 Virtual Destructor / ABC / final / Three Kinds of Inheritance

### 3.4.1 Virtual Destructor (EXAM FOCUS!)

First, destruction of objects accessed through their own type works fine:

```
#include <iostream>       /* File: concrete-destructors.cpp */
using namespace std;
#include "v-student.h"

int main()
{
    UPerson* p = new UPerson("Adam", ECE);
    delete p;

    Student* s = new Student("Simpson", CSE, 3.8);
    s->add_course("COMP1021");
    s->add_course("COMP2012");
    delete s;
}
```

`delete p` calls `UPerson`'s destructor, and `delete s` calls `Student`'s destructor respectively. Everything works fine.

**Non-virtual destructor + polymorphic `delete` = memory leak + UB**

```
#include <iostream>       /* File: require-v-destructors.cpp */
using namespace std;
#include "v-student.h"

int main()
{
    Student* s = new Student("Simpson", CSE, 3.8);
    s->add_course("COMP1021");
    s->add_course("COMP2012");

    UPerson* p = s;
    delete p; // Can we call UPerson's destructor on a Student?
}
```

Here `p` actually points to a `Student` object. `delete p` calls the `UPerson`'s destructor, and not `Student`'s destructor. The behavior of destructing a derived class object by its base class destructor is undefined!

**If your class is ever inherited from, its destructor MUST be virtual**

The solution is again using dynamic binding, and making the destructors virtual:

```
class UPerson                          /* File: v-uperson2.h */
{
   public: virtual ~UPerson() = default;
   ...
};

class Student : public UPerson         /* File: v-student2.h */
{
   public:
     virtual ~Student()
```

```
    {
        for (int j = 0; j < num_courses; ++j)
            delete enrolled[j];
    }
  ...
};
```

Now, `delete p` correctly calls the `Student`'s destructor if `p` points to a `Student` object:

```
#include <iostream>     /* File: v-destructors.cpp */
using namespace std;
#include "v-student2.h" // With virtual destructor

int main()
{
    Student* s = new Student("Simpson", CSE, 3.8);
    s->add_course("COMP1021");
    s->add_course("COMP2012");

    UPerson* p = s;
    delete p;              // Actually call Student's destructor
}
```

When a class does not have a virtual destructor, it is a strong hint that the class is not designed to be used as a base class.

### 3.4.2 Order of Construction and Destruction with Polymorphism

**Without** virtual destructor — Question: What is the output?

```
#include <iostream> // construction-destruction-order.cpp
using namespace std;

class Base
{
  public:
    Base() { cout << "Base's constructor\n"; }
    ~Base() { cout << "Base's destructor\n"; }
};

class Derived : public Base
{
  public:
    Derived() { cout << "Derived's constructor\n"; }
    ~Derived() { cout << "Derived's destructor\n"; }
};

int main()
{
    Base* p = new Derived;
    delete p;
}
```

**Answer.**

```
Base's constructor
Derived's constructor
Base's destructor
```

**Solution.** Construction always runs base first, then derived. Destruction is the reverse — but only if the destructor is **virtual**. Here `p` has static type `Base*` and `~Base()` is **not** virtual, so `delete p` resolves the call **statically** to `Base`'s destructor. Only the `Base` part of the object is destroyed; the `Derived` sub-object is never destructed, so any resources owned by `Derived` leak. (The memory is freed, but the derived destructor body never runs.) This is exactly why a class meant to be used polymorphically must declare a virtual destructor.

**With** virtual destructor — Question: What is the output when virtual destructors are used?

```
#include <iostream>       // construction-v-destruction-order.cpp
using namespace std;

class Base
{
   public:
     Base() { cout << "Base's constructor\n"; }
     virtual ~Base() { cout << "Base's destructor\n"; }
};

class Derived : public Base
{
   public:
     Derived() { cout << "Derived's constructor\n"; }
     virtual ~Derived() { cout << "Derived's destructor\n"; }
};

int main()
{
    Base* p = new Derived;
    delete p;
}
```

**Answer.**

```
Base's constructor
Derived's constructor
Derived's destructor
Base's destructor
```

**Solution.** Now `~Base()` is virtual, so `delete p` performs **dynamic dispatch**: the most-derived destructor (`Derived`'s) is invoked first, and at the end of it the compiler automatically calls the base destructor. Hence destruction order is exactly the reverse of construction — the extra `Derived's destructor` line appears, and no resources leak. Note that once `~Base()` is virtual, `~Derived()` is implicitly virtual too; the explicit `virtual` keyword there is only documentation.

### 3.4.3 Virtual Functions Called from Constructor / Destructor

```
#include <iostream>         /* File: construct-vf.cpp */
using namespace std;

class Base {
   public:
     Base() { cout << "Base's constructor\n"; f(); }
     virtual void f() { cout << "Base::f()" << endl; }
};
```

```
class Derived : public Base {
   public:
     Derived() { cout << "Derived's constructor\n"; }
     virtual void f() override { cout << "Derived::f()" << endl; }
};

int main() {
    Base* p = new Derived;
    cout << "Derived-class object created" << endl;
    p->f();
}
```

The output is:

```
Base's constructor
Base::f()
Derived's constructor
Derived-class object created
Derived::f()
```

**Do not rely on the virtual function mechanism during the execution of a constructor**

This is not a bug, but necessary — how can the derived object provide services if it has not been constructed yet?
Similarly, if a virtual function is called inside the base class destructor, it represents base class' virtual function: when a derived class is being deleted, the derived-specific portion has already been deleted before the base class destructor is called!

**2024S Midterm, Problem 1(d) [2 pts] — virtual calls inside constructors**

**Problem.** What is the output?

```
#include <iostream>
using namespace std;
class A {
public:
    A() { f(); }
    virtual void f() const { cout << "A::f" << endl; }
};
class B: public A {
public:
    B(): A() { f(); }
    void f() const { cout << "B::f" << endl; }
};
int main() {
    B obj;
    A& ref = obj;
    return 0;
}
```

**Answer.**

```
A::f
B::f
```

**Solution.** `B obj;` constructs the `A` part first:

- While `A::A()` runs, the object is still an A — the `B` part does not exist yet. The virtual mechanism dispatches according to the class currently under construction, so `f()` inside `A()`'s body calls `A::f` (exactly the trap warned about above).

- Once the base is done, `B::B()`'s body runs on a fully-formed-A-plus-B-under-construction object; the vtable is now `B`'s, so `f()` calls `B::f`.

**Takeaway:** during construction (and destruction), the dynamic type is the class whose constructor/destructor is running — never a more-derived class.

### 3.4.4 Abstract Base Classes (ABC) and Pure Virtual Functions

Let's design a system for maintaining our assets: stocks, bank accounts, real estate, cars, yachts, etc. Each asset has a net worth (monetary value). We would like to be able to make listings and compute the total net worth. There are different kinds of assets, and they are all derived from `Personal_Asset`.

```
class Personal_Asset    /* File: personal-asset.h */
{
  public:
    Personal_Asset(const string& date) : purchase_date(date) { }
    void set_purchase_date(const string& d);
    virtual double compute_net_worth() const; // Current net worth
    virtual bool is_insurable() const;
    // Can this asset be insured?

     private:
       string purchase_date;
};

class Bank_Acc_Asset : public Personal_Asset
/* File: bank-acc-asset.h */
{
  public:
    Bank_Acc_Asset(const string& d, double m, double r = 0.0)
      : Personal_Asset(d), balance(m), interest_rate(r) { }
    virtual double compute_net_worth() const override
        { return balance; }

     private:
       double balance;
       double interest_rate;
};
```

There can be other classes of assets such as `Car_Asset`, `Securities_Asset`, `House_Asset`, etc. One may compute the total asset value for an array of different kinds of assets as follows:

```
/* File: compute-assets.cpp */
double compute_total_worth(const Personal_Asset* asset[],
                           int num_assets)
{
    double total_worth = 0.0;

    for (int i = 0; i < num_assets; i++)
        total_worth += assets[i]->compute_net_worth();

    return total_worth;
}
```

Now we have to implement the member functions of the base class `Personal_Asset`. How to implement `Personal_Asset::compute_net_worth()`? It depends completely on

the actual type of asset. There is no "standard way" of doing it!

```
/* File: personal-asset.cpp */
Personal_Asset::Personal_Asset(const string& date)
     : purchase_date(date) { }

void Personal_Asset::set_purchase_date(const string& date)
     { purchase_date = date; }

double Personal_Asset::compute_net_worth() const
     { return /* What? */ }
```

The truth is: It makes no sense to have objects of type `Personal_Asset`. Such an object has only a purchase date, but otherwise no meaning. We cannot implement the `compute_net_worth()` method in the base class as the information needed to implement it is missing. However, we don't want to remove the method because that would make a polymorphic function like `compute_total_worth()` impossible.

### 3.4.5 Solution: Abstract Base Class (ABC)

The solution is to make `Personal_Asset` an abstract base class (ABC), and `compute_net_worth()` now becomes a pure virtual function.

```
class Personal_Asset     /* File: personal-asset-abc.h */
{
  public:
    Personal_Asset(const string& date) : purchase_date(date) { }
    void set_purchase_date(const string& d);
    virtual bool is_insurable() const; // Can this asset be insured?

      // A pure virtual function to compute the current net worth
      virtual double compute_net_worth() const = 0;

    private:
      string purchase_date;
};

Personal_Asset p_asset("1997/07/01");         // Error
Bank_Acc_Asset b_asset("2000/01/01", 100.0); // Ok
```

**Two properties of an ABC (EXAM FOCUS!)**

1. No objects of ABC can be created.
2. Its derived classes **must implement the pure virtual functions**, otherwise they will also be ABC's.

If a derived class, e.g., `Securities_Asset`, does not implement the pure virtual functions, then the derived class is also an ABC, and there cannot be objects of that type, but it can be used as a base class itself, for instance for `Stocks_Asset`, `Bonds_Asset`, etc.

### 3.4.6 ABC as an Interface

An abstract base class provides a uniform interface to deal with a number of different derived classes. A base class contains what is common about several classes. If the only thing that is common is the interface, then the base class is a "pure interface", called ABC in C++.

We discussed before that code re-use is an advantage of inheritance. For ABC's, we do not re-use code, but create an interface that can be re-used by its derived classes.

Interfaces are the soul of object-oriented programming. They are the most effective way of separating the use and implementation of objects. The user (of `compute_total_worth()`) only knows about the abstract interface, objects from different derived classes of the ABC may implement the interface in different ways.

### 3.4.7 Final Remarks on ABC

A pure virtual function is inherited as a pure virtual function by a derived class unless it implements the function. An abstract base class cannot be used:

- as an argument type that is passed by value
- as a function return type that is returned by value
- as the type of an explicit conversion

However, pointers and references to an ABC can be declared. Calling a pure virtual function from the constructor of an ABC is undefined — don't do that.

```
#include <string>          /* File: can-and-cant.cpp */
using namespace std;

#include "personal-asset-abc.h"
#include "bank-acc-asset.h"

Personal_Asset x("20010/01/01");
// Error: can't create ABC object
Personal_Asset f1(int x) { /* .. */ }
// Error: can't return ABC object
int f2(Personal_Asset x) { /* .. */ }
// Error: can't CBV with ABC object

Bank_Acc_Asset b("01/01/2000", 0.0);          // OK!
Personal_Asset* p_asset_ptr = &b;             // OK!
Personal_Asset& p_asset_ref = b;              // OK!

Personal_Asset* f3(const Personal_Asset& x) { /* incomplete */ }
// OK!
```

### 3.4.8 C++11 `final`: Two Separate Uses

**Use (1) — on a class: forbid further inheritance**

```
#include <iostream>           /* File: final-class-error.cpp */
using namespace std;

class A {};
class B: public A {};
class C final: public B {};
class D: public B {};
class E: public C {};

int main()
{
    A a; B b; C c; D d; E e;
    return 0;
}
```

No sub-classes can be derived from a `final` class.

Another example: No `PG_Student` if `Student` class is `final`:

```
#include <iostream>          /* File: pg-final-error.cpp */
using namespace std;

class UPerson { /* incomplete */ };
class Student final : public UPerson { /* incomplete */ };
class PG_Student : public Student { /* incomplete */ };

int main()
{
    UPerson abby("Abby", CBME);
    Student bob("Bob", CIVL, 3.0);
    PG_Student matt("Matt", CSE, 3.8);
}
```

**Use (2) — on a virtual function: forbid further overriding**

```
#include <iostream>         /* File: final-vfcn-error.cpp */
using namespace std;

class UPerson {
public: /* Other data and functions */
    virtual void print() const { /* incomplete */ }
};

class Student : public UPerson {
public: /* Other data and functions */
    virtual void print() const override final { /* incomplete */ }
};

class PG_Student : public Student {
public: /* Other data and functions */
    virtual void print() const override { /* incomplete */ }
};

int main() { PG_Student jane("Jane", CSE, 4.0); jane.print(); }
```

### 3.4.9 Three Kinds of Class-Wide Inheritance Access

So far we have only used `class Derived : public Base`. Changing the keyword after the colon **downgrades the access level of everything inherited from Base** by a uniform amount:

| **Base member ↓** | After `public` | After `protected` | After `private` |
|---|---|---|---|
| Base `public` | `public` | `protected` | `private` |
| Base `protected` | `protected` | `protected` | `private` |
| Base `private` | inaccessible | inaccessible | inaccessible |

**Semantics of the three inheritance styles**

- **Public inheritance (99% of all production code): is-a**. A `Student` is substitutable everywhere a `UPerson` is expected. External code can implicitly upcast `Student*` → `UPerson*`; slicing is also expressible.
- **Protected inheritance:** "implemented in terms of". Not is-a; external code cannot perform the upcast (it is ill-formed) (so slicing disabled too). However, descendants of the derived class (grandchildren of the original Base) can still access

Base's protected members as their own protected heritage.
- **Private inheritance:** strongest "implemented in terms of". Even grandchildren are blocked from Base's contents, and external upcasting remains forbidden. Semantically very close to **composition ("Has-a" via a member object)**, with the single distinction that private inheritance grants access to the ancestor's *protected* members, which plain composition does not.

The key observable consequence of protected/private inheritance for COMP2012 exams:

```
#include <string>              /* File: no-slicing.cpp */
using namespace std;

// class Student: protected UPerson { ... }
#include "protected-student.h"

int main()
{
    Student ug("UG", ECE, 3.0);
    UPerson p = ug;      // Allowed or not?
    UPerson* q = &ug;    // Allowed or not?
    UPerson& r = ug;     // Allowed or not?
    return 0;
}
```

**Quiz: Why does the protected/private-inheritance compiler error mention `const Student&`?**

Scenario: `Student` inherits from `UPerson` protected-ly. You write:

```
Student s;
UPerson p = s;   // triggers copy assignment
```

The error message cites `const Student&`. Why the `const` reference?

**Answer:** The canonical copy-assignment operator signature is `UPerson& UPerson::operator=(const UPerson& other)`. The compiler therefore tries to form an implicit conversion from the lvalue `s` (a `Student`) to `const UPerson&` so that it can bind to the assignment parameter. Protected and private inheritance explicitly **disable** the external-user upcast, so this conversion is ill-formed. The diagnostic refers to the `const`-qualified parameter type.

**When to use non-public inheritance?**

You will almost always be better served by **public inheritance (is-a)** or by **composition / Has-a via member objects (encapsulated reuse)**. Protected/private inheritance is an advanced technique reserved almost exclusively for:
- code that needs access to a base class's **protected** members (otherwise just use a member object), **and** simultaneously
- code that wishes to **hide the is-a relationship** from downstream users (preventing external upcast).

For COMP2012 exam-level mastery: learn the accessibility table above and remember that non-public inheritance blocks external upcasting (and thus object slicing) by external user code.

**2024S Midterm, Problem 4 [20 pts] — debugging (II): inheritance & polymorphism**

**Problem.** Same 115-line program as in the constructor section (reproduce the listing

from there). The remaining five errors:

#3 **Line 42:** `Button::display()` prints `name` directly, but `name` is a **private** member of the base class `Object`. Private members are invisible even to derived classes — it must go through `get_name()` (or the base would have to declare it `protected`).

#6 **Line 67:** `void display(int i=0) const override;` — `override` requires the signature to match a base virtual function, but `Object::display()` takes no parameter. Different parameter list = different function = `override` fails to compile. (It would also never be called polymorphically.)

#7 **Line 78:** `objs[i]->trigger(t);` — `objs[i]` is an `Object*`, and `Object` has no member named `trigger` (`trigger` exists only in the derived class `Button`). A base-class pointer can only call base-class members (or virtual functions).

#8 **Line 89:** `new Object("Obj1")` — `Object` is an **abstract base class** (it declares the pure virtual function `display() const = 0`), so no object of type `Object` may be instantiated, even dynamically.

#10 **Line 26** (equivalently visible at lines 65 or 113): `~Object() = default;` is not virtual, yet the program deletes derived objects (`Button`, `Frame`) through `Object*` pointers (`delete objs[i];`, `delete root;`). Deleting a derived object via a base pointer without a virtual destructor is **undefined behaviour** (in practice the derived destructor never runs and the `text`/`objs` arrays leak). Fix: `virtual ~Object() = default;`. With it, `delete root;` destroys the `Frame` recursively, so `child` (stored inside `root`) is deallocated too.

## 2025S Midterm, Problem 6 [25 pts] — task management: ABCs, `typeid` & `dynamic_cast`

**Problem.** Build a task system with five classes: `Task` (ABC) → `CodingTask` (ABC, adds `status`) → `FeatureImplement`/`Debug`; and `Task` → `DocumentationTask` directly. `main()` keeps a `Task* jobs[6]`, then (i) displays only the coding tasks, (ii) executes all, (iii) displays all. Expected output (excerpt):

```
Class: 16 FeatureImplement
Task: 1 Status: open Implementation of Feature Task 1
...
Implementing Feature Task Write classes
Debugging Feature Task 1
Writing Documentation for Task 1
...
Task: 1 Status: completed Implementation of Feature Task 1
...
Task: 5 Documentation for Task 1
```

**Solution.**

```
/* (a) Task.h --- fill in the two TODOs */
Task(const string& t) : taskID(t) {}
virtual ~Task() {}

/* (b) CodingTask.h */
class CodingTask : public Task {
  private:
    bool status;
  protected:
    void setStatus() { status = true; }
    // derived classes may set...
  public:
    CodingTask(const string& t) : Task(t), status(false) {}
```

```
    void displayInfo() const override {
      Task::displayInfo();   // reuse: "Task: <id>"
      cout << " Status: " << (status ? "completed" : "open");
    }
};

/* (c) FeatureImplement.h (with include guards) */
#ifndef FEATUREIMPLEMENT_H
#define FEATUREIMPLEMENT_H
class FeatureImplement : public CodingTask {
  private:
    string description;
  public:
    FeatureImplement(const string& t, const string& d)
    : CodingTask(t), description(d) {}
    void execute() override {
      cout << "Implementing Feature Task " << description;
      setStatus();
    }
    void displayInfo() const override {
      CodingTask::displayInfo();
      cout << " Implementation of Feature Task " << getID();
    }
};
#endif

/* (d) Debug.h */
class Debug : public CodingTask {
  private:
    FeatureImplement* Feature;
  public:
    Debug(const string& t, FeatureImplement* ta)
        : CodingTask(t), Feature(ta) {}
    void execute() override {
      cout << "Debugging Feature Task " << Feature->getID();
      setStatus();
    }
    void displayInfo() const override {
      CodingTask::displayInfo();
      cout << " Debugging for Feature Task " << Feature->getID();
    }
};

/* (e) DocumentationTask.h (needs #include "FeatureImplement.h") */
class DocumentationTask : public Task {        // NOT a CodingTask:
                                               // no status!
  private:
    FeatureImplement* referenceTask;
  public:
    DocumentationTask(const string& t, FeatureImplement* rt)
    : Task(t), referenceTask(rt) {}
    void execute() override {
      cout << "Writing Documentation for Task "
           << referenceTask->getID();
    }
```

```
    void displayInfo() const override {
      Task::displayInfo();
      cout << " Documentation for Task " << referenceTask->getID();
    }
};

/* (f) the two incomplete lines in main() */
if (dynamic_cast<CodingTask*>(job)) {
  cout << "Class: " << typeid(*job).name() << "\n";
```

**Key points.**

- **Two ABCs on purpose.** `Task` and `CodingTask` keep `execute()` pure virtual, so neither can ever be instantiated — only the three concrete leaves can.
- `protected` **mutator.** `setStatus()` is protected: `FeatureImplement`/`Debug` may flip the flag, external code cannot; `DocumentationTask` inherits from `Task` directly, so it has no status — matching the output where tasks 5/6 never print `Status:`.
- `displayInfo` **chaining.** Each override calls its parent's version with the scope operator (`Task::displayInfo()`, `CodingTask::displayInfo()`) and appends its own text.
- **Part (f) filter.** `dynamic_cast<CodingTask*>(job)` succeeds exactly for `FeatureImplement` and `Debug` (they are-a `CodingTask`) and returns `nullptr` for `DocumentationTask` — the idiomatic "is it one of these?" test without `dynamic_cast` exceptions.
- `typeid(*job).name()` must dereference: `typeid(job)` (a pointer) would report the static type `Task*` for every element. With GCC's mangling, `name()` prints `16FeatureImplement` / `5Debug` (length-prefixed) — exactly the expected output.

# 4 Generic Programming

## 4.1 Function Templates

### 4.1.1 Motivation: Copy-and-Paste Does Not Scale

Suppose we need a "larger of two values" function. Since C++ supports **function overloading**, we can write one version per type:

```
/* File: larger-overloads.cpp */
inline const int& larger(const int& a, const int& b)
    { return (a < b) ? b : a; }

inline const char& larger(const char& a, const char& b)
    { return (a < b) ? b : a; }

inline const double& larger(const double& a, const double& b)
    { return (a < b) ? b : a; }

#include <string>
inline const string& larger(const string& a, const string& b)
    { return (a < b) ? b : a; }

#include "teacher.h"
inline const Teacher& larger(const Teacher& a, const Teacher& b)
    { return (a < b) ? b : a; }
```

The five function bodies are **character-for-character identical** — only the type name changes. This "**copy-and-paste-and-modify**" style has three costs:

- **Maintenance:** every bug fix or improvement must be repeated in every copy. Forget one and the behaviour silently diverges.
- **Repetition:** the source file is bloated with near-duplicate code, which obscures what is actually different.
- **Extensibility:** a new type (say `Teacher`) requires yet another copy. The type writer must remember to do it.

The identical problem appears with **data structures**:

```
/* File: ll-node-overloads.h */
class ll_int_node {
  private: int data; ll_int_node* next;
  public: ll_int_node(const int&); ~ll_int_node();
    void ll_print() const; void ll_insert(const int&);
};

class ll_char_node {
  private: char data; ll_char_node* next;
  public: ll_char_node(const char&); ~ll_char_node();
    void ll_print() const; void ll_insert(const char&);
};

#include "student.h"
class ll_student_node {
  private: Student data; ll_student_node* next;
  public: ll_student_node(const Student&); ~ll_student_node();
    void ll_print() const; void ll_insert(const Student&);
};
```

The problem is worse here: overloading rescues functions, but there is **no overloading for class names** — one name such as `ll_node` cannot denote three classes, so we must invent three names (`ll_int_node`, `ll_char_node`, `ll_student_node`). A further consequence: a function written for `ll_int_node` cannot be reused for `ll_char_node` — the two types are unrelated.

What we want is to write the code **once**, with the manipulated type as a **parameter**:

- For `larger()`, all five versions share the same **general form**, where `T` stands for "whatever type is supplied":

```
inline const T& larger(const T&, const T&) { ... }
```

- This idea is called **generic programming**, and C++ supports it with **function templates** and **class templates**.
- Templates look like ordinary function and class definitions, except that the types they manipulate are **parameterized with type variables** (such as `T`).

**Generic programming in one sentence**

Write the algorithm **once**, **parameterize the types**, and let the compiler generate a concrete version for each type actually used. No more copy-paste-and-modify.

### 4.1.2 Function Template Syntax

The `larger()` overloads collapse into a single definition, introduced by the keyword **template**:

```
/* File: larger-template.h --- the function template definition */
template <typename T>
inline const T& larger(const T& a, const T& b)
{
    return (a < b) ? b : a;
}
```

The keyword `typename` may equivalently be written as `class`:

```
template <class T>
inline const T& larger(const T& a, const T& b)
{
    return (a < b) ? b : a;
}
```

Two points about this definition:

- The `<typename T>` part is called the **template parameter list**. `T` here is a **type parameter** — a placeholder for a type, not a variable and not an object.
- The listing above is only a **template definition** — a pattern for generating functions. It is **not itself a function**, it is never called directly, and **no machine code is generated for it on its own**. Code appears only when some translation unit actually uses it (see instantiation below).

**`typename` vs. `class` in template headers**

In `template <typename T>` and `template <class T>`, the two keywords are **fully interchangeable**. `typename` is the modern, clearer choice; `class` is kept for backward compatibility. Neither restricts `T` to be a class type — `T` can be `int`, `double`, a pointer, etc.

### 4.1.3 Instantiation, Argument Matching, and Explicit Instantiation

A template may be used with any types for which the body **makes sense**. Here the body needs <, so T must support <:

```
#include <iostream>      /* File: larger-calls.cpp */
using namespace std;
template <typename T> inline const T&
    larger(const T& a, const T& b) { return (a < b) ? b : a; }

int main()
{
    int x = 4, y = 8;
    cout << larger(x, y) << " is a bigger number!" << endl;

    string a("cheetah"), b("gorilla");
    cout << larger(a, b) << " is stronger!" << endl;
    return 0;
}
```

One template serves both calls. The first line is `8 is a bigger number!` — here `T` is `int`, and `larger(x, y)` returns a reference to `y`. The second is `gorilla is stronger!`, because `T` is `string` and `"cheetah" < "gorilla"` lexicographically. Nothing in `main()` mentions `T` — the compiler works it out.

- Based on the template definition, the compiler creates the code of **only** those functions that are **actually used (called)** in the program. This is called **template instantiation**.
- `T` in the template definition is the **formal parameter** of the template.
- For `larger-calls.cpp`, the compiler instantiates **two** `larger()` functions, substituting the **actual arguments** `int` and `string` for `T`:

```
template <typename T> inline const T&
larger(const T& a, const T& b) { return (a < b) ? b : a; }
```

Matching the arguments is the whole mechanism:

```
#include <iostream>      /* File: larger-match-arg.cpp */
using namespace std;
template <typename T> inline const T&
    larger(const T& a, const T& b) { return (a < b) ? b : a; }

int main()
{
    cout << larger(3, 5) << endl;      // T is int;
    cout << larger(4.3, 5.6) << endl;  // T is double
}
```

It prints `5` and then `5.6`. The compiler determines `T` purely from the argument types:

- When the compiler **instantiates a template**, it tries to determine the **actual type** of the template parameter by looking at the types of the **actual arguments** in a function call.
- If you call a template with **different types**, the compiler generates **separate instantiated function code** for each type, and the size of the final executable **increases** accordingly.

There is, however, **no automatic type conversion** for template arguments — unlike ordinary overloaded functions. So this is a **compile-time error**:

```
cout << larger(4, 5.5);
// Error: no matching function for call to 'larger(int, double)'
```

The compiler cannot deduce a single `T`: the arguments say `int` and `double`. To force a `double` comparison, name the type after the function name using `< >` — **explicit instantiation**:

```
cout << larger<double>(4, 5.5);   // T is forced to double
```

Now `4` is converted to `4.0` and the call is well formed. Equivalently, you could define a template with **two** type parameters (see below).

**No implicit conversion for template arguments**

Unlike ordinary (overloaded) functions, the compiler does **not** perform implicit type conversion when deducing template parameters. `larger(4, 5.5)` fails because `T` cannot be both `int` and `double`. Either use **explicit instantiation** `larger<double>(4, 5.5)`, or define a template with **two** type parameters.

Deduction also fails when the type parameter does not appear in the function's parameter list at all:

```
#include <iostream>      /* File: f-template-err.cpp */
using namespace std;
template <typename T> T* create() { return new T; };
template <typename T> void f() { T a; cout << a << endl; }
int main() { create(); f(); }

f-template-err.cpp:5:21: error: no matching function for call to
   'create()'
    int main() { create(); f(); }
                    ^
f-template-err.cpp:3:23: note: template argument
   deduction/substitution failed
f-template-err.cpp:5:21: note: couldn't deduce template parameter 'T'
    int main() { create(); f(); }
                    ^
f-template-err.cpp:5:26: error: no matching function for call to 'f()'
    int main() { create(); f(); }
                           ^
f-template-err.cpp:4:25: note: template argument
   deduction/substitution failed
f-template-err.cpp:5:26: note: couldn't deduce template parameter 'T'
```

The compiler **cannot deduce** `T` here, because `T` appears only in the **return type** (in `create()`) or only **inside the body** (in `f()`), and neither is consulted during argument matching. The calls must be written `create<int>()` and `f<int>()`.

### 4.1.4 When the General Rule Is Wrong: Template Specialization

Now apply `larger()` to C-strings:

```
#include <iostream>      /* File: template-problem.cpp */
using namespace std;

template <typename T> const T&
    larger(const T& a, const T& b) { return (a < b) ? b : a; }

int main()
{
    const char* m = "microsoft";
    const char* a = "apple";
```

```
    cout << larger(a, m) << " is better!" << endl;
    cout << larger(m, a) << " is better!" << endl;
    return 0;
}
```

**Isn't "microsoft" bigger than "apple" in alphabetical order?** Yes — but the program does not say that. Here `T` is deduced as `const char*`, so `a < b` compares the two **pointer addresses**, not the text. The output depends on where the compiler placed the literals in memory: not alphabetical order, and not even stable across recompiles.

**`const char*` compares addresses, not text**

For C-strings (`const char*`), the built-in `<` operator compares **pointer addresses**, not lexicographic order. To compare string contents, either use `std::string` (which overloads `<`) or `strcmp()`.

The general template is right for most types but wrong for this one. Instead of abandoning it, provide an **exception** for that type — a **template specialization**, marked by `template <>` (an empty parameter list):

```
#include <iostream>      // template-specialization-cstring.cpp
#include <cstring>
using namespace std;

/* General case  */
template <typename T>
const T& larger(const T& a, const T& b)
    { cout << "general case: "; return (a < b) ? b : a; }

/* Exceptional case  */
template <>
const char* const& larger(const char* const& a, const char* const& b)
    { cout << "special case: "; return (strcmp(a, b) < 0) ? b : a; }

int main()
{
    const char* m = "microsoft"; // Smaller address
    const char* a = "apple";     // Bigger address

    cout << larger(a, m) << " is better!" << endl;
    cout << larger(m, a) << " is better!" << endl;
    cout << larger(22, 88) << " is greater!" << endl;
    return 0;
}
```

The tag shows which version ran:

```
special case: microsoft is better!
special case: microsoft is better!
general case: 88 is greater!
```

Both `const char*` calls take the **special case** — thanks to `strcmp`, "microsoft" wins whichever argument comes first — while `larger(22, 88)` uses the **general case**. Note the specialized signature `const char* const&`: `const T&` with `T` = `const char*`, exactly as written above.

The same technique works for any type where `<` is the wrong criterion:

```
#include <iostream>      // template-specialization-student.cpp
using namespace std;
```

```
#include "student.h"

/* General case  */
template <typename T>
const T& larger(const T& a, const T& b)
    { cout << "general case: "; return (a < b) ? b : a; }

/* Exceptional case  */
template <>
const Student& larger(const Student& a, const Student& b)
    { cout << "special case: ";
      return (a.get_GPA() < b.get_GPA()) ? b : a; }

int main()
{
    Student a("Amy", ECE, 3.2);
    Student b("Bob", CSE, 4.2);

    cout << larger(a, b).get_name() << " is better!" << endl;
    cout << larger(b, a).get_name() << " is better!" << endl;
    cout << larger(22, 88) << " is greater!" << endl;
    return 0;
}
```

Assume `Student` has no `operator<`: the general case would not even compile for it. The specialization compares GPAs to return `const Student&`, so `get_name()` can be called on the result. Both calls print `Bob is better!` — the specialization returns the larger object either way — and the third line is again `general case: 88 is greater!`.

**Template specialization = exception to the general rule**

The **primary template** handles the common case (e.g. `larger<T>`). For types where the general logic is wrong (e.g. `const char*` needs `strcmp`, or `Student` needs `get_GPA()`), provide a **full specialization** with `template <>`. The compiler picks the most specific match.

### 4.1.5 Multiple Type Parameters and the Return-Type Problem

A template may take more than one type parameter, each introduced by its own keyword:

```
#include <iostream>      /* File: fcn-template-2arg.cpp */
using namespace std;

template <typename T1, typename T2> inline const T1&
    larger(const T1& a, const T2& b) { return (a < b) ? b : a; }

int main()
{
    cout << larger(4, 5.5) << endl; // T1 is int, T2 is double
    cout << larger(5.5, 4) << endl; // T1 is double, T2 is int
}
```

Now the arguments may differ in type, which is what `larger(4, 5.5)` needed. But `T1` comes from the **first** argument only, so the return type is `const T1&`. When `(a < b)` holds, `b` (type `T2`) is returned — so a `T2` value must be handed back as a `const T1&`. When the types differ, the compiler manufactures a **temporary** `T1` to bind the reference to, and warns:

```
fcn-template-2arg.cpp:5:47: warning: returning reference to local
   temporary
    object [-Wreturn-stack-address]
larger(const T1& a, const T2& b) { return (a < b) ? b : a; }
                    ~~~~~~~~~~~~~~~
fcn-template-2arg.cpp:9:13: note: in instantiation of function
   template
    specialization 'larger<int, double>' requested here
    cout << larger(4, 5.5) << endl; // T1 is int, T2 is double
            ^
fcn-template-2arg.cpp:5:47: warning: returning reference to local
   temporary
    object [-Wreturn-stack-address]
larger(const T1& a, const T2& b) { return (a < b) ? b : a; }
                    ~~~~~~~~~~~~~~~
fcn-template-2arg.cpp:10:13: note: in instantiation of function
   template
    specialization 'larger<double, int>' requested here
    cout << larger(5.5, 4) << endl; // T1 is double, T2 is int
            ^
2 warnings generated.
```

The note lines name the call that triggered each instantiation (`larger<int, double>` from line 9, `larger<double, int>` from line 10). **Can we return a `T2` value as a `T1&`?** Not safely — that temporary dies when the function returns, leaving a **dangling reference**. The warnings are real bugs.

The simplest fix is to return by **value**:

```
#include <iostream>      /* File: fcn-template-2arg-rbv.cpp */
using namespace std;

template <typename T1, typename T2> inline T1
    larger(const T1& a, const T2& b) { return (a < b) ? b : a; }

int main()
{
    cout << larger(4, 5.5) << endl; // T1 is int, T2 is double
    cout << larger(5.5, 4) << endl; // T1 is double, T2 is int
}
```

Each call now copies (or moves) the result out, so the temporary never outlives the reference. The output is `5` and `5.5`: in the first call `4 < 5.5`, so `b` = `5.5` is returned and converted to `T1` = `int`, giving `5`; in the second, `5.5 < 4` is false, so `a` = `5.5` is returned unchanged. The first call truncates deliberately: returning `T1` expresses the answer in the first argument's type.

An alternative avoids the mismatch by never returning the value — just print it:

```
#include <iostream>      /* File: fcn-template-2arg-print.cpp */
using namespace std;

template <typename T1, typename T2>
void print_larger(const T1& a, const T2& b)
{
    if (a > b)
        cout << a << endl;
    else
        cout << b << endl;
}
```

```
int main()
{
    print_larger(4, 5.5);
    print_larger(5.5, 4);
    return 0;
}
```

Inside the body, `a` and `b` keep their types `T1` and `T2`, so nothing is converted prematurely: both calls print `5.5`. A `void` return type is immune to the problem — there is no value to convert.

**Never return a reference to a local/temporary object**

When a template has multiple type parameters (e.g. `T1, T2`) and returns `const T1&`, returning a `T2` value forces creation of a **temporary** `T1` object. Returning a reference to that temporary is **undefined behavior** (dangling reference). Fix: return **by value** (`T1`), or restructure to avoid the type mismatch.

### 4.1.6 Return by Reference vs. Return by Value: Why the Temporary Bites

The underlying rule, with no templates involved: binding a **non-const** reference of one type to an object of another type is illegal:

```
#include <iostream>      /* File: convert-reference-err.cpp */
using namespace std;

int main()
{
    double x = 5.6;
    int& ip = x;          // Error: would need a temporary
    cout << ip << endl;
    return 0;
}
```

```
convert-reference-err.cpp:7:10: error: non-const lvalue reference to
    type 'int' cannot bind to a value of unrelated type 'double'
    int& ip = x;
         ^  ~
```

No `int` object exists here: binding `ip` would require a temporary `int` converted from `x`. A non-const reference must denote a real, modifiable object — otherwise writing through `ip` would silently modify a temporary — so the binding is forbidden.

A **const** reference may bind to such a temporary, because it cannot be used to modify it:

```
#include <iostream>      /* File: convert-reference.cpp */
using namespace std;

int main()
{
    double x = 5.6;
    const int& ip = x;
    cout << ip << endl;
    return 0;
}
```

This compiles and prints `5` — the truncated temporary, which `ip` refers to. In `main` it lives as long as the reference, so the program is safe. Inside a function the same rule is a trap:

```
#include <iostream>      /* File: larger-cond-statement.cpp */
using namespace std;

template <typename T1, typename T2>
inline const T1& larger(const T1& a, const T2& b)
{
    if (a < b)
        return b;
    else
        return a;
}

int main()
{
    cout << larger(4, 5.5) << endl; // T1 is int, T2 is double
    cout << larger(5.5, 4) << endl; // T1 is double, T2 is int
}
```

```
larger-cond-statement.cpp: In instantiation of const T1& larger(const
   T1&,
const T2&) [with T1 = int; T2 = double]:
larger-cond-statement.cpp:8:16: warning: returning reference to
   temporary
[-Wreturn-local-addr]
        return b;
        ^
larger-cond-statement.cpp: In instantiation of const T1& larger(const
   T1&,
const T2&) [with T1 = double; T2 = int]:
larger-cond-statement.cpp:8:16: warning: returning reference to
   temporary
[-Wreturn-local-addr]
        return b;
        ^
```

When the returned value has a different type from the declared `const T1&`, a **temporary variable** of type `T1` is created from `b` so that it can be returned by reference. That temporary lives only on `larger`'s **activation record**; on return the record is released, and `main` reads a temporary on freed stack space ⇒ **runtime error**! The warning appears only for `return b;` — the `return a;` branch is harmless, because `a` genuinely is a `T1`.

The conditional operator `?:` adds a twist when the branches have different types:

```
#include <iostream>      /* File: larger-cond-expression-rbcr.cpp */
using namespace std;

template <typename T1, typename T2>
inline const T1& larger(const T1& a, const T2& b)
    { return (a < b) ? b : a; }

int main()
{
    cout << larger(4, 5.5) << endl; // T1 is int, T2 is double
    cout << larger(5.5, 4) << endl; // T1 is double, T2 is int
}
```

- The expression `(a < b) ? b : a` can only produce **one single type**, but `a` and `b` have types `T1` and `T2`.
- The rule is that when `T1` and `T2` are different, the result of `?:` is an **rvalue** of the **common type** between them — in this case `double`, for both calls.
- So a temporary `double` is produced, and converting it to the declared return type `const T1&` creates **yet another** temporary — the same dangling-reference problem as before.

Switching from **RBR** (return by reference) to **RBV** (return by value) fixes it:

```
#include <iostream>      /* File: larger-cond-expression-rbv.cpp */
using namespace std;

template <typename T1, typename T2>
inline T1 larger(const T1& a, const T2& b)
    { return (a < b) ? b : a; }

int main()
{
    cout << larger(4, 5.5) << endl; // T1 is int, T2 is double
    cout << larger(5.5, 4) << endl; // T1 is double, T2 is int
}
```

The output is again `5` then `5.5`, with no warnings: each call copies the result into the caller, so no reference ever outlives the temporary it points to.

**RBR vs. RBV for multi-type templates**

When a function template has **different** parameter types (`T1`, `T2`) and you need to return one of them, **return by value** (`T1`), **not** by reference (`const T1&`). Returning by reference to a value of a different type forces a temporary and causes a dangling reference. Return by value copies (or moves) the result safely.

### 4.1.7 Code Bloat

Templates are not free: each distinct combination of template arguments produces its own compiled copy.

```
/* File: many-combinations.cpp */
short s = 1; char c = 'A';
int i = 1023; double d = 3.1415;
print_larger(s, s); print_larger(s, c);
print_larger(c, s); print_larger(s, i);
// ... And all other combinations; 16 in total.
```

Here `print_larger` has two type parameters drawn from four types, so:

- the compiler will **instantiate** a `print_larger()` for each of the **16 different combinations** of arguments;
- with current compiler technology this gives 16 (almost identical) fragments of code in the executable program — there is **no sharing of code** between them;
- so even a small source program may have a surprisingly **large binary size** if we are not careful.

**Code bloat = one instantiation per type combination**

Each distinct set of template arguments produces a **separate** compiled copy of the function/class, with **no code sharing**. A 2-parameter template called with 4 types each can generate up to **16** instantiations. Mitigate by reducing the number of type parameters or by factoring type-independent code into non-template helpers.

## 4.2 Class Templates

### 4.2.1 Defining a Class Template

The template mechanism works for classes too, which is particularly useful for **container classes** — those holding objects of the **same** kind, such as arrays, lists and sets. The linked-list node is the building block:

```
#ifndef LISTNODE_H      /* File: listnode.h */
#define LISTNODE_H

template <typename T>
class List_Node
{
  public:
    List_Node(const T& x) : data(x) { }

    T data;
    List_Node* next {nullptr};
};

#endif
```

Compared with the three hand-written node classes above, the differences are exactly the parameterization:

- The class name is followed by the template parameter list: `class List_Node`, then `<typename T>`.
- The stored data member is now `T data` instead of `int data` or `Student data`.
- The constructor parameter is `const T&`, and it uses a member initializer list to copy the item into `data`.
- `List_Node* next` is already implicitly `List_Node<T>* next` — **inside** the class template, the bare name `List_Node` means the current instantiation. (Outside the class you must write `List_Node<T>`.)
- `next {nullptr}` is a default member initializer, so a fresh node is always safe to traverse.

The container itself is also a class template:

```
#ifndef LIST_H          /* File: list.h */
#define LIST_H

#include "listnode.h"
template <typename T> class List
{
  public:
    List() = default;
    void prepend(const T& item) {
        List_Node<T>* new_node = new List_Node<T>(item);
        if (!head)
            head = new_node;
        else
        { new_node->next = head; head = new_node; }
    }
    void print() const {
        for (const List_Node<T>* p = head; p; p = p->next)
            cout << p->data << endl;
    }
    // ... Other member functions
```

```
  private:
    List_Node<T>* head {nullptr};
};

#endif
```

Member by member:

- `head` is a pointer to a `List_Node<T>`, starting as `nullptr`. The whole list is therefore homogeneous in `T` — a `List<int>` can never hold a `string`.
- `prepend` allocates a node with `new` and pushes it at the front. Note the two cases: an empty list (the new node becomes `head`) versus a non-empty list (splice the new node in front of the old head).
- `print` walks the chain with `p = p->next` until `nullptr`. The `cout << p->data` statement hides a requirement on `T`: it **must support** `operator<<` or this member will not compile.
- The definition is placed **in full inside the header** (see the separate-compilation rule at the end of this subsection).

### 4.2.2 Using a Class Template

The parameterized class now creates lists of any element type:

```
#include <iostream>      /* File: list-example.cpp */
using namespace std;
#include "list.h"

int main()
{
    List<char> letters; letters.prepend('a'); letters.prepend('b');
    cout << "*** print char list *** \n"; letters.print();

    List<int> primes;
    primes.prepend(2); primes.prepend(3); primes.prepend(5);
    cout << "### print int list ###\n"; primes.print();

    return 0;
}
```

The type argument is written explicitly at each definition, and each is a **separate class** with its own instantiated members. Because `prepend` inserts at the **front**, the output is in reverse insertion order:

```
*** print char list ***
b
a
### print int list ###
5
3
2
```

The compiler instantiated `List<char>` and `List<int>` from the one `List` template, and with them the two node classes — four concrete classes from two templates.

### 4.2.3 Nontype Parameters

Template parameters need not be types: a template may also have **nontype parameters**, ordinary values supplied at compile time. Here `max_num_items` is an `int`:

```
#include "listnode.h"   /* File: nontype-list.h */
template <typename T, int max_num_items>
class List {
  public:
    void prepend(const T& item) {
        if (num_items++ == max_num_items)
            cerr << "List is full; can't insert " << item << endl;
        else {
            List_Node<T>* new_node = new List_Node<T>(item);
            if (!head)
                head = new_node;
            else
            { new_node->next = head; head = new_node; }
        }
    }
    void print() const {
        for (const List_Node<T>* p = head; p; p = p->next)
            cout << p->data << endl;
    } // ... Other member functions
  private:
    int num_items {0};
    List_Node<T>* head {nullptr};
};
```

This is the same `List` with a capacity limit baked into the type. The guard increments `num_items` on **every** attempt — note the post-increment, which counts a rejected insert too — and reports failure on `cerr` instead of growing the list. Since the limit is part of the **type**, two lists with different limits are different classes.

```
#include <iostream>      /* File: nontype-list-example.cpp */
using namespace std;
#include "nontype-list.h"
#include "student.h"

int main()
{
    List<char, 5> letters;
    letters.prepend('a'); letters.prepend('b');
    cout << "*** print char list *** \n"; letters.print();

    const int N = 2; // non-type parameter must be a const expression
    List<int, N> primes;
    primes.prepend(2); primes.prepend(3); primes.prepend(5);
    cout << "### print int list ###\n"; primes.print();

    List<Student, 8> students;
    students.prepend(Student("James", CSE, 4.0));
    // Why don't we call students.print() ?
    return 0;
}
```

The arguments now include a **value** as well as a type. Two details:

- `N` must be a **constant expression** known at compile time. `const int N = 2;` qualifies; a plain `int N = 2;` would not, because the compiler needs the value to form the type.
- The `primes` list has room for 2 items but is given 3. The first two insertions succeed and the third prints `List is full; can't insert 5` to `cerr`, so the list prints `3` then `2`. Because `cerr` is unbuffered while `cout` is buffered, the error line may appear

earlier in your terminal than the surrounding `cout` output — the messages are correct, only the interleaving looks surprising.

- `students.print()` is **not** called, and the commented question is the reason: `print` contains a `cout <<` on the stored data, which requires an `operator<<` for `Student`. None exists, so the instantiated `print` would fail to compile. The `prepend` call is fine — copying a `Student` needs only its copy constructor. (The fix is either to define `operator<<` for `Student`, or to pass `print` a printing callback.)

Nontype parameters also capture sizes that would otherwise be lost:

```
#include <iostream>      /* File: array-size-by-template.cpp */
using namespace std;

// Here, x is a reference to an array of N objects of type T
template <typename T, int N>
int f(T (&x) [N]) { return N; }

int main()
{
    int a[] = {10, 11, 12, 13};
    double b[] = {0.0, 0.1, 0.2};
    bool c[] = {true, false};

    cout << f(a) << endl;
    cout << f(b) << endl;
    cout << f(c) << endl;
    return 0;
}
```

The parameter `T (&x) [N]` is a **reference to an array**: taking a reference (rather than a pointer) is what preserves the size, because an array decays to a pointer when passed by value. Both `T` and `N` are then **deduced** from the argument, giving `4`, `3` and `2` — the three array lengths. This idiom is how a template can "measure" a built-in array at compile time.

**2025S Midterm, Problem 3 [9 pts] — function template that measures arrays**

**Problem.** Write a function template `count_common_elements` so that the given program compiles and prints:

```
Number of common elements: 2
Number of common elements: 1
```

for the `int` arrays `{1,2,3,4,5}`/`{4,5,6,7,9}` and the `string` arrays `{"Dimitris","Pedro","Desmond","Cecia"}`/`{"Brian","Desmond"}`. Elements within each array are distinct; only the template may be added. **Solution.**

```
template <typename T, int N, int M>
int count_common_elements(T (&arr1)[N], T (&arr2)[M]) {
    int count = 0;
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < M; ++j) {
            if (arr1[i] == arr2[j]) {
                ++count;
                break;   // each element of arr1
                         // counts at most once
            }
        }
    }
    return count;
```

```
}
```

**Solution breakdown (how the 9 points are earned).**

- `template <typename T, int N, int M>` — one type parameter `T` plus two non-type (compile-time `int`) parameters; the compiler deduces all three from each call: `<int,5,5>` and `<string,4,2>`.
- `T (&arr1)[N]` — a **reference to an array of N Ts** (the parentheses are mandatory: `T &arr1[N]` would be "array of N references"). Passing by reference is what stops array-to-pointer decay, so `N` is deducible — the idiom shown above.
- Return type `int`; `int count = 0;` accumulator; `break` after a match because elements are distinct (and it would otherwise over-count when the same value repeats in `arr2`).
- No library functions are used, satisfying the "no helpers" rule of the exam.

### 4.2.4 Class Templates vs. Function Templates: Deduction and Separate Compilation

The two kinds of templates differ in one important respect — who supplies the template arguments.

For **function templates**, the compiler may **deduce** the template arguments from the function call:

```
int i = larger(4, 5);        // Rely on compilers to deduce larger<int>
int j = larger<int>(7, 2);   // Explicit instantiation
```

For **class templates**, you always have to specify the actual template arguments when creating objects; the compiler does **not** deduce them:

```
List primes;          // Error: how can compilers deduce the type?
primes.append(2);     // Error: too late; compilers can't lookahead!
```

The second line is the crux: deduction would have to wait for the first `append`, but the object needs a complete type at its definition.

**Class templates always need explicit arguments**

**Function templates**: the compiler **deduces** `T` from the call arguments (or you write `f<T>(...)` explicitly). **Class templates**: you **must** always write the arguments explicitly, e.g. `List<int>`. There is no deduction from subsequent usage.

The second difference is where the code lives. We normally put **declarations** in a header and **definitions** in a `.cpp` file. Applying that habit to templates fails:

```
/* File: larger.h */
template <typename T> inline const T&
    larger(const T& a, const T& b);

/* File: larger.cpp */
template <typename T> inline const T&
    larger(const T& a, const T& b) { return (a < b) ? b : a; }
```

A function/class template is **instantiated only when it is used**, and to substitute the actual type the compiler needs the **definition** available at that point. With the code above, the caller sees only the declaration, so it cannot instantiate anything, and the linker looks for a `larger<int>` that `larger.cpp` never produced (it had no use of its own to instantiate from).

## Template definitions go in the header

Put the template function/class **definitions** in the header file as well, and include the template header file in **every** file which uses the template. That is why `listnode.h` and `list.h` above contain complete definitions, not just declarations.

## 2025S Midterm, Problem 5 [25 pts] — `SportsPark`: class template over `T` and `MAX_FACILITIES`

**Problem.** Given the class template `Facility<T>` (name, capacity, type `T`, current occupancy; `displayInfo()`, `displayStatus()`, `updateOccupancy()`, getters) and the enum `FacilityCategory { INDOOR_POOL, GYM, YOGA_STUDIO }`, implement `SportsPark.h` so that `SportsPark<string,10>` and `SportsPark<FacilityCategory,5>` both work: add (up to the limit, else print `Maximum limit reached.`), remove by name (shift left, no leaks, else print `The facility is not found.`), display all, total occupancy, and find by name (return `Facility<T>*` or `nullptr`). **Solution.**

```
#include "Facility.h"
/* Filename: SportsPark.h */
template <typename T, int MAX_FACILITIES>
// type + non-type parameters
class SportsPark {
  private:
    Facility<T>* facilities[MAX_FACILITIES]{};
    // array of pointers, all nullptr
    int facilityCount;
  public:
    SportsPark() : facilityCount(0) {}
    ~SportsPark() {   // prevent memory leaks
      for (int i = 0; i < facilityCount; ++i)
        delete facilities[i];
    }
    void addFacility(string name, int capacity, T type) {
      if (facilityCount < MAX_FACILITIES) {
        facilities[facilityCount] =
            new Facility<T>(name, capacity, type);
        ++facilityCount;
      } else {
        cout << "Maximum limit reached." << endl;
      }
    }
    void removeFacility(string name) {
      for (int i = 0; i < facilityCount; ++i) {
        if (facilities[i]->getName() == name) {
          delete facilities[i];   // no leak:
                                  // delete BEFORE shifting
          for (int j = i; j < facilityCount-1; ++j)
            facilities[j] = facilities[j+1];
            // shift left to fill the gap
          --facilityCount;
          return;
        }
      }
      cout << "The facility is not found" << endl;
    }
    void displayAllFacilities() const {
```

```
    for (int i = 0; i < facilityCount; ++i) {
      facilities[i]->displayInfo();
      facilities[i]->displayStatus();
    }
  }
  int totalCurrentOccupancy() const {
    int total = 0;
    for (int i = 0; i < facilityCount; ++i)
      total += facilities[i]->getCurrentOccupancy();
    return total;
  }
  Facility<T>* findFacilityByName(string name) {
    for (int i = 0; i < facilityCount; ++i)
      if (facilities[i]->getName() == name)
        return facilities[i];
    return nullptr;
  }
};
```

**Key points.**

- `template <typename T, int MAX_FACILITIES>` mixes a type parameter with a non-type (`int`) parameter — the same two-parameter pattern as Problem 3, but at class level: the array size is a compile-time constant per instantiation.
- `Facility<T>* facilities[MAX_FACILITIES]{};` — the trailing `{}` value-initialises every pointer to `nullptr`, so the constructor need not loop.
- The destructor owns the elements: `delete` each object (never `delete[]` the element pointers individually), and since the array itself is a plain member (not `new`ed), no `delete[]` is needed for it.
- `removeFacility` deletes before shifting and shrinks `facilityCount` by one; skipping the delete leaks the `Facility`.
- `findFacilityByName` returns the raw pointer so `main` can chain `->updateOccupancy(...)`; note the enum `T` simply prints as an integer (`0`, `1`, `2`) inside `displayInfo()`.

## 4.3 Operator Overloading

### 4.3.1 Motivation: Operators Are Just Functions

To program the mathematical formula

$$c = 2(a - 3) + 5b$$

with named functions only, we would have to write

```
/* File: formula-calls.cpp */
c = add(multiply(2, subtract(a,3)), multiply(5,b));
```

which is hard to read because the nesting hides the structure of the expression. Most programming languages therefore provide **operators** that let us mimic the mathematical notation:

```
c = 2*(a - 3) + 5*b;
```

However, many languages define operators only for the **built-in types**. C++ is an exception: it lets you re-use **most**, but **not all**, of its operators and **re-define** them for new **user-defined types**, so you may give meaning to `+`, `-`, and so on for types such as `Vector`, `Matrix`, `Student` or `Word`.

**Operator overloading = syntactic sugar for functions**

Operators are just functions with a "nickname" (e.g. `+` for `operator+`). They let you write math-like syntax for user-defined types. They add **no** new capability — everything doable with an operator can be done with an ordinary named function.

The running example for the rest of this section:

```
using namespace std;       /* File: vector0.h */
class Vector
{
  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }
    double getx() const { return x; }
    double gety() const { return y; }
    void print() const { cout << "(" << x << ", " << y << ")\n"; }
  private:
    double x, y;
};
```

Watch the constructor: because both parameters have defaults, it also accepts **one** argument (`Vector(22)`), making it a **conversion constructor** — a `double` converts implicitly to a `Vector`. This fact explains most of the results below.

The first attempt uses an ordinary global function:

```
#include <iostream>       /* File: vector0-add.cpp */
#include "vector0.h"
Vector add(const Vector& a, const Vector& b)
    { return Vector(a.getx() + b.getx(), a.gety() + b.gety()); }

int main()
{
    Vector a(1, 3), b(-5, 7), c(22), d;
    d = add(add(a, b), c); d.print(); // d = a + b + c
}
```

`add` receives both operands by `const` reference and returns a **new** `Vector` by value (leaving the operands untouched). Here `a + b` = `(1-5, 3+7)` = `(-4, 10)`, and adding `c` = `(22, 0)` gives `(18, 10)` — the printed result. It works, but the call site is the unreadable nested form.

Replacing the **name** `add` with `operator+` gives the nice syntax:

```
#include <iostream>       /* File: vector0-op-add.cpp */
#include "vector0.h"

Vector operator+(const Vector& a, const Vector& b)
    { return Vector(a.getx() + b.getx(), a.gety() + b.gety()); }

int main()
{
    Vector a(1, 3), b(-5, 7), c(22), d;
    d = a + b + c; cout << "vector + vector: a + b + c = ";
    d.print();

    d = b + 1.0; cout << "vector + scalar: b + 1.0 = ";
    d.print();

    d = 8.2 + a; cout << "scalar + vector: 8.2 + a = ";
    d.print();
```

```
    return 0;
}
```

The body is unchanged from `add`. Output: `(18, 10)`, `(-4, 7)` and `(9.2, 3)` — **all three forms work with one non-member function**. **Why?** The scalar is converted: `b + 1.0` becomes `operator+(b, Vector(1.0, 0))` = `(-5+1, 7+0)` = `(-4, 7)`. Likewise `8.2 + a` becomes `operator+(Vector(8.2, 0), a)` = `(9.2, 3)`. Both orders work because a **free function** lets the conversion apply to either argument — a freedom a member function lacks (we will see that fail shortly).

The operator name is a real function name:

- `operator+` is a **formal function name** that can be used like any other function name.
- We could have called it in the formal way: `d = operator+(operator+(a, b), c);` — legal, but nobody would want to write that.
- Operator functions are just like ordinary functions, except that they **also** can be called with a nicer syntax similar to mathematical notation.
- So `+` has a **formal name** `operator+` (two keywords) and a "**nickname**" `+`. The formal name requires the call form `operator+(a, b)`; the nickname allows `a + b`.
- The nickname can **only** be used when **calling** the function. The formal name can be used in **any** context — declaring, defining, calling, or taking its address.

That notational freedom cuts both ways:

- Nothing stops you from coding `operator+` to perform subtraction.
- Nothing stops you from defining `operator+` and `operator+=` so that `a = a + b` and `a += b` mean different things.

Either trick makes your code unreadable. The rule:

**Don't shock the user!**

Finally, what an operator is cannot be changed. Almost all operators can be overloaded **except**

`.` `::` `?:` `.*`

and you can neither invent new ones nor change the following properties:

1. **Arity:** the number of arguments an operator takes.
   e.g. `!x` `x+y` `a%b` `s[j]`
   (So you are not allowed to re-define the `+` operator to take 3 arguments instead of 2.)
2. **Associativity:** e.g. `a+b+c` is always identical to `(a+b)+c`.
3. **Precedence:** which operator is done first?
   e.g. `a+b*c` is treated as `a+(b*c)`.

The C++ **parser is fixed**, so only existing operators can be re-defined; and since every operator already has a meaning for the built-in types, you may only add meanings for your **own** classes. Every operator function must therefore have **at least one argument** of a user-defined class type.

**What you CANNOT change about operators**

You can overload most operators for user-defined types, but you **cannot**:
1. overload `.`, `::`, `?:`, `.*`
2. define **new** operator symbols
3. change **arity**, **associativity**, or **precedence**
4. change the meaning of an operator for the **built-in** types

### 4.3.2 Member Operator Functions: `+` and `+=`

An operator function may be defined either as a **member function** of a class or as a global **non-member function**. As a global function, `operator+` has **2 arguments**,

and the expression `a + b` is equivalent to `operator+(a, b)`. As a member function it is invoked with dot syntax:

- If **a** is a `Vector`, the expression `a+b` is equivalent to `a.operator+(b)`.
- To call `operator+` as a member function, the class object must be the **left operand** (here **a**).
- Thus a member `operator+` of `Vector` has only **one** explicit argument — the first argument is **implicitly** the object on which the member function is invoked.
- This is the implicit `this` pointer at work. A member declared as

  ```
  Vector operator+(const Vector& b) const;
  ```

  is compiled into the following global function — the same two arguments, with `this` made explicit:

  ```
  Vector Vector::operator+(const Vector* this, const Vector& b);
  ```

Both `+` and `+=` are usually defined together:

```
#include <iostream>      /* File: vector-op-add.h */
class Vector
{
  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }
    double getx() const { return x; }
    double gety() const { return y; }
    Vector operator+(const Vector& b) const;
    const Vector& operator+=(const Vector& b);
  private:
    double x, y;
};

Vector Vector::operator+(const Vector& b) const
{   // Return by value; any copy constructor?
    return Vector(x + b.x, y + b.y);
}

const Vector& Vector::operator+=(const Vector& b)
{
    x += b.x; y += b.y;
    return *this; // Return by const reference. Why?
}
```

The return types answer the two comments in the code:

- `operator+` computes a **new** value and must not modify either operand, hence the trailing `const` on `this` and the **return by value**. A temporary `Vector` is built and returned; its copy constructor (the compiler-generated one) does the copying.
- `operator+=` **modifies** the left operand, so it returns `*this` by `const` reference — no new object is built, and a reference makes `a += b += c` and `(a += b) += c` work. It is returned as `const` reference to prevent bizarre expressions such as `(a += b) = c` from compiling.

```
#include "vector-op-add.h" /* File: vector-op-add-test.cpp */
using namespace std;

ostream& operator<<(ostream& os, const Vector& a)
{
    return (os << '('  << a.getx() << ", " << a.gety() << ')');
```

```
}

int main()
{
    Vector a(1.1, 2.2);
    Vector b(3.3, 4.4);

    cout << "vector + vector: a + b = " << a + b << endl;
    cout << "vector + scalar: b + 1.0 = " << b + 1.0 << endl;
    cout << "scalar + vector: 8.2 + a = " << 8.2 + a << endl; //Error

    a += b;
    cout << "After += : a = " << a << "  b = " << b << endl;
    return 0;
}
```

With `operator+` as a **member**, the first two lines still work but the third **does not compile**. The reason is the left operand: `a + b` becomes `a.operator+(b)` and `b + 1.0` becomes `b.operator+(Vector(1.0))`, both with a `Vector` on the left, but `8.2 + a` becomes `8.2 .operator+(a)` — `double` is not a class. Comment out that statement and the program prints `(4.4, 6.6)` then `(4.3, 4.4)`; after `a += b` it prints `a = (4.4, 6.6)` while `b` stays `(3.3, 4.4)` — `+=` modified only `a`.

### 4.3.3 Commutativity, Implicit Conversion, and the Left-Operand Trap

Whenever the compiler sees `a+b`, it considers **two** possible representations,

`operator+(a, b)` and `a.operator+(b)`

and checks whether one of them is defined. It is possible to define **both**, but then they must not be **ambiguous** in overload resolution.

We expect `+` to be **commutative**, so (vector + scalar) and (scalar + vector) should both work. But as a **member** function the **left operand** is **always** a `Vector`, so it covers only (vector + vector) and (vector + scalar). First the working direction:

```
#include "vector-op-add.h" /* File: vector-op-add-ok.cpp */
using namespace std;

ostream& operator<<(ostream& os, const Vector& a)
    { return (os << '('  << a.getx() << " , " << a.gety() << ')'); }

int main()
{
    Vector a(1.1, 2.2);
    cout << "vector + scalar: a + 5 = " << a + 5 << endl;
}
```

This direction **works**: the scalar on the **right** is converted by the conversion constructor. The call becomes the following, and prints `(6.1 , 2.2)`:

```
a + 5   ==>   a.operator+(Vector(5))
```

Now reverse the operands:

```
#include "vector-op-add.h" /* File: vector-op-add-error.cpp */
using namespace std;

ostream& operator<<(ostream& os, const Vector& a)
    { return (os << '('  << a.getx() << " , " << a.gety() << ')'); }
```

```
int main()
{
    Vector a(1.1, 2.2);
    cout << "scalar + vector: 5 + a = " << 5 + a << endl;
}

vector-op-add-error.cpp:10:46: error: no match for operator+
    (operand types are int and Vector)
    cout << "scalar + vector: 5 + a = " << 5 + a << endl;
                                          ~~~~~
```

- Isn't `operator+` commutative? Isn't `(5 + a)` equivalent to `(a + 5)`?
- We expect them to be, but `(5 + a)` is converted to `5.operator+(a)`, and `int` is **not a class** — there is no `operator+` member function for `int`, nor can we re-define one.
- Couldn't **5** be converted to a `Vector` by the conversion constructor, so that the result calls its `operator+` member function with `a` as argument?
- **No.** The compiler will not do that, even though a conversion constructor from `double` exists. Implicit conversion is never applied to the object on which a member function is invoked.

**No implicit conversion on the left operand of a member operator**

For a member `operator+`, the expression `5 + a` becomes `5.operator+(a)` — but `int` is not a class, so this fails. The compiler does **not** convert the left operand `5` to a `Vector`. Fix: provide a **non-member** `operator+(double, const Vector&)`.

Two fixes exist. The minimal one is a global function taking the scalar first and forwarding to the member:

```
Vector operator+(double a, const Vector& b) { return b + a; }
```

The **better** fix is the global non-member `operator+` taking two `Vector` arguments: since either operand may be converted, one function covers all three cases:

```
Vector operator+(const Vector& a, const Vector& b)
    { return Vector(a.getx()+b.getx(), a.gety()+b.gety()); }
```

The requirement is that `Vector` exposes enough public interface to compute the sum — here `getx()` and `gety()`.

### 4.3.4 Non-member `operator<<` and Cascading Output

So far a `Vector` was printed by calling `print`. An `operator<<` prints it like an `int`:

```
#include <iostream>      /* File: vector0-op-add-os.cpp */
#include "vector0.h"
using namespace std;

ostream& operator<<(ostream& os, const Vector& a)
    { return (os << '('  << a.getx() << ", " << a.gety() << ')'); }

Vector operator+(const Vector& a, const Vector& b)
    { return Vector(a.getx() + b.getx(), a.gety() + b.gety()); }

int main()
{
    Vector a(1.1, 2.2);
    Vector b(3.3, 4.4);
    cout << "vector + vector: a + b = " << a + b << endl;
```

```
    cout << "vector + scalar: b + 1.0 = " << b + 1.0 << endl;
    cout << "scalar + vector: 8.2 + a = " << 8.2 + a << endl;
    return 0;
}
```

The three printed vectors are `(4.4, 6.6)`, `(4.3, 4.4)` and `(9.3, 2.2)`. Three design points sit in the signature:

- The first parameter is `ostream&`, not `Vector`. `cout` and `cerr` are objects of the `ostream` class, which is the **base class** for all output streams; generalizing to `ostream` means the same function also works for `cerr`, string streams, and file streams.
- To allow the usual `cout << x` syntax with `cout` on the **left**, the `ostream` object must be the **first argument**. (A member function cannot do this — see the Friend subsection.)
- The function returns `ostream&`, which is what makes chaining work.

**Why return `ostream&`?** To **cascade** outputs in one statement:

```
Vector a(1, 0);
cout << " a = " << a << "\n";
```

The second line is equivalent to

```
operator<<(operator<<(operator<<(cout," a = "),a),"\n");
```

For that nesting to type-check, each inner `operator<<` must yield the same `ostream` object again. Returning **by value** would hand back a copy of the stream — and streams are not copyable at all — breaking the chain.

**Why `operator<<` returns `ostream&`**

To **cascade** outputs (`cout << a << b << endl`), each `operator<<` must return the same `ostream` object by **reference**. This chains the calls: `operator<<(operator<<(operator<<(cout,a),b),endl)`. Returning by value would break the chain.

**Could `operator<<` be a member function?** Syntactically yes, but the call order becomes wrong — see the next subsection.

### 4.3.5 Assignment: `operator=`

C++ **requires** `operator=` to be a member function. For a class without owned data the generated version usually suffices, but writing it out shows the two obligations — self-assignment and returning `*this`:

```
#include <iostream>      /* File: vector-op=.h */

class Vector
{
  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }
    const Vector& operator=(const Vector& b);
  private:
    double x, y;
};

const Vector& Vector::operator=(const Vector& b)
{
    if (this != &b) // Avoid self-assignment to save time
    {
        x = b.x;
```

```
        y = b.y;
    }
    return *this; // Why return const Vector& ?
};
```

The `this != &b` test skips the work when an object is assigned to itself (`a = a`), which would otherwise be wasted effort. Returning `*this` by `const` reference is what makes chained assignment `a = b = c` work, evaluated right to left as `a = (b = c)`; the `const` prevents `(a = b) = c` from being written. For a class like this one, where the members are plain `double`s, the generated memberwise assignment would have done the same thing.

Everything changes once the class owns heap memory:

```
class Word                      /* File: word.h */
{
  private:
    int freq; char* str;
    void setstr(const char* s)
        { str = new char[strlen(s)+1]; strcpy(str,s); }

  public:
    // The following str{nullptr} is necessary. Why?
    Word(const Word& w): str{nullptr} { cout << "Copy: "; *this = w; }
    Word(const char* s, int k = 1) : freq(k)
        { cout << "Conversion: from \"" << s << "\"\n"; setstr(s); }
    ~Word() { delete [] str; }

    const Word& operator=(const Word& w)
    {
        if (this != &w)
        {
            cout << "op= with " << w.str << endl;
            freq = w.freq; delete [] str; setstr(w.str);
        }
        return *this;
    }
};
```

Three resource-aware parts:

- `str` is a **raw pointer to heap memory**, so the class **owns** a resource. The destructor must `delete [] str`, and the class therefore must not be copied shallowly.
- `operator=` does a **deep copy**: it releases the old buffer (`delete [] str`) before allocating a fresh one for the incoming characters. Assigning the pointer directly would make two objects share (and later double-free) the same buffer.
- The copy constructor delegates its work to `operator=` (`*this = w`), since the two operations are otherwise identical. But that only works because `str` is initialized to `nullptr` **first**: `operator=` begins with `delete [] str`, and deleting an **uninitialized** pointer is undefined behaviour. Hence the comment's answer — `str{nullptr}` is not optional.

```
#include <iostream>       /* File: word-test.cpp */
#include <cstring>
using namespace std;
#include "word.h"

int main()
{
    Word ship("Titanic");            // Which constructor?
```

```
    Word movie(ship);                  // Which constructor?
    Word song("My heart will go on"); // Which constructor?

    song = song;                       // Call assignment operator
    song = movie;                      // Call assignment operator
}
```

The `cout` traces make this program a readable experiment:

```
Conversion: from "Titanic"
Copy: op= with Titanic
Conversion: from "My heart will go on"
op= with Titanic
```

Line by line: `ship` uses the **conversion constructor** (a `const char*`, not a `Word`); `movie(ship)` uses the **copy constructor**, which prints `Copy:` then calls `operator=` (printing `op= with Titanic`); `song` again uses conversion. Then `song = song` prints **nothing** — the `this != &w` guard rejected the self-assignment — while `song = movie` prints `op= with Titanic`. Note that only five output lines appear for six suspicious-looking statements; the self-assignment is the missing one.

- If a class contains **pointer data members** and **dynamic memory allocation** is required, the default memberwise assignment — **shallow copy** — is **not** adequate.
- The copy constructor and `operator=` should be implemented using **deep copy**, so that each object has its own copy of the **owned data**.
- Since the two usually do the same thing, one may be defined in terms of the other; here the copy constructor **calls** `operator=`.

**Owned pointer data ⇒ deep copy in copy ctor and `operator=`**

If a class owns heap memory (pointer members), the default **shallow copy** causes double-free, memory leaks, and aliasing bugs. Always implement **deep copy** in both the copy constructor and `operator=`, and free the old buffer in `operator=` before copying. Check for **self-assignment** (`this != &other`).

### 4.3.6 Indexing and Increment: `operator[]` and `operator++`

`operator[]` takes the index as its argument — never the class itself — and like `=` it must be a **member** function. A good version comes in **two** overloads differing in constness:

```
#include <iostream>       /* File: vector-op-index.h */
using namespace std;
class Vector {
  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }
    double operator[](int) const; // Read-only; c.f. getx() and gety()
    double& operator[](int);      // Allow read and write
  private:
    double x, y;
};
double Vector::operator[](int j) const {
    switch (j) {
        case 0: return x;
        case 1: return y;
        default: cerr << "op[] const: invalid dimension!\n"; } }

double& Vector::operator[](int j) {
    switch (j) {
        case 0: return x;
```

```
        case 1: return y;
        default: cerr << "op[]: invalid dimension!\n"; } }
```

The two overloads:

- The `const` version returns `double` **by value** — a copy. That is all a `const` object should be allowed to give out. It is the direct replacement for `getx()` and `gety()`.
- The non-`const` version returns `double&`, a reference into the object, so the caller can write `a[0] = 5.6`. Returning by value here would make `a[0] = 5.6` a compile error (you cannot assign to a temporary).
- The overloads are chosen by the **constness of the object**, not by the index expression — as the test program demonstrates.

```
#include "vector-op-index.h" /* File: vector-op-index-test.cpp */

// Replace getx(), gety() by op[]
ostream& operator<<(ostream& os, const Vector& a) // Which op[]?
{
    return (os << '('  << a[0] << " , " << a[1] << ')');
}

int main()
{
    Vector a(1.2, 3.4);
    cout << "Before assignment: " << a << endl;

    a[0] = 5.6; a[1] = 7.8;   // Which op[]?
    cout << "After assignment:  " << a << endl;

    a[2] = 9;                 // Which op[]? Error!
    return 0;
}
```

The three uses:

- Inside `operator<<`, the parameter is `const Vector& a`, so `a[0]` selects the **const** overload — the one returning `double`.
- In `main`, `a` is a non-const `Vector`, so `a[0] = 5.6` selects the **non-const** overload and assigns through the returned reference. Output: `(1.2 , 3.4)` then `(5.6 , 7.8)`.
- `a[2]` matches neither case label; the `default` branch prints `op[]: invalid dimension!` and then, as written, falls off the end of a function that must return a `double&`, which is **undefined behaviour**. The commented `Error!` is a reminder that the class should refuse an out-of-range index (`assert`, or `throw`) instead of printing a message and continuing.

Dropping either overload gives a characteristic error — the best argument for keeping both. With **only** the non-const version:

```
vector-op-index-test.cpp:6:28: error: no viable overloaded
    operator[] for type 'const Vector'
    return (os << '('  << a[0] << " , " << a[1] << ')');
                          ~~~
./vector-op-index.h:17:17: note: candidate function not viable:
    'this' argument has type 'const Vector', but method
    is not marked const
    double& Vector::operator[](int j) {
```

The note says it: the candidate exists, but `this` would be `const Vector` and the function is not marked `const` — so a `const Vector` could not be printed at all. With **only** the const version the other use breaks:

```
vector-op-index-test.cpp:14:10:
```

```
error: expression is not assignable
a[0] = 5.6; a[1] = 7.8; // Which op[]?
~~~~ ^
```

Here the returned `double` is a temporary, and a temporary cannot appear on the left of `=`.

**Two versions of `operator[]`: const & non-const**

A well-designed class provides **two** overloads of `operator[]`:

1. `T& operator[](int)` — non-const, returns a mutable reference (for `a[i] = x`)
2. `const T& operator[](int) const` — const, returns a read-only reference (for printing a `const` object)

Without the `const` version, you cannot index a `const` object. Without the non-const version, you cannot assign through the index.

`operator++` needs **the same name** for two different behaviours; C++ distinguishes them by a **dummy argument**:

```
class Vector {                  /* File: vector-op-incr.h */
  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }
    double operator[](int) const; // Read-only; c.f. getx() and gety()
    double& operator[](int);      // Allow read and write
    Vector& operator++();         // Pre-increment returns an l-value
    Vector operator++(int);       // Post-increment returns a r-value
  private:
    double x, y;
};

Vector& Vector::operator++() { ++x; ++y; return *this; }

// The dummy must be an int argument. Why is it needed?
Vector Vector::operator++(int)
{
    Vector temp(x,y);
    x++; y++; return temp;
}

/* Plus the operator[] function definitions not shown here */
```

- **Pre-increment** `++a` calls `operator++()`, which increments the object itself and returns `*this` by reference — an **l-value**, since `++a` denotes the object `a`.
- **Post-increment** `a++` calls `operator++(int)`, which must return the **old** value, so it saves a copy in `temp` first, then increments, then returns the copy by value — an **r-value**.
- The `int` parameter is a **dummy**: it is never used and its value is meaningless. It exists only because the two forms have the same name and the same (empty) parameter list otherwise; the compiler uses the presence of the `int` to tell the post-increment overload apart.
- The two return types must differ, matching the semantics: `Vector&` for pre (the object) and `Vector` for post (a snapshot).

```
#include <iostream>      /* File: vector-op-incr-test.cpp */
#include "vector-op-incr.h"
using namespace std;

ostream& operator<<(ostream& os, const Vector& a)
    { return (os << '('  << a[0] << " , " << a[1] << ')'); }
```

```
int main()
{
    Vector a(1.1, 2.2);
    Vector b(3.3, 4.4);
    Vector c;

    c = ++a;
    cout << "a = " << a << "\nc = " << c << endl;

    c = b++;
    cout << "b = " << b << "\nc = " << c << endl;
    return 0;
}
```

The output shows the difference:

```
a = (2.1 , 3.2)
c = (2.1 , 3.2)
b = (4.3 , 5.4)
c = (3.3 , 4.4)
```

For `c = ++a`, `a` is incremented before the assignment; `a` and `c` end up as `(2.1, 3.2)`. For `c = b++`, the copy of the old `b` is returned, so `c` receives `(3.3, 4.4)` while `b` itself has moved on to `(4.3, 5.4)`.

**Pre-increment vs. post-increment signatures**

- **Pre-increment** `++a`: `T& operator++()` — returns `*this` by reference (l-value).
- **Post-increment** `a++`: `T operator++(int)` — the `int` is a **dummy parameter**; returns the **old** value by value (r-value).

Pre-increment is more efficient (no copy); prefer `++a` over `a++` unless you need the old value.

### 4.3.7 Summary: Member or Non-member?

The choice follows from what must appear on the **left** of the operator:

- The operators `=` (assignment), `[ ]` (indexing), `( )` (call) are **required by C++** to be defined as class **member functions**.
- A member operator function has an **implicit first argument** of the class. Thus, if the **left operand** of an operator must be an object of the class, it can be a member function.
- If the **left operand** of an operator must be an object of other classes, it must be a **non-member function**. E.g. `operator<<`.
- For **commutative** operators like `+` and `*`, it is usually preferred to be defined as **non-member functions** to allow **automatic conversion** of types using the conversion constructors.

The standard library's `string` shows the last point:

```
string x("dot"), y("com"), z;
z = x + y;        // string + string
z = x + "com";    // string + const char*
z = "dog" + y;    // const char* + string
```

All three lines are legal. In the second, only the right operand needs converting (a `const char*` becomes a `string`); in the third, a `const char*` sits on the **left** — which would be impossible for a member `operator+`, exactly as `5 + a` was impossible for a member `Vector::operator+`. The library gets away with it by providing **non-member** overloads of `operator+`. Design your own classes the same way.

## 4.4 Friend Functions and Classes

### 4.4.1 Why `operator<<` Cannot Be a Member

Suppose we implement `operator<<` as a member function of `Vector`:

```
#include <iostream>      /* File: vector-os-nonfriend.h */

class Vector
{
  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }
    double getx() const { return x; }
    double gety() const { return y; }
    ostream& operator<<(ostream& os);

  private:
    double x, y;
};

ostream& Vector::operator<<(ostream& os)
{
    return (os << '('  << x << " , " << y << ')');
}
```

As a member, its **implicit first argument** is the `Vector`, so the `Vector` must sit on the **left** of `<<`. The test program comes out backwards:

```
#include <iostream>      /* File: vector-os-nonfriend.cpp */
using namespace std;
#include "vector-os-nonfriend.h"

Vector operator+(const Vector& a, const Vector& b)
    { return Vector(a.getx() + b.getx(), a.gety() + b.gety()); }

int main()
{
    Vector a(1.1, 2.2);
    Vector b(3.3, 4.4);
    Vector d = a + b;

    // Do you notice the strange output syntax?
    d << (cout << "vector + vector: a + b = ") << endl;
    (b + 1.0) << (cout << "vector + scalar: b + 1.0 = ") << endl;
    (8.2 + a) << (cout << "scalar + vector: 8.2 + a = ") << endl;
}
```

It compiles and prints the same three vectors, but the call sites are strange:

- `operator<<` is a **binary operator**. As a member function, the `Vector` object must be on the **left** of `<<` and `cout` on the right.
- To print a `Vector x` you now write `x << cout;`.
- Worse, cascading (`x`, `y`, `z`) nests as `z << (y << (x << cout));`. The usual order is `cout << x << y << z;`.
- The reason is again the left-operand rule: the compiler will never convert `cout` (an `ostream`) into a `Vector` to make the member call work.

So a **global non-member function** is the right choice here — but that raises two new issues:

1. A global function cannot access **private** data members, so the class must expose **public**

**assessor** functions such as `getx()` and `gety()` — notice that the header above only needs them because of this.
2. Non-member operators are then **less efficient**, because every access costs an extra function call. Worse, exposing accessors widens the public interface of the class just to make printing work.

One language feature removes both: **make the operator a friend.**

### 4.4.2 Friend Functions and Classes

A `friend` declaration inside a class grants a named external function (or another class) access to the class's **non-public** members, without making it a member:

```
#include <iostream>      /* File: vector-with-friends.h */
using namespace std;

class Vector
{
    friend ostream& operator<<(ostream& os, const Vector& a);
    friend Vector operator+(const Vector& a, const Vector& b);

  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }

  private:
    double x, y;
};

ostream& operator<<(ostream& os, const Vector& a)
    { return (os << '('  << a.x << " , " << a.y << ')'); }

Vector operator+(const Vector& a, const Vector& b)
    { return Vector(a.x + b.x, a.y + b.y); }
```

Compared with the two earlier headers, what has been gained:

- The `friend` declarations appear **inside** the class but the functions themselves are defined **outside** it — a friend is **not** a member function and has no `this` pointer. That is exactly why `operator<<` can now take `ostream&` as its **first** parameter and support `cout << a`.
- Because the two functions are now friends, they can read `a.x` and `a.y` **directly**. There is no need for `getx()` or `gety()` at all, so the public interface stays minimal and no extra accessor calls are made.
- The definitions are written in the header here for brevity, but a friend function may equally be defined in a `.cpp` file — the `friend` declaration in the class is what grants the privilege.

```
#include "vector-with-friends.h" /* File: vector-with-friends.cpp */

int main()
{
    Vector a(1.1, 2.2);
    Vector b(3.3, 4.4);

    // Now we get the usual output syntax
    cout << "vector + vector: a + b = " << a + b << endl;
    cout << "vector + scalar: b + 1.0 = " << b + 1.0 << endl;
    cout << "scalar + vector: 8.2 + a = " << 8.2 + a << endl;
    return 0;
}
```

The output syntax is natural again and all three expressions — `8.2 + a` included — work, since `operator+` is a non-member and either operand may be converted. This is the design to aim for: **friends plus non-member operators** — familiar syntax with direct access to private data.

Two rules of use:

- A class **X** may **grant** a function or another class friend status. Friend functions are **not** considered member functions, and **member access qualifiers** (`public`, `protected`, `private`) are **irrelevant** to them — where the declaration sits inside the class has no effect on access.
- Friend declarations may appear **anywhere** inside the class definition, but by convention they are placed **before all the members**, as above, to make the granted privileges easy to audit.
- Friends of **X** may access **all** data members of `X`, both public and non-public. So be careful! Likewise, all member functions of an `X`'s **friend class** can access **all** data members of `X`.

Four properties govern friendship (a favourite exam point):

**Four properties of friendship**

1. **Granted, not taken** — a class determines who its friends are during design; afterwards it cannot add more without **rewriting** the class definition. A function cannot declare itself a friend.
2. **Not symmetric** — if A is B's friend, B is not automatically A's friend.
3. **Not transitive** — if A is B's friend and B is C's friend, A is not automatically C's friend.
4. **Not inherited** — friends of a base class do not automatically become friends of its derived classes.

### 4.4.3 Example: Student and Hacker: When Friendship Goes Wrong

This example shows what "access **all** data members" really permits. First, a `Student` that declares a friend class:

```
#ifndef V_STUDENT_H      /* File: v-student.h */
#define V_STUDENT_H
#include "course.h"
#include "v-uperson.h"

class Student : public UPerson
{
    // No forward declaration of class Hacker is needed
    friend class Hacker; // Got a Hacker friend! Good luck!

  private:
    float GPA; Course* enrolled[50]; int num_courses;

  public:
    Student(string n, Department d, float x) :
        UPerson(n, d), GPA(x), num_courses(0) { }

    ~Student()
        { for (int j = 0; j < num_courses; ++j) delete enrolled[j]; }

    bool add_course(const string& s)
        { enrolled[num_courses++] = new Course(s); return true; };

    virtual void print() const
```

```
    {
        cout << "--- Student Details --- \n"
             << "Name: " << get_name()
             << "\nDept: " << get_department()
             << "\nGPA: " << GPA
             << "\n" << num_courses << " Enrolled courses: ";

        for (int j = 0; j < num_courses; ++j)
            { enrolled[j]->print(); cout << ' '; }

        cout << endl;
    }
};

#endif // V_STUDENT_H
```

Points to note:

- `Student` derives from `UPerson` and holds an array of `Course*` that it **owns** — the destructor deletes each enrolled course, and `num_courses` tracks how many slots are live.
- `GPA` is `private`, so normally only `Student`'s own members can touch it.
- `friend class Hacker;` grants all of that to `Hacker`. Note the comment: no forward declaration of `Hacker` is needed — a `friend class` declaration introduces the name itself.

```
#ifndef HACKER_H        /* File: hacker.h */
#define HACKER_H

#include "v-student.h"

class Hacker
{
  private:
    string name;

  public:
    Hacker(const string& s) : name(s) { }
    void add_course(Student& s) { s.GPA = 0.0; }    // Uh oh!!
};

#endif
```

The only interesting line is the one marked `// Uh oh!!`: `add_course` claims to enroll a course but writes `s.GPA = 0.0`, reaching straight into the private member of the `Student` it was handed. Nothing in `Student` stops it: friendship is total.

```
#include <iostream>      /* File: bad-friend.cpp */
using namespace std;

#include "v-student.h"
#include "hacker.h"

int main()
{
    Student freshman("Naive", CIVL, 4.0);
    Hacker cool_guy("$#%&");

    freshman.print();
    freshman.add_course("COMP2012");
```

```
    freshman.print();

    cool_guy.add_course(freshman);
    freshman.print();
    return 0;
}
```

The output shows the damage: first `GPA: 4` with `0 Enrolled courses`; then `GPA: 4` with `1 Enrolled courses: [COMP2012]`; then the same course but `GPA: 0`. No member function of `Student` ever ran to change the GPA, so no invariant could be checked, logged, or prevented. The student's GPA was simply overwritten from outside — by a class the `Student` author chose to trust.

**Friends bypass encapsulation — use sparingly**

A friend function/class can touch **all** private members of the granting class, silently breaking invariants (e.g. the Hacker sets `GPA = 0.0`). Friendship is a **design decision**, not a convenience hack. Prefer public member functions (getters/setters) unless you genuinely need the efficiency or expressiveness of direct private access.

# 5 Lambda Functions & Function Pointers

A **lambda expression** is an **anonymous function**. Having no name, it can only be used where it is written: either called immediately, or stored (in a variable, in a function pointer) and passed to templates such as STL algorithms.

## 5.1 Lambda Expressions

### 5.1.1 Syntax and Rules

```
[ <capture-list> ] ( <parameter-list> )
        mutable -> <ret-type> { <body> }
```

- Lambdas are anonymous — functions without a name. They are usually defined locally inside a function, though global lambdas are also possible.
- The **capture list** lets the lambda use local variables already defined in the enclosing function:
  - `[=]`: capture all local variables used, **by value**;
  - `[&]`: capture all local variables used, **by reference**;
  - `[variables]`: capture only the variables listed.
- **Global variables** can always be used in a lambda without being captured — it is an **error** to capture them.
- The **return type**:
  - is `void` by default if there is no `return` statement;
  - is automatically **inferred** if there is a `return` statement;
  - may be given explicitly with the `->` syntax.
- `mutable` is needed only when the body modifies a variable captured **by value**.

### 5.1.2 Example: Simple Lambdas with No Captures

```
#include <iostream>      /* File : simple-lambdas.cpp */
using namespace std;

int main()
{
    // A lambda for computing squares
    int range[] = { 2, 5, 7, 10 };
    for (int v : range)
        cout << [](int k) { return k * k; } (v) << endl;

    // A lambda for doubling numbers
    for (int& v : range) [](int& k) { return k *= 2; } (v);
    for (int v : range) cout << v << "\t";
    cout << endl;

    // A lambda for computing max between 2 numbers
    int x[3][2] = { {3, 6}, {9, 5}, {7, 1} };
    for (int k = 0; k < sizeof(x)/sizeof(x[0]); ++k)
        cout << [](int a, int b) { return (a > b) ? a : b; }
             (x[k][0], x[k][1]) << endl;

    return 0;
}
```

Output:

```
4
25
49
100
4   10   14   20
6
9
7
```

**What each loop demonstrates.**

- Loop 1: the lambda `[](int k) { return k*k; }` takes `v` **by value**, so `range` is untouched; the return type is inferred as `int`. Values printed: $2^2, 5^2, 7^2, 10^2$.
- Loop 2: the parameter is `int& k`, so the lambda modifies the array element in place ($2, 5, 7, 10 \to 4, 10, 14, 20$). The returned `int` (the new value) is discarded — the lambda is called purely for its **side effect**.
- Loop 3: the lambda returns the larger of two arguments; the call is written directly inside the output expression: `lambda(x[k][0], x[k][1])`. Prints $6, 9, 7$.

**A lambda with no capture is just a function**

`[](int k) { return k*k; }` behaves exactly like a named function `int square(int k)`; the only difference is that the lambda converts implicitly to a function pointer (see below).

### 5.1.3 Example: Lambdas with Captures

```
#include <iostream>      /* File : lambda-capture.cpp */
using namespace std;
int main()
{
    int sum = 0, a = 1, b = 2, c = 3;

    // Evaluate a quadratic polynomial
    for (int k = 0; k < 4; ++k)
        cout << [=](int x) { return a*x*x + b*x + c; } (k) << endl;
    cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;

    // a and b are used as accumulators
    for (int k = 0; k < 4; ++k)
        cout << [&](int x) { a += x*x; return b += x; } (k) << endl;
    cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;

    // Only variable sum is captured
    for (int v : { 2, 5, 7, 10 })
        cout << [=]() mutable { return sum += a*v; } () << endl;
    // Error! a is not in the capture list
    // cout << [&sum](int x) { return sum += a*x; } (v) << endl;
    cout << "sum = " << sum << endl;

    return 0;
}
```

Output:

```
3
6
11
```

```
18
a = 1   b = 2   c = 3
2
3
5
8
a = 15  b = 8   c = 3
30
75
105
150
sum = 0
```

**Step-by-step.**

- **Loop 1** `[=]` freezes `a`,`b`,`c` at $1, 2, 3$ when the closure is built (each iteration builds a fresh closure from the same values). Results $ak^2 + bk + c$: $3, 6, 11, 18$. The outer variables are unchanged $\Rightarrow$ `a = 1 b = 2 c = 3`.
- **Loop 2** `[&]` works on the outer `a` and `b`: `a += x*x` then `return b += x`. Trace: $(a, b) = (1, 2) \to (1, 2), (2, 3), (6, 5), (15, 8)$, printing the new `b`: $2, 3, 5, 8$. After the loop `a = 15`, `b = 8`, `c = 3` (never used).
- **Loop 3**: with `[=]` the lambda captures `a` ($= 15$) and `sum` ($= 0$) **by value**; `mutable` lets the body assign to its own copy of `sum`. Each of the four iterations creates a **new** closure, so the copy always restarts from the outer value 0: prints $15 \cdot 2, 15 \cdot 5, 15 \cdot 7, 15 \cdot 10 = 30, 75, 105, 150$, and the outer `sum` stays **0**. (The slide comment "only variable `sum` is captured" means only `sum` is written to; the value never flows back. Use `[&]` or `[&sum, a]` if an accumulator is wanted.)
- **The commented-out line** is the compiler error shown below: `[&sum]` captures only `sum`, yet the body also uses `a`, which is therefore not captured at all.

```
lambda-capture.cpp:16:47: error: variable 'a' cannot be
implicitly captured in a lambda with no capture-default specified
cout << [&sum](int x) { return sum += a*x; } (v) << endl;
                                          ^
```

**An explicit list is exhaustive**

With an explicit capture list, every local variable used in the body **must** be listed. Listing `[&sum]` does not implicitly bring in `a`. Fix: `[&, a]` or `[&sum, &a]`.

### 5.1.4 Question: When Are Values Captured?

```
#include <iostream>   /* File : lambda-value-binding.cpp */
using namespace std;
int main()
{
    int a = 1, b = 2, c = 3;
    auto f = [=](int x) { return a*x*x + b*x + c; };

    for (int k = 0; k < 4; ++k)
        cout << f(k) << endl;
    cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;

    a = 11, b = 12, c = 13;
    for (int k = 0; k < 4; ++k)
        cout << f(k) << endl;  // Will f use the new a, b, c?
```

```
    cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;

    return 0;
}
```

**Answer: no — `f` keeps using the old values.** The output is

```
3
6
11
18
a = 1   b = 2   c = 3
3
6
11
18
a = 11  b = 12  c = 13
```

**Reason:** a by-value capture copies the variable **once**, at the moment the closure is constructed (the `auto f = ...;` line), not when `f(k)` is called. Afterwards the closure and the outer variables are completely independent objects; assigning `a = 11` changes only the local `a` in `main`. The two loops therefore print identical numbers.

**`auto` — a course rule**

`auto` declares a variable whose type is inferred by the compiler. It is the only way to name a lambda's type (each lambda has a unique, unnameable closure type). **Warning: `auto` is not allowed in COMP2012 assignments/exams** — it is used here only to show lambda semantics.

### 5.1.5 Question: When Are References Captured?

```
#include <iostream>     /* File : lambda-ref-binding.cpp */
using namespace std;
int main()
{
    int a = 1, b = 2, c = 3;
    auto f = [&](int x) { a *= x; b += x; c = a + b; };

    for (int k = 1; k < 3; f(k++))
        ;
    cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;

    a = 11, b = 12, c = 13;
    for (int k = 1; k < 3; f(k++))  // Will f use the new a, b, c?
        ;
    cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;

    return 0;
}
```

**Answer: the printout is**

```
a = 2   b = 5   c = 7
a = 22  b = 15  c = 37
```

**Trace of the first loop** (note that the third expression `f(k++)` passes the old `k` and then increments it; the loop body is the empty statement `;`):

- `k=1`: condition true, body does nothing, then `f(1)`: $a = 1 \cdot 1 = 1$, $b = 2 + 1 = 3$, $c = 1 + 3 = 4$; `k` becomes 2.
- `k=2`: condition true, `f(2)`: $a = 1 \cdot 2 = 2$, $b = 3 + 2 = 5$, $c = 2 + 5 = 7$; `k` becomes 3.
- `k=3`: condition $3 < 3$ false $\Rightarrow$ loop ends.

**Second loop:** because the capture is **by reference**, the closure always reads/writes the current variables, so the new initial values are used: `f(1)` gives $(11, 13, 24)$, then `f(2)` gives $(22, 15, 37)$.

**Value vs. reference capture in one line**

By-value: a **snapshot** taken at closure creation. By-reference: a hidden **pointer** to the original variable, so the latest value is always seen.

### 5.1.6 Capture Semantics: Value, Reference, `mutable`

- Capture by value copies once, when the lambda is defined. It is like pass-by-value.
- **Unlike** pass-by-value, a by-value captured variable is **read-only inside the lambda** — unless the lambda is marked `mutable`, in which case the body may modify the closure's own copy (the outer variable is still untouched).
- Capture by reference is like pass-by-reference: the lambda sees and may modify the original.

```
/* File: mutable-lambda.cpp*/
int a = 1, b = 2;
// Error! a's copy is const inside a non-mutable lambda
cout << [a](int x) { return a += x; } (20) << endl;
// OK! mutable permits modifying the by-value copy
cout << [b](int x) mutable { return b *= x; } (20) << endl;
cout << "a = " << a << "\tb = " << b << endl;
```

Output: `21` is not printed — the first line does not compile. After the second line prints `40`, the last line prints `a = 1  b = 2`: both outer variables are intact, because `b *= x` multiplied the closure's **copy** ($2 \times 20 = 40$).

### 5.1.7 Mixing a Capture Default with Exceptions: `[&, a]`

```
#include <iostream>  /* File : mutable-lambda-with-return.cpp */
using namespace std;
int main()
{
    float a = 1.6, b = 2.7, c = 3.8;

    // [&, a]: all except a are captured by reference; a by value
    auto f = [&, a](int x) mutable ->int
             { a *= x; b += x; return c = a+b; };

    for (int k = 1; k < 3; ++k)
        cout << "a = " << a << "\tb = " << b << "\tc = " << c
             << "\tf(" << k << ") = " << f(k) << endl;

    cout << "a = " << a << "\tb = " << b << "\tc = " << c << endl;
    return 0;
}
```

Output (with the usual left-to-right evaluation of the `<<` chain):

```
a = 1.6 b = 2.7 c = 3.8 f(1) = 5
a = 1.6 b = 3.7 c = 5.3 f(2) = 8
a = 1.6 b = 5.7 c = 8.9
```

**Why these numbers.**

- `[&, a]` means: everything except `a` by reference, `a` by value. The explicit exceptions must come **after** `[=]` or `[&]`.
- `f(1)`: the copy of `a` becomes $1.6 \times 1 = 1.6$; the outer `b` becomes 3.7; the outer `c` becomes $1.6 + 3.7 = 5.3$; the declared return type is `int`, so 5.3 is truncated to `5`.
- `f(2)`: copy of `a` $= 3.2$ (invisible outside), `b` $= 3.7 + 2 = 5.7$, `c` $= 3.2 + 5.7 = 8.9$, returned as `8`.
- The final line shows the outer `a` untouched at 1.6 while `b`, `c` (captured by reference) keep their updated values.

**Declared return type truncates**

`->int` for a `float` computation silently truncates $5.3 \to 5$ and $8.9 \to 8$; it also shows that the closure's copy of `a` keeps growing $(1.6, 3.2)$ while the outer `a` never changes.

## 5.2 Function Pointers

Inherited from C, C++ allows a **function to be passed as an argument** to another function. Strictly speaking we pass the **function pointer**, i.e. the address of the function's code.

```
int square(int x) { return x*x; }
int main( )
{
    int (*fp) = square;
    ...
}
```

For the template function `larger()` studied in the generic-programming section,

```
template <typename T>
inline const T& larger(const T& a, const T& b)
{
    return (a < b) ? b : a;
}
```

the type of a pointer pointing to one instantiation is `inline const T& (*)(const T&, const T&)`.

### 5.2.1 Example: Choosing a Function at Run Time

```
#include <iostream>   /* File: fp-smaller-larger.cpp */
using namespace std;

int larger(int x, int y) { return (x > y) ? x : y; }
int smaller(int x, int y) { return (x > y) ? y : x; }

int main()
{
    int choice;
    cout << "Choice: (1 for larger; others for smaller): ";
    cin >> choice;

    int (*f)(int, int) = (choice == 1) ? larger : smaller;
```

```
    cout << (*f)(3, 5) << endl;
    return 0;
}
```

Reading `int (*f)(int, int)` from the inside out: `f` is a pointer to a function taking two `int` and returning `int`. `(*f)(3,5)` and `f(3,5)` are equivalent (the `*` is optional when calling). Output: `5` if the choice is 1, otherwise `3`.

### 5.2.2 Example: Array of Function Pointers — Calculator

```
#include <iostream>      /* File: fp-calculator.cpp */
using namespace std;
double add(double x, double y) { return x+y; }
double subtract(double x, double y) { return x-y; }
double multiply(double x, double y) { return x*y; }
double divide(double x, double y) { return x/y; }  // No check
int main()
{
    double (*f[])(double x, double y) // Array of function pointers
    = { add, subtract, multiply, divide };

    int operation; double x, y;
    cout << "Enter 0:+, 1:-, 2:*, 3:/, then 2 numbers: ";
    while (cin >> operation >> x >> y)
    {
        if (operation >= 0 && operation <= 3)
            cout << f[operation](x, y) << endl;  // Call + - * /
        cout << "Enter 0:+, 1:-, 2:*, 3:/, then 2 numbers: ";
    }
    return 0;
}
```

Here `f` is an **array of four function pointers**; indexing it selects the operation, exactly like a jump table. `divide` performs **no** error checking, so `x/0` yields `inf` (or a floating-point exception), not an error message.

### 5.2.3 Example: A Function Pointer Pointing to a Lambda

```
#include <iostream> /* File: fp-smaller-larger-lambda.cpp */
using namespace std;
int main()
{
    int choice;
    cout << "Choice: (1 for larger; others for smaller): ";
    cin >> choice;

    int (*f)(int, int);

    if (choice == 1)
        f = [] (int x, int y) { return (x > y) ? x : y; };
    else
        f = [] (int x, int y) { return (x > y) ? y : x; };

    cout << f(3, 5) << endl;
    return 0;
```

```
}
```

A **captureless** lambda has an implicit conversion to a function pointer, so it can be assigned to `f` and called through it.

**A function pointer cannot point to a capturing lambda**

A lambda that captures anything needs storage for the captured copies/references — it is a closure object, not a plain function, so there is no function address to take. Only **captureless** lambdas convert to function pointers. (For capturing lambdas the STL uses **function objects** / `std::function` instead — see the STL section.)

## 5.3 Nested Lambdas

```
#include <iostream>          /* File : nested-lambda.cpp */
using namespace std;
int main()
{
    int a = 1, b = 1, c = 1;

    auto f = [a, &b, &c]() mutable
    {
        auto g = [a, b, &c]() mutable   // Nested lambda
        {
            cout << a << b << c << endl;
            a = b = c = 4;
        };

        a = b = c = 3; g();
    };

    a = b = c = 2; f();
    cout << a << b << c << endl;
    return 0;
}
```

Output:

```
123
234
```

**Trace.** `f` captures `a` by value (frozen at `1`) and `b`, `c` by reference; then `main` sets all three to 2.

- Entering `f()`: its own `a` is 1, while `b`, `c` read the outer 2, 2.
- `g` is constructed **before** `a = b = c = 3` runs, so `g` freezes `a` (the enclosing copy, 1) and `b` (=2), and captures `c` by reference.
- `a = b = c = 3` sets `f`'s copy of `a` to 3 and the outer `b`, `c` to 3.
- `g()` prints 1, 2, 3 (no separators, so "123"). Its body `a = b = c = 4` modifies only its own copies of `a` and `b`, but `c` is the outer variable ⇒ outer `c` becomes 4.
- Back in `main`: `a` is still 2, `b` was set to 3 by `f`, `c` is 4 ⇒ "234".

**Quiz: capture `b` by value in `f` and by reference in `g`**

Change the lists to `f = [a, b, &c]` and `g = [a, &b, &c]`.

**Answer:** the printout becomes

```
133
```

```
244
```

**Why:** `b` is now `f`'s own copy. `g` (built before the assignment) freezes `a` = 1 and takes `b` by reference to `f`'s copy. After `a = b = c = 3`: `a` (f's copy) = 3, `b` (f's copy) = 3, outer `c` = 3, so `g` prints "1 3 3". Inside `g`, `a = 4` touches only `g`'s copy; `b = 4` writes through the reference into `f`'s copy; `c = 4` writes the outer `c`. `main`'s own `b` was never touched, so it is still 2: "244".

# 6 Trees, Binary Trees, and Binary Search Trees

## 6.1 Complexity Review: Big-O Notation

Analyzing an algorithm predicts the resources it requires: memory, communication bandwidth, and (usually most important) **computational time**. Although the content of an input may affect running time, the main consideration is normally the **input size** $N$:

- sorting: the number of items to be sorted;
- matrix multiplication: the total number of elements in the two matrices.

**Big-O**

$O(f(N))$ gives the **asymptotic upper bound** of how the running time grows with the input size $N$. A running time is written as $\alpha f(N) + \beta$; only $f$ matters asymptotically.

| $f$ | Name | $f$ | Name |
|---|---|---|---|
| $c$ | constant | $N \log N$ | — |
| $\log N$ | logarithmic | $N^2$ | quadratic |
| $\log^2 N$ | log-squared | $N^3$ | cubic |
| $N$ | linear | $2^N$ | exponential |

The growth is dramatic: for small $N$ all curves look alike, but for large $N$ an $O(N^2)$ or $O(2^N)$ algorithm becomes unusable while $O(\log N)$ stays cheap.

**Question: why do we need trees?**

**Question.** The linear access time $O(N)$ of linked lists is prohibitive for large amounts of data. Does there exist any simple data structure for which the **average** running time of most operations (search, insert, delete) is $O(\log N)$?

**Answer.** Yes — **trees**. A balanced tree stores $N$ items with height $O(\log N)$, and search/insert/delete all cost $O(\text{height})$. This part of the course builds: basic tree concepts, tree traversal, general binary trees, **binary search trees (BST)**, and balanced trees (AVL).

## 6.2 Tree Terminology

**Recursive definition**

A tree $T$ is a collection of **nodes** connected by **edges**.

- **Base case:** $T$ is empty.
- **Recursive case:** if not empty, $T$ consists of a **root** node $r$ and zero or more non-empty sub-trees $T_1, T_2, \ldots, T_k$.

| | |
|---|---|
| **Root** | the only node with no parent |
| **Parent / child** | every node except the root has exactly one parent; a node may have zero or more children |
| **Leaves** | nodes with no children |
| **Siblings** | nodes with the same parent |
| **Path** $n_1 \to n_k$ | a sequence $\{n_1, n_2, \ldots, n_k\}$ where $n_i$ is the parent of $n_{i+1}$ |
| **Length of a path** | the number of edges on it |
| **Depth of a node** | length of the unique path from the root to that node |
| **Height of a node** | length of the longest path from it to a leaf (all leaves have height 0) |
| **Height of a tree** | height of the root = depth of the deepest leaf |
| **Ancestor / descendant** | if a path goes from $n_1$ to $n_2$, then $n_1$ is an ancestor of $n_2$ and $n_2$ a descendant of $n_1$ ("proper" if $n_1 \neq n_2$) |

**Example 1: Unix directories**

A file system is a tree: `/` is the root, directories are internal nodes, files are leaves. The **depth** of a file is the number of directories on the way from `/`; the **height** of the tree is the longest such path. A directory cannot have two parents — that is exactly the "tree" restriction.

## 6.3 Binary Trees

A **generic binary tree** is a tree in which no node has more than two children. The height of an "average" binary tree with $N$ nodes is considerably smaller than $N$:

- **best case** (a well-balanced tree): height $O(\log N)$;
- **worst case** (a degenerate "chain"): height $N - 1$.

### 6.3.1 A Typical Implementation

```
#include <iostream>              /* File: btree.h */
using namespace std;

template <class T> class BTnode
{
public:
    BTnode(const T& x, BTnode* L = nullptr, BTnode* R = nullptr)
    : data(x), left(L), right(R) { }

    ~BTnode()
    {
        delete left;
        delete right;
        cout << "delete the node with data = " << data << endl;
    }

    const T& get_data() const { return data; }
    BTnode* get_left() const { return left; }
    BTnode* get_right() const { return right; }

private:
    T data;                  // Stored information
    BTnode* left;            // Left child
    BTnode* right;           // Right child
};
```

The destructor is **recursive for free**: `delete left` destroys the whole left sub-tree (each node deletes its own children), then the right sub-tree, then the node itself. This is why the class needs no separate `destroy()` function — and why it must be used carefully in the BST section below.

### 6.3.2 Traversals: `action` as a Function Pointer

The three depth-first orders differ only in where the visit (`action(root)`) is placed.

```
/* File: btree-inorder.cpp
 *
 * To traverse a binary tree in the order of:
 *    left subtree, node, right subtree
 * and do some action on each node during the traversal.
 */

template <class T>
void btree_inorder(BTnode<T>* root,
                   void (*action)(BTnode<T>* r)) // on r->data
{
    if (root)
    {
        btree_inorder(root->get_left(), action);
        action(root);
        btree_inorder(root->get_right(), action);
    }
}

/* File: btree-preorder.cpp
 *
 * To traverse a binary tree in the order of:
 *    node, left subtree, right subtree
 */

template <class T>
void btree_preorder(BTnode<T>* root,
                    void (*action)(BTnode<T>* r)) // on r->data
{
    if (root)
    {
        action(root);
        btree_preorder(root->get_left(), action);
        btree_preorder(root->get_right(), action);
    }
}

/* File: btree-postorder.cpp
 *
 * To traverse a binary tree in the order of:
 *    left subtree, right subtree, node
 */

template <class T>
void btree_postorder(BTnode<T>* root,
                     void (*action)(BTnode<T>* r)) // on r->data
{
    if (root)
```

```
    {
        btree_postorder(root->get_left(), action);
        btree_postorder(root->get_right(), action);
        action(root);
    }
}
```

**Why a function pointer?**

The traversal code knows how to walk the tree but not what to do at a node. Passing `void (*action)(BTnode<T>*)` delegates the operation (print, count, sum, modify, ...) to the caller — traversal and action are separated. Only **captureless** lambdas could also be passed here (a capturing lambda cannot become a function pointer).

### 6.3.3 Example 2: Tree Creation and Traversal

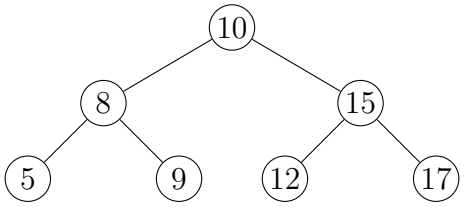
```
#include "btree.h"              /* File: test-btree.cpp */
#include "btree-preorder.cpp"
#include "btree-inorder.cpp"
#include "btree-postorder.cpp"

template <typename T>
void print(BTnode<T>* root)
{ cout << root->get_data() << endl; }

int main()                      // Build the tree from bottom up
{
    // Create the left subtree
    BTnode<int>* node5 = new BTnode<int>(5);
    BTnode<int>* node9 = new BTnode<int>(9);
    BTnode<int>* node8 = new BTnode<int>(8, node5, node9);
    // Create the right subtree
    BTnode<int>* node12 = new BTnode<int>(12);
    BTnode<int>* node17 = new BTnode<int>(17);
    BTnode<int>* node15 = new BTnode<int>(15, node12, node17);
    // Create the root node
    BTnode<int>* root = new BTnode<int>(10, node8, node15);

    cout << "\nInorder traversal result:\n";
    btree_inorder(root, print);
    cout << "\nPreorder traversal result:\n";
    btree_preorder(root, print);
    cout << "\nPostorder traversal result:\n";
    btree_postorder(root, print);
    cout << "\nDeleting the binary tree ...\n";
    delete root; return 0;
}
```

The tree is



Output of the three traversals:

```
Inorder traversal result:
5 8 9 10 12 15 17
Preorder traversal result:
10 8 5 9 15 12 17
Postorder traversal result:
5 9 8 12 17 15 10
```

Then `delete root` triggers the recursive destructors in postorder fashion:

```
delete the node with data = 5
delete the node with data = 9
delete the node with data = 8
delete the node with data = 12
delete the node with data = 17
delete the node with data = 15
delete the node with data = 10
```

**Example 3: Unix directory traversal**

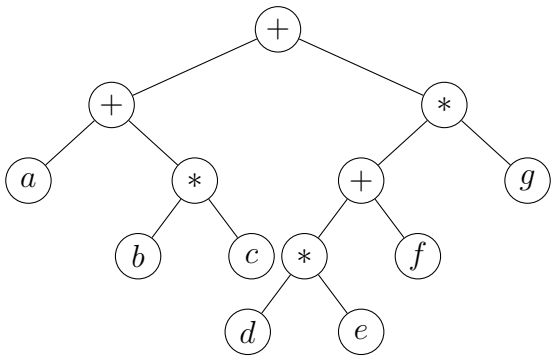
**Preorder** printing of a directory tree lists a directory before its contents — the natural "outline" order. **Postorder** computes something after the children are handled, e.g. the total size of a directory (sum of the sizes of all files below it). Inorder makes sense only for binary trees, so it is not used for general file systems.

### 6.3.4 Example 4: Expression Trees

An expression tree stores an expression: **leaves are operands** (constants or variables) and **internal nodes are operators** (unary or binary). For

$$(a + b * c) + ((d * e + f) * g)$$

the tree is




| | |
|---|---|
| Preorder | node, left, right ⇒ **prefix** notation: `+ + a * b c * + * d e f g` |
| Inorder | left, node, right ⇒ **infix** notation: `a + b * c + d * e + f * g` |
| Postorder | left, right, node ⇒ **postfix** notation: `a b c * + d e * f + g * +` |

**Infix loses the parentheses**

The inorder walk of the tree produces the operators in the right order but **drops all parentheses**; `a + b * c + d * e + f * g` re-parsed by normal precedence gives a different tree. Prefix and postfix need no parentheses at all — that is why compilers and stack machines use them.

## 6.4 Binary Search Trees

**BST property**

For **every** node $x$:
- all keys in the **left** sub-tree are **smaller** than the key of $x$;
- all keys in the **right** sub-tree are **larger** than the key of $x$.

A binary tree that violates this anywhere (e.g. a larger value in a left sub-tree) is not a BST, just a binary tree.

- BSTs are **not unique**: the same set of values can be stored in many different BSTs (e.g. insert $1, 2, 3$ in that order vs. in the order $2, 1, 3$).
- **Average** depth of a node: $O(\log N)$. **Maximum** depth: $O(N)$ (a degenerate chain built by inserting sorted data).

### 6.4.1 BST Search

Searching for $x$ compares $x$ with the root:

- equal $\Rightarrow$ done (found at the root);
- $x <$ root $\Rightarrow$ recursively search the **left** sub-tree;
- $x >$ root $\Rightarrow$ recursively search the **right** sub-tree.

Every comparison discards one whole sub-tree, so the cost is the length of the root-to-node path: $O(\text{height})$.

**Example 5: searching for 9**

In the tree of Example 2 (which is a valid BST):

| Step | Comparison | Move |
|---|---|---|
| 1 | $9 < 10$ | go left to 8 |
| 2 | $9 > 8$ | go right to 9 |
| 3 | $9 = 9$ | found |

Three comparisons instead of scanning all seven values. Searching for 11 would end at the `nullptr` child of 12 $\Rightarrow$ **not found**.

### 6.4.2 Our BST Implementation (different from the textbook's)

Instead of pointers, each `BSTnode` contains two `BST` objects as its sub-trees. This makes the code object-oriented: "a BST is a root node plus a left BST plus a right BST".

```
template <typename T> class BST /* File: bst.h */
{
private:
    struct BSTnode           // A node in a binary search tree
    {
        T value;
        BST left;            // Left sub-tree (= left child)
        BST right;           // Right sub-tree (= right child)
        // Assume a copy constructor for T
        BSTnode(const T& x) : value(x) { }
        // same as: BSTnode(const T& x)
        // : value(x), left(), right() { }
        BSTnode(const BSTnode&) = default;   // Copy constructor
        ~BSTnode() { cout << "delete: " << value << endl; }
    };
```

```
    BSTnode* root = nullptr;

public:
    BST() = default;                // Empty BST
    ~BST() { delete root; }         // Actually recursive
    // Shallow BST copy using move constructor
    BST(BST&& bst) { root = bst.root; bst.root = nullptr; }

    BST(const BST& bst)             // Deep copy via copy ctor
    {
        if (bst.is_empty())
            return;

        root = new BSTnode(*bst.root);   // Recursive
    }

    bool is_empty() const { return root == nullptr; }
    bool contains(const T& x) const;
    void print(int depth = 0) const;
    const T& find_max() const;      // Find the maximum value
    const T& find_min() const;      // Find the minimum value

    void insert(const T&);          // Insert an item with a policy
    void remove(const T&);          // Remove an item
};
```

Two design points to notice:

- `BSTnode(const BSTnode&) = default` is enough for a **deep** copy: `value` is copied, and the members `left`, `right` are `BST` objects, so their own copy constructors run recursively.
- The **move constructor** is a shallow copy: it steals the pointer and sets the source's `root` to `nullptr`. Because the source no longer owns anything, its destructor deletes nothing, so no double delete occurs.

### 6.4.3 BST Code: Search

```
/* Goal: To check if the BST contains the value x.
 * Return: (bool) true or false
 * Time complexity: O(height of BST)
 */

template <typename T>      /* File: bst-contains.cpp */
bool BST<T>::contains(const T& x) const
{
    if (is_empty())             // Base case #1
        return false;

    if (root->value == x)       // Base case #2
        return true;

    else if (x < root->value)  // Recursion on the left sub-tree
        return root->left.contains(x);

    else                        // Recursion on the right sub-tree
        return root->right.contains(x);
```

```
}
```

Both sub-trees are `BST` objects (not pointers), so a recursive call is simply:

```
return root->left.contains(x);
```

No null check is needed before the call, because an empty sub-tree is a valid empty `BST` whose `contains` immediately returns `false`.

### 6.4.4 BST Code: Print by Rotating the Tree 90 Degrees

```
/* Goal: To print a BST
 * Remark: The output is the BST rotated 90 degrees
 */

template <typename T>      /* File: bst-print.cpp */
void BST<T>::print(int depth) const
{
    if (is_empty())            // Base case
        return;

    root->right.print(depth+1);  // Recursion: right sub-tree

    for (int j = 0; j < depth; j++)  // Print the node value
        cout << '\t';
    cout << root->value << endl;

    root->left.print(depth+1);   // Recursion: left sub-tree
}
```

Printing the **right** sub-tree first, then the node, then the **left** sub-tree, and indenting by the depth, produces the tree lying on its side: the root at the left, larger values above it, smaller values below it.

### 6.4.5 Finding the Minimum and Maximum

```
/* Goal: To find the min value stored in a non-empty BST.
 * Remark: The min value is stored in the leftmost node.
 * Time complexity: O(height of BST)
 */

template <typename T>       /* File: bst-find-min.cpp */
const T& BST<T>::find_min() const
{
    const BSTnode* node = root;

    while (!node->left.is_empty())   // Look for leftmost node
        node = node->left.root;

    return node->value;
}

/* Goal: To find the max value stored in a non-empty BST.
 * Remark: The max value is stored in the rightmost node.
 * Time complexity: O(height of BST)
 */
```

```
template <typename T>       /* File: bst-find-max.cpp */
const T& BST<T>::find_max() const
{
    const BSTnode* node = root;

    while (!node->right.is_empty())  // Look for rightmost node
        node = node->right.root;

    return node->value;
}
```

**Answer to the "min/max" question**

The **minimum** is always the **leftmost** node (repeatedly go left: every left child is smaller), and the **maximum** the **rightmost** node. Both are iterative and cost $O(\text{height})$. The functions are defined only for a **non-empty** tree — calling them on an empty BST dereferences `nullptr`.

### 6.4.6 Insertion

Proceed down the tree exactly as in a search:

- if $x$ is found, do nothing (or update the satellite data attached to that key);
- otherwise insert $x$ at the **last spot** on the traversed path.

Example: inserting 13 into the tree of Example 2 goes $13 > 10 \to$ right, $13 < 15 \to$ left, $13 > 12 \to$ right, and attaches 13 as the right child of 12. Time complexity $O(\text{height})$.

```
template <typename T>       /* File: bst-insert.cpp */
void BST<T>::insert(const T& x)
{
    if (is_empty())             // Find the spot
        root = new BSTnode(x);

    else if (x < root->value)
        root->left.insert(x);  // Recursion on the left sub-tree

    else if (x > root->value)
        root->right.insert(x); // Recursion on the right sub-tree

    else                        // This line is optional; just for
        ;                       // illustration: x == root->value
}
```

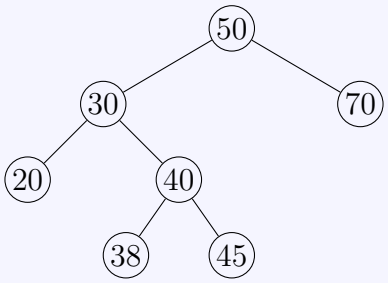
### 6.4.7 Deletion: the Three Cases

- **Leaf:** delete it immediately.
- **One child:** adjust the parent's pointer to **bypass** the deleted node.
- **Two children:** you have two choices — replace the deleted node with the
  - **maximum** node of its left sub-tree, or
  - **minimum** node of its right sub-tree.

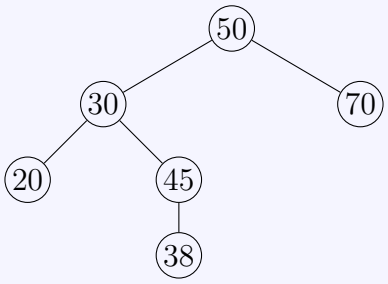
  Then remove the max/min node from that sub-tree (it has at most one child, so the first two cases apply).

**Examples 6.1–6.3: three deletions on one BST**

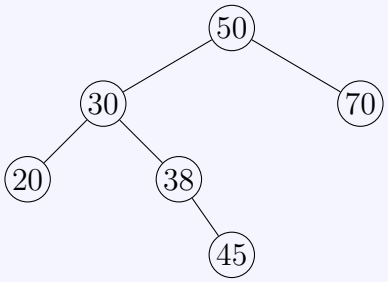
Start from



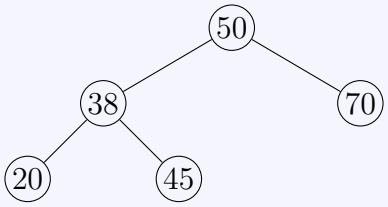

**6.1 Remove 40, replacing it with the minimum of its right sub-tree** (that minimum is 45). The node 45 is copied into the position of 40 and then unlinked from the lower level:




**6.2 Instead remove 40, replacing it with the maximum of its left sub-tree** (that maximum is 38):




**6.3 Now remove 30 from the second tree**, again two children, again the minimum of its right sub-tree (38) is promoted; the 45 hangs below it:




Either choice of replacement keeps the BST property; both cost $O(\text{height})$: at most one descent to find the replacement plus one to remove it.

### 6.4.8 BST Code: Deletion

```
template <typename T>       /* File: bst-remove.cpp */
void BST<T>::remove(const T& x) // leftmost of its right subtree
{
```

```
    if (is_empty())             // Item not found; do nothing
        return;

    if (x < root->value)        // Remove from the left subtree
        root->left.remove(x);

    else if (x > root->value)   // Remove from the right subtree
        root->right.remove(x);

    // The found node has 2 children
    else if (root->left.root && root->right.root)
    {
        root->value = root->right.find_min();
        // min is copied; can be deleted now
        root->right.remove(root->value);
    }

    else                        // Found node has 0 or 1 child
    {
        BSTnode* deleting_node = root;  // Save the root first
        root = (root->left.is_empty())
               ? root->right.root : root->left.root;

        // Set subtrees to nullptr before removal:
        // the destructor is recursive
        deleting_node->left.root = deleting_node->right.root
                                 = nullptr;
        delete deleting_node;
    }
}
```

**Question in the code: does `T` need `operator=`?**

**Question (comment `// operator= defined?`).** `root->value = root->right.find_min();` assigns one `T` to another. Is that always legal?

**Answer.** It compiles only because `T` has a **copy-assignment operator**. For `int`, `double` or `string` this is the compiler-generated memberwise one and works. For a class that owns a heap resource the generated version is wrong (it copies the pointer), so such a class **must** define a deep-copy `operator=` — otherwise `remove()` would silently create two owners of one buffer (recall the "default memberwise assignment" bug). The "copy then delete" order is essential: the minimum is copied into `value` before `remove` destroys the node that held it.

**Question in the code: why reset the child pointers before `delete`?**

**Question.** Why the line `deleting_node->left.root = deleting_node->right.root = nullptr;`? Deleting a node is going to destroy its members anyway.

**Answer.** Because the destructor is **recursive**: `delete deleting_node` runs `~BSTnode`, which in turn destroys the members `left` and `right`, i.e. each of those `BST` objects deletes its `root` — the whole sub-tree. But in the one-child case the surviving child has already been **hoisted** into this position (`root = ... root->left.root`), so it is now reachable from both the new root and the doomed node. Without the reset, the sub-tree would be deleted once through the hoisted pointer and again through the old node ⇒ **double delete**. Setting the two `root` pointers to `nullptr`

first makes the recursive destructor of the doomed node harmless.

### 6.4.9 BST Testing Program

```
#include <iostream>          /* File: test-bst.cpp */
using namespace std;
#include "bst.h"
#include "bst-contains.cpp"
#include "bst-print.cpp"
#include "bst-find-max.cpp"
#include "bst-find-min.cpp"
#include "bst-insert.cpp"
#include "bst-remove.cpp"

int main()
{
    BST<int> bst;
    while (true)
    {
        char choice; int value;
        cout << "Action: d/f/i/m/M/p/q/r/s "
             << "(deep-cp/find/insert/min/Max/print/quit/"
             << "remove/shallow-cp): ";
        cin >> choice;

        switch (choice)
        {
        case 'd':                    // Deep copy
        {
            BST<int>* bst2 = new BST<int>(bst);
            bst2->print(); delete bst2;
            break;
        }
        case 'f':                    // Find a value
            cout << "Value to find: "; cin >> value;
            cout << boolalpha << bst.contains(value) << endl;
            break;
        case 'i':                    // Insert a value
            cout << "Value to insert: "; cin >> value;
            bst.insert(value);
            break;
        case 'm':                    // Find the minimum value
            if (bst.is_empty())
                cerr << "Can't search an empty tree!" << endl;
            else
                cout << bst.find_min() << endl;
            break;
        case 'M':                    // Find the maximum value
            if (bst.is_empty())
                cerr << "Can't search an empty tree!" << endl;
            else
                cout << bst.find_max() << endl;
            break;
        case 'p':                    // Print the whole tree
        default:
```

```
            cout << endl; bst.print();
            break;
        case 'q':                       // Quit
            return 0;
        case 'r':
            cout << "Value to remove: "; cin >> value;
            bst.remove(value);
            break;
        case 's':                       // Shallow copy
        {
            BST<int> bst3 { std::move(bst) };
            bst3.print();
            bst.print();
            break;
        }
        }
    }
}
```

## What the test program teaches

After `'s'` (move) the original `bst.print()` prints **nothing** — the move constructor stole `root` and left `nullptr`. After `'d'` (deep copy) both trees are independent: deleting `bst2` prints its own `delete: ...` lines and leaves `bst` intact. With `'m'`/`'M'` on an empty tree the program reports the error instead of dereferencing `nullptr`.

## Complexity summary

| Operation | Average | Worst (degenerate tree) |
|---|---|---|
| search / insert / delete | $O(\log N)$ | $O(N)$ |
| find min / find max | $O(\log N)$ | $O(N)$ |
| inorder traversal (sorted output) | $O(N)$ | $O(N)$ |

The worst case is exactly why balanced trees (AVL) are needed: they guarantee height $O(\log N)$ by rebalancing after each insertion/deletion.

# 7 Hashing

## 7.1 Motivation

How would you find a student record given only the student's name? How does an electronic dictionary look up the word "computer"? Each machine has an IP address; how does an Internet service look up an IPv6 address?

**Answer: all three are the same problem**

**Answer.** In each case the key (a name, a word, an address) is mapped by a **hash function** to an index of an array, and the record is stored at that index. Looking it up is then a constant number of array accesses — $O(1)$ **on average** — instead of a scan or a binary search. This is what a **hash table** does.

## 7.2 The Hash Table ADT

- A hash table is an **array of fixed size** $m$ containing the data items.
- Each item has a **key**; searching is done on the key.
- Each key is mapped to a position in $[0, m-1]$; the mapping is the **hash function**.
- It supports **search, insertion, deletion** (deletion may be unnecessary in some applications).
- Items are **not ordered**. Therefore a hash table does **not** support: find min / find max, find successor / predecessor, reporting data in a range, or listing the data in order.
- Example applications: **symbol tables** in compilers (to track declared identifiers), **online spell checkers** (hash the whole dictionary once, then check each word of the document in constant time and report misspellings in order of appearance).

### 7.2.1 Unrealistic Solution: One Slot per Key

Let $U$ be the **universe of keys**, i.e. all possible key values. In direct addressing every slot of $T$ corresponds to one key of $U$: $T[k]$ holds the item with key $k$, or `nil` if there is none.

- Insertion, deletion and search are all $O(1)$ — excellent.
- **Problem:** it wastes space when $|U|$ is large compared with the number of items actually stored. HKUST student IDs are 8-digit numbers, so $|U| = 10^8$, but there are only about 7000 students.

### 7.2.2 Hash Functions

A hash function $h$ maps the universe of keys into the slots of a table $T[0, 1, \ldots, m-1]$:

$$h : \{k_0, k_1, \ldots, k_{|U|-1}\} \longrightarrow \{0, 1, \ldots, m-1\}, \qquad m \ll |U|.$$

The keys are assumed to be **natural numbers** (any other key type can be interpreted as one). $h(k)$ is the **hash value** of $k$, and the item with key $k$ is stored in $T[h(k)]$. Several keys may map to the same slot.

**Question: can we guarantee that distinct keys get distinct slots?**

**Question.** Can we ensure that any two distinct keys are hashed to different slots?

**Answer: No.** If $N > m$, where $m$ is the table size and $N$ the number of data items, then by the pigeonhole principle at least two keys must share a slot; this is called a **collision**. A hash function can at best minimise collisions, never eliminate them. Hence every hashing scheme needs **two** ingredients:

1. a good hash function that is fast to compute and minimises collisions;
2. a **collision-resolution** method for when they occur.

### 7.2.3 Hash Function Design

The simplest reasonable strategy is

$$h(k) = k \bmod m,$$

e.g. $m = 12$, $k = 100 \Rightarrow h(k) = 4$. It needs only one division — very fast. But:

- avoid $m = 2^p$: then $h(k)$ is just the $p$ **lowest-order bits** of $k$, so the function does not depend on the other bits;
- similarly, if the keys are decimal numbers, $m$ should not be a power of 10;
- it is good practice to choose $m$ a **prime**, not too close to an exact power of 2. Example: to hold 2000 numbers and tolerate an average of 3 per slot, choose $m = 701$ (a prime near 2000/3).

**String keys, method 1: add up the character values.** Most hash functions assume natural-number keys, so a string must first be turned into a number. The simplest way adds the ASCII values (or the alphabet positions) of its characters:

$$h(\text{key}) = \left(\textstyle\sum_i \text{key}[i]\right) \bmod m.$$

Two problems:

- all **permutations** of the same characters ("stop", "tops", "pots", "spot") get the **same** hash value;
- if $m$ is large the keys do not spread: with $m = 10007$ and words of at most 8 characters, ASCII values $\leq 127$ give sums between 0 and $127 \times 8 = 1016$, so only about 10% of the table can ever be used.

**Method 2: use the first three characters with positional weights.**

$$h(\text{key}) = \big(\text{key}[0] + 27 \cdot \text{key}[1] + 27^2 \cdot \text{key}[2]\big) \bmod m.$$

If the first three characters were random and $m = 10007$, this would distribute reasonably. But letters in English are not random: according to one dictionary there are only 2851 different combinations for the first three letters, so (with $m = 10007$) at most about 28% of the table can be reached.

**Method 3: include every character.**

$$h(\text{key}) = \left(\sum_{i=0}^{L-1} 37^{\,L-1-i} \cdot \text{key}[i]\right) \bmod m = \big(37^{L-1}\text{key}[0] + 37^{L-2}\text{key}[1] + \cdots + \text{key}[L-1]\big) \bmod m,$$

where $L$ is the key length. This involves **all** characters and distributes well. It is evaluated by Horner's rule in $O(L)$ with no overflow problems if the modulus is taken at each step:

```
unsigned int h = 0;
for (int i = 0; i < L; ++i)
    h = (h * 37 + key[i]) % m;
```

## 7.3 Collision Handling I: Separate Chaining

**Idea: a table of linked lists**

Using the idea of **equivalence classes**, each slot holds a **linked list** of all keys with the same hash value. Keys $k$ with $h(k) = i$ are chained on the list rooted at $T[i]$.

**Example.** Keys are the squares $\{1, 4, 9, 16, 25, 36, 49, 64, 81, 100, \dots\}$ and $h(k) = k \bmod 10$:

$$\begin{array}{ll} 0 & 100 \to 40 \to \dots \\ 1 & 81 \to 1 \\ 4 & 64 \to 4 \\ 5 & 25 \\ 6 & 36 \to 16 \\ 9 & 49 \to 9 \end{array}$$

**Operations.** To insert $k$: compute $h(k)$; if $T[h(k)]$ is null, create a one-node list; otherwise add $k$ at the **beginning** of the list. To delete $k$: compute $h(k)$, search that list, and delete the node if found.

**Features.** If the hash function works well, each list is a small constant long, so search/insert/delete are $O(1)$ on average. **Advantage:** deletion is easy. **Disadvantage:** the memory allocations/deallocations of linked-list nodes slow the operations down.

## 7.4 Collision Handling II: Open Addressing

There are **no separate chains**: the table itself stores all objects, of different or equal keys. If the home slot is occupied, the key is **relocated** to another slot; the method must define the sequence of slots examined.

**Generic open addressing**

To insert $k$ compute $h_0(k)$. If $T[h_0(k)]$ is empty, put it there. Otherwise probe $h_1(k), h_2(k), \dots$ until an empty cell is found, where

$$h_i(k) = \big(\text{hash}(k) + f(i)\big) \bmod m, \qquad f(0) = 0.$$

Different open-addressing methods differ only in $f(\cdot)$. The two questions they must answer are: what is the relocation scheme? and how do we search for $k$ later?

### 7.4.1 Linear Probing

$$f(i) = i, \qquad h_i(k) = \big(\text{hash}(k) + i\big) \bmod m.$$

Cells are probed **sequentially with wrap-around**. Algorithm: for $i = 0$ to $m - 1$, compute $j = (\text{hash}(k) + i) \bmod m$; if $T[j]$ is empty put $k$ there and stop. If no empty slot exists, the table is **full** — report an error.

**Worked example: insert 89, 18, 49, 58, 69 with $h(k) = k \bmod 10$**

- 89: $h = 9$, empty $\Rightarrow T[9]$.
- 18: $h = 8$, empty $\Rightarrow T[8]$.
- 49: $h = 9$ occupied $\Rightarrow$ probe $T[0]$ (wrap-around), empty $\Rightarrow T[0]$.
- 58: $h = 8 \Rightarrow$ probe $T[8], T[9], T[0], T[1] \Rightarrow T[1]$.
- 69: $h = 9 \Rightarrow$ probe $T[9], T[0], T[1], T[2] \Rightarrow T[2]$.

| Index | 8 | 9 | 0 | 1 | 2 | others |
|---|---|---|---|---|---|---|
| Value | 18 | 89 | 49 | 58 | 69 | empty |

Note how the probe of 58 had to cross the wrap-around boundary: the cluster 18, 89 has merged with 49 to form the block $T[8], T[9], T[0]$.

**Primary clustering**

A block of contiguously occupied entries is a **cluster**. On average a new key hits the middle of a cluster, so the insertion time is proportional to half the cluster size — the larger the cluster, the slower everything.

- Once $h(k)$ falls into a cluster, that cluster **definitely** grows by one, worsening future insertions.
- If two clusters are separated by exactly one empty entry, a single insertion into that entry **merges** them, so the cluster size (and the insertion time) can jump dramatically after one insertion.
- Large clusters are easy targets for collisions.

### 7.4.2 Quadratic Probing

$$f(i) = i^2, \qquad h_i(k) = \big(\text{hash}(k) + i^2\big) \bmod m.$$

**Same keys, same $h(k) = k \bmod 10$, quadratic probing**

- $89 \Rightarrow T[9]$; $\quad 18 \Rightarrow T[8]$.
- 49: $h = 9$ occupied $\Rightarrow$ probe $T[(9+1) \bmod 10] = T[0] \Rightarrow T[0]$.
- 58: probes $T[8], T[9], T[(8+4) \bmod 10] = T[2] \Rightarrow T[2]$.
- 69: probes $T[9], T[(9+1) \bmod 10] = T[0]$, $T[(9+4) \bmod 10] = T[3] \Rightarrow T[3]$.

| Index | 8 | 9 | 0 | 2 | 3 | others |
|---|---|---|---|---|---|---|
| Value | 18 | 89 | 49 | 58 | 69 | empty |

Compared with linear probing, 58 and 69 jumped away from the cluster (+4 instead of +2), so the cluster stays short.

Two keys with different home positions get **different** probe sequences: with $m = 101$, $h(k_1) = 30$, $h(k_2) = 29$ the sequences are $30, 31, 34, 39, \ldots$ and $29, 30, 33, 38, \ldots$. If $m$ is prime, a new key can always be inserted as long as the table is at least **half empty** (proof in Weiss' textbook).

**Secondary clustering**

Keys that hash to the same home position probe the same alternative cells. Simulations suggest this costs less than an extra half probe per search, so it is mild. To avoid it altogether the probe sequence must depend on the **original key value**, not on the home position — that is double hashing.

### 7.4.3 Double Hashing

$$f(i) = i \times \text{hash}_2(k), \qquad h_i(k) = \big(\text{hash}(k) + i \times \text{hash}_2(k)\big) \bmod m.$$

A common choice is $\text{hash}_2(k) = R - (k \bmod R)$ with $R < m$ and $R$ prime.

**Worked example: $m = 10$, $R = 7$, $h_2(k) = 7 - (k \bmod 7)$**

$\text{hash}(k) = k \bmod 10$ as before.

- 89: $h = 9$ empty $\Rightarrow T[9]$; 18: $h = 8$ empty $\Rightarrow T[8]$.
- 49: $h = 9$ occupied; $h_2(49) = 7 - (49 \bmod 7) = 7 - 0 = 7 \Rightarrow$ probe $T[(9 + 7) \bmod 10] = T[6] \Rightarrow T[6]$.
- 58: $h = 8$ occupied; $h_2(58) = 7 - (58 \bmod 7) = 7 - 2 = 5 \Rightarrow$ probe $T[(8 + 5) \bmod 10] = T[3] \Rightarrow T[3]$.
- 69: $h = 9$ occupied; $h_2(69) = 7 - (69 \bmod 7) = 7 - 6 = 1 \Rightarrow$ probe $T[(9 + 1) \bmod 10] = T[0] \Rightarrow T[0]$.

| Index | 0 | 3 | 6 | 8 | 9 |
|---|---|---|---|---|---|
| Value | 69 | 58 | 49 | 18 | 89 |

The three colliding keys 49, 58, 69 end up in three different slots, each reached with its own step size.

**Rules for `hash2`**

- $\text{hash}_2(k)$ must **never be zero** — otherwise every probe stays at the same slot forever.
- $\text{hash}_2(k)$ must be **relatively prime** to $m$; otherwise only a fraction of the table can be examined. Example: if $\text{hash}(k) = 0$ and $\text{hash}_2(k) = m/2$, the only entries ever probed are $T[0]$ and $T[m/2]$.
- Safe recipe: make $m$ prime and $R$ a prime smaller than $m$ with $\text{hash}_2(k) = R - (k \bmod R)$.
- Quadratic probing needs no second hash function, so it is simpler and usually faster in practice.

## 7.5 Deletion in Open Addressing

Actual deletion cannot be performed in an open-addressing table, because emptying a slot would **break the probing sequence** of other keys: a later search stops at the first empty cell and would wrongly conclude "not found".

**Why: an example**

With linear probing and the table of the example above (18 at 8, 89 at 9, 49 at 0, 58 at 1, 69 at 2), suppose we really delete 49 from $T[0]$. Searching for 58 now starts at $T[8]$: not there; $T[9]$: 89; $T[0]$: **empty** $\Rightarrow$ the search stops and reports 58 missing — although 58 sits in $T[1]$.

**Solution: lazy deletion.** Give each entry a state field `EMPTY` / `ACTIVE` / `DELETED`. A deleted entry becomes `DELETED` (a "tombstone"): probing **continues** through it, but it may be reused by an insertion. (Slots marked `DELETED` must not accumulate forever, otherwise searches degrade.)

## 7.6 Load Factor and Re-hashing

- The **load factor** is $\alpha = N/m$, where $N$ is the number of items actually hashed into the table of size $m$.
- Operations slow down drastically when $\alpha$ becomes large.
- When $\alpha$ gets too high, (roughly) **double** the table size and **re-hash** every item with a new hash function.

- Re-hashing is expensive, but it happens infrequently in a well-designed table, so the amortised cost stays constant.

**Summary**

| Method | Strength | Weakness |
|---|---|---|
| separate chaining | easy deletion, simple insertion | pointer allocation overhead |
| linear probing | simplest, cache-friendly | primary clustering |
| quadratic probing | no clusters, no second hash function | secondary clustering, needs prime $m$ |
| double hashing | almost no clustering | extra hash function, $h_2$ tricky |

All of them give $O(1)$ average search/insert/delete, at the price of **no ordering** (no min/max, no successor, no sorted listing).

# 8 The Standard Template Library (STL)

The STL is a collection of powerful, template-based, reusable code. It implements many general-purpose **containers** (data structures) together with **algorithms** that work on them. Three topics must be understood:

1. **containers** — where the data lives;
2. **iterators** — generalized pointers that let algorithms walk any container;
3. **algorithms** — global function templates (find, count, sort, ...) written once for all containers.

## 8.1 Container Classes

**Container class**

A container class holds a collection of **homogeneous** objects (all of the same type). It is the typical use of a **class template**: we need containers of different element types at different times, and the element type need not be known when the container is designed.

The exercise below designs a sequence container that looks like an array but is a **first-class type**: assignment and call-by-value are possible. (The STL `vector` is better; this is for understanding.)

### 8.1.1 The Array Class Template

```
template <typename T>          /* File: arrayT.h */
class Array
{
private:
    int _size;
    T* _value;

public:
    Array<T>(int n = 10);   // Default + conversion ctor
    Array<T>(const Array<T>&);      // Copy constructor
    ~Array<T>();

    int size() const { return _size; }
    void init(const T& k);

    // Copy assignment operator
    Array<T>& operator=(const Array<T>& a);
    T& operator[](int i) { return _value[i]; }       // lvalue
    const T& operator[](int i) const { return _value[i]; }
};                                                   // rvalue
```

**Simplification inside the template**

Within a class template, the type argument may be omitted:

```
template <typename T>          /* File: array.h */
class Array
{
private:
    T* _value;
```

```
    int _size;
public:
    Array(int n = 10);              // Default + conversion
    Array(const Array&);            // Copy constructor
    ~Array();

    int size() const { return _size; }
    void init(const T& k);
    Array& operator=(const Array&);    // Copy assignment
    T& operator[](int i) { return _value[i]; }
    const T& operator[](int i) const { return _value[i]; }
};
```

`Array<T>` outside the class body, `Array` inside it.

```
template <typename T>     /* File: array-constructors.h */
Array<T>::Array(int n) : _value( new T [n] ), _size(n) { }

template <typename T>
Array<T>::Array(const Array<T>& a)
: Array(a._size)              // Delegating constructor
{
    for (int i = 0; i < _size; ++i)
        _value[i] = a._value[i];
}

template <typename T> Array<T>::~Array() { delete [] _value; }

template <typename T> void Array<T>::init(const T& k)
{
    for (int i = 0; i < _size; ++i)
        _value[i] = k;
}

template <typename T>     /* File: array-op=.h */
// Deep copy
Array<T>& Array<T>::operator=(const Array<T>& a)
{
    if (&a != this)           // Avoid self-assignment: a = a
    {
        delete [] _value;     // First remove the old data
        _size = a._size;
        _value = new T [_size];  // Re-allocate memory

        for (int j = 0; j <_size; ++j)  // Copy new data
            _value[j] = a[j];
    }

    return (*this);
}
```

The three pieces of a resource-owning class appear again: constructor `new`s, destructor `delete []`s, and assignment **deep-copies** after checking for **self-assignment** (`a = a` would otherwise delete the buffer it is about to copy from).

### 8.1.2 `operator<<` as a Global or a Friend Template

```
template <typename T>       /* File: array-op-os.h */
ostream& operator<<(ostream& os, const Array<T>& a)
{
    os << "#elements stored = " << a.size() << endl;

    for (int j = 0; j < a.size(); ++j)
        os << a[j] << endl;

    return os;
}
```

A **friend function template** may also be declared inside the class and defined outside:

```
template <typename T>    /* File: array-w-os-friend.h */
class Array
{
    template <typename S>
    friend ostream& operator<<(ostream& os, const Array<S>& x);

private:
    T* _value;
    int _size;

public:
    Array(int n = 10);              // Default + conversion
    Array(const Array&);            // Copy constructor
    ~Array();

    int size() const { return _size; }
    void init(const T& k);
    Array& operator=(const Array&); // Copy assignment
    T& operator[](int i) { return _value[i]; }
    const T& operator[](int i) const { return _value[i]; }
};

template <typename T>   /* File: array-op-os-friend.h */
ostream& operator<<(ostream& os, const Array<T>& a)
{
    os << "#elements stored = " << a._size << endl;

    for (int i = 0; i < a._size; ++i)
        os << a._value[i] << endl;

    return os;
}
```

Inside the friend version the private members `_size` and `_value` can be used directly — that is the only gain over the global version, which must go through `size()` and `operator[]`.

### 8.1.3 Example: Using the Class

```
#include <iostream>        /* File: test-array.cpp */
using namespace std;
#include "array.h"
#include "array-constructors.h"
```

```
#include "array-op=.h"
#include "array-op-os.h"

int main()
{
    Array<int> a(3);
    a.init(98); cout << a << endl;
    a = a; a[2] = 17; cout << a << endl;

    Array<char> b(4);
    b.init('g'); b[0] = a[1]; cout << b << endl;

    const Array<char> c = b;
    // c[2] = 5; // Error: assignment of read-only location
    cout << c << endl;

    Array<int> d;
    d = a; cout << d << endl;
    return 0;
}

#elements stored = 3
98
98
98
#elements stored = 3
98
98
17
#elements stored = 4
b
g
g
g
#elements stored = 4
b
g
g
g
#elements stored = 3
98
98
17
```

- `a = a` is **self-assignment**: without the `&a != this` test the buffer would be deleted and then read, so the test is imperative.
- `b[0] = a[1]` assigns the `int` 98 into a `char` slot, i.e. the character `'b'` (ASCII 98) replaces `'g'`.
- `c` is `const`, so `c[2] = 5` fails — the **const** `operator[]` is selected and returns `const T&`. The copy-initialisation `c = b` is still fine: it uses the copy constructor, not assignment.
- `d` is first built with the default `Array(int n = 10)` (10 elements) and then replaced by a deep copy of `a` (3 elements) — the old 10-element buffer is released by `operator=`.

## 8.2 Containers in the STL

| | |
|---|---|
| **Sequence** | `vector`, `list`, `deque` |
| **Associative** | `map`, `multimap`, `multiset`, `set` |
| **Adaptors** | priority queue, queue, stack |
| **Near-containers** | `bitset`, `valarray`, `string` |

- **Sequence containers** represent sequential data structures starting from index/location 0.
- **Associative containers** are non-sequential and store `(key, value)` pairs.
- **Container adaptors** are adapted containers supporting only a limited set of operations.
- **Near-containers** are C-like pointer-based arrays (`string`, `bitset`, `valarray`): they behave like sequence containers but do not support all their capabilities.

Containers in the same category share a set of same or similar public member functions.

### 8.2.1 Properties of Sequence Containers

| Container | Access | Add / Remove |
|---|---|---|
| `vector` (1D array) | $O(1)$ random access | $O(1)$ at the end, $O(n)$ in front/middle |
| `list` (doubly-linked list) | $O(n)$ in the middle | $O(1)$ at any position, front, end |
| `deque` (double-ended queue) | $O(1)$ random access | $O(1)$ at front/back, $O(n)$ middle |

- Element access for **all**: `front()` (first), `back()` (last).
- Element access for `vector` and `deque`: `[]` (subscript, **index not checked**).
- Add/remove for all: `push_back()`, `pop_back()`.
- Add/remove for `list` and `deque`: `push_front()`, `pop_front()`.
- `insert(p, x)` and `erase(p)` take an **iterator** `p`: fast for `list`, available but slow (element shifting) for `vector`/`deque`.
- Miscellaneous: `clear()`, `size()`, `empty()`, `resize(int)`; comparison operators `==`, `!=`, `<` are defined.

## 8.3 Container Adaptors: Stack and Queue

### 8.3.1 Stack (LIFO)

Think of a pile of cookies: new cookies go on **top**, and cookies are consumed starting at the top — **last-in first-out**. Its operations are `top`, `push`, `pop` (plus `empty`, `size`).

```
template<typename T, typename Sequence = deque<T> >
class stack
{
protected:
    Sequence c;                 // Underlying container

public:
    typedef typename Sequence::value_type value_type;
    typedef typename Sequence::reference reference;
    typedef typename Sequence::const_reference const_reference;
    typedef typename Sequence::size_type size_type;

    // (Default) Constructor
    explicit stack(const Sequence& _c = Sequence()) : c(_c) { }
```

```
    bool empty() const { return c.empty(); }
    size_type size() const { return c.size(); }

    // R/W reference to the data at the first element
    reference top() { return c.back(); }

    // Read-only version of top()
    const_reference top() const { return c.back(); }

    // Create an element at the top and assign x to it
    void push(const value_type& x) { c.push_back(x); }

    // Shrink the stack by one. No data is returned.
    void pop() { c.pop_back(); }
};
```

**`typedef` and the adaptor idea**

`typedef <a type expression> <type-synonym>` introduces a synonym, e.g. `typedef typename Sequence::value_type value_type;` makes `stack<int>::value_type` mean `int`. The adaptor **restricts** the underlying container's interface: of all `deque` operations only `back`, `push_back`, `pop_back`, `empty`, `size` are exposed, which is exactly LIFO behaviour. Note that `pop()` returns `void` — get the value with `top()` before popping.

```
#include <iostream>   /* File: decimal2binary.cpp */
#include <stack>
using namespace std;

// Convert +ve decimal number to binary using a stack
int main()
{
    stack<int> a;
    int x, number;

    while (cin >> number)
    {
        // Conversion: decimal to binary
        x = number;
        do { a.push(x % 2); x /= 2; } while (x > 0);

        // Print a binary that is stored on a stack
        cout << number << " (base 10) = ";
        while (!a.empty()) { cout << a.top(); a.pop(); }
        cout << " (base 2)" << endl;
    }

    return 0;
}
```

**Trace:** $26_{10}$

Step 1: divide repeatedly, pushing each remainder: $26/2 \to 0$, $13/2 \to 1$, $6/2 \to 0$, $3/2 \to 1$, $1/2 \to 1$; the stack (top first) is $1, 1, 0, 1, 0$. Step 2: popping prints 11010. Hence $26_{10} = 11010_2$. **Why a stack?** the remainders come out backwards, and LIFO reverses them for free.

```
#include <iostream>  /* File: balanced-paren.cpp */
#include <stack>
using namespace std;

const char L_PAREN = '('; const char R_PAREN = ')';
const char L_BRACE = '{'; const char R_BRACE = '}';
const char L_BRACKET = '['; const char R_BRACKET = ']';
bool balanced_paren(const char* expr);

int main()   // To check if a string has balanced parentheses
{
    char expr[1024];
    cout << "Input an expression containing parentheses: ";
    cin >> expr;
    cout << boolalpha << balanced_paren(expr) << endl;
    return 0;
}

bool check_char_stack(stack<char>& a, char c)
{
    if (a.empty()) return false;
    if (a.top() != c) return false;
    a.pop(); return true;
}

bool balanced_paren(const char* expr)
{
    stack<char> a;
    for (const char* s = expr; *s != '\0'; ++s)
        switch (*s)
        {
        case L_PAREN: case L_BRACE: case L_BRACKET:
            a.push(*s); break;

        case R_PAREN:
            if (!check_char_stack(a, L_PAREN)) return false;
            break;
        case R_BRACE:
            if (!check_char_stack(a, L_BRACE)) return false;
            break;
        case R_BRACKET:
            if (!check_char_stack(a, L_BRACKET)) return false;
            break;

        default: break;
        }

    return a.empty();
}
```

**Two traces: `[()][()()]()` and `[(])`**

**Balanced.** Push `[`, push `(`; `)` matches the top `(` ⇒ pop; `]` matches the top `[` ⇒ pop; the second pair of brackets behaves the same; final `()` matches. At the end the stack is **empty** ⇒ `true`.

**Not balanced.** Push `[`, push `(`; the next character is `]`, but the top is `(` ⇒

`check_char_stack` returns `false` ⇒ `false`. This is exactly the case a counter-based algorithm gets wrong: the counts match, the order does not.

The final `return a.empty();` catches the other failure mode: an expression like `(()` has correctly matching right parentheses but leaves two unmatched left ones on the stack.

### 8.3.2 Queue (FIFO)

People lining up for tickets: newcomers join the **back**, the customer at the **front** leaves — **first-in first-out**. Operations: `front`, `push` (enqueue), `pop` (dequeue), plus `back`, `empty`, `size`.

```
template<typename T, typename Sequence = deque<T> >
class queue
{
protected:
    Sequence c;                 // Underlying container

public:
    typedef typename Sequence::value_type value_type;
    typedef typename Sequence::reference reference;
    typedef typename Sequence::const_reference const_reference;
    typedef typename Sequence::size_type size_type;

    explicit queue(const Sequence& _c = Sequence()) : c(_c) { }

    bool empty() const { return c.empty(); }
    size_type size() const { return c.size(); }

    reference front() { return c.front(); }
    const_reference front() const { return c.front(); }

    reference back() { return c.back(); }
    const_reference back() const { return c.back(); }

    // Create an element at the end; i.e., enqueue
    void push(const value_type& x) { c.push_back(x); }

    // Shrink the queue by one; i.e., dequeue. No data returned.
    void pop() { c.pop_front(); }
};

#include <iostream>      /* File: int-queue-test.cpp */
#include <queue>
using namespace std;

void print_queue_info(const queue<int>& a) {
    cout << "\nNo. of data currently on the queue = "
         << a.size() << endl;
    if (!a.empty()) {
        cout << "First: " << a.front() << "\nLast: "
             << a.back() << endl; }
}

int main()
{
```

```
    queue<int> a; print_queue_info(a);
    a.push(4);   print_queue_info(a);
    a.push(15);  print_queue_info(a);
    a.push(26);  print_queue_info(a);
    a.push(37);  print_queue_info(a);
    a.pop();     print_queue_info(a);
    a.push(48);  print_queue_info(a);
    a.push(59);  print_queue_info(a);
    a.pop();     print_queue_info(a);
    a.pop();     print_queue_info(a);
    a.pop();     print_queue_info(a);
    a.pop();     print_queue_info(a);
    a.pop();     print_queue_info(a); return 0;
}
```

**What is printed?**

```
size 0            (empty, so no First/Last line)
size 1, First 4  Last 4
size 2, First 4  Last 15
size 3, First 4  Last 26
size 4, First 4  Last 37
size 3, First 15 Last 37        (4 dequeued)
size 4, First 15 Last 48
size 5, First 15 Last 59
size 4, First 26 Last 59        (15 dequeued)
size 3, First 37 Last 59
size 2, First 48 Last 59
size 1, First 59 Last 59
size 0            (empty again)
```

**Point:** `pop()` removes the **front** but returns nothing; the value must be read with `front()` first. Popping an empty queue is undefined behaviour — the code above avoids it by checking `empty()` only for printing, and by popping exactly as many times as it pushed.

## 8.4 STL Iterators: Generalized Pointers

### 8.4.1 Pointers Traverse an Array

```
#include <iostream>   /* File: print-int-array.cpp */
using namespace std;

int main()
{
    const int LENGTH = 5;
    int x[LENGTH];

    for (int j = 0; j < LENGTH; ++j)
        x[j] = j;

    // x_end points to a non-existing element just beyond
    const int* x_end = &x[LENGTH];

    for (const int* p = x; p != x_end; ++p)
        cout << *p << endl;
```

```
    return 0;
}
```

For a sequence of values of a basic type, a pointer `p` supports exactly the operations an algorithm needs:

| Operation | Goal |
|---|---|
| `p = x` | initialize to the beginning of an array |
| `*p` | access an element by dereferencing |
| `p->` | access a member of the pointed-to element |
| `--p` / `++p` | move to the previous / next element |
| `==`, `!=` | pointer comparisons |

**Iterators are generalized pointers**: an object with the same operations, provided by the container. `list<int>::iterator` is the iterator of a list of `int`; the `const_iterator` is its read-only version (the element it "points" to cannot be modified). Each container provides `begin()` and `end()`; `end()` is the **one-past-the-last** position, so a loop is `for (p = x.begin(); p != x.end(); ++p)`.

```
#include <iostream>          /* File: print-list.cpp */
using namespace std;
#include <list>              // STL list

int main()
{
    list<int> x;             // An int STL list
    for (int j = 0; j < 5; ++j)
        x.push_back(j);      // Append items to an STL list

    list<int>::const_iterator p;    // STL list iterator
    for (p = x.begin(); p != x.end(); ++p)
        cout << *p << endl;

    return 0;
}
```

### 8.4.2 Iterators Decouple Algorithms from Containers

```
typedef int* Int_Iterator;  /* File: find-int-iterator.cpp */

/* Actually this find function is already defined in STL */
Int_Iterator
find(Int_Iterator begin, Int_Iterator end, const int& value)
{
    while (begin != end && *begin != value)
        ++begin;

    return begin;
}

#include <iostream>          /* File: find-test.cpp */
using namespace std;
typedef int* Int_Iterator;

int main()
```

```
{
    const int SIZE = 10; int x[SIZE];
    for (int i = 0; i < SIZE; i++)
        x[i] = 2 * i;

    Int_Iterator begin = x; Int_Iterator end = &x[SIZE];
    while (true)
    {
        cout << "Enter number: "; int num; cin >> num;
        Int_Iterator position = find(begin, end, num);

        if (position == end)
            cout << "Not found\n";
        else if (++position != end)
            cout << "Found before the item " << *position << '\n';
        else
            cout << "Found as the last element\n";
    }
    return 0;
}

template <class Iterator, class T>  /* File: find-template.h */
Iterator find(Iterator begin, Iterator end, const T& value)
{
    while (begin != end && *begin != value)
        ++begin;

    return begin;
}
```

**Why iterators are so great**

The template `find` contains **no** information about the container's implementation nor about how to move from one element to the next — it only requires the six operations listed above. Therefore **the same** `find` works for a raw array, a `vector`, a `list`, a `deque`, etc. This is what "separating algorithms from containers" means, and it is the whole design idea of the STL.

```
#include <iostream>  /* File: find-iterator-test.cpp */
using namespace std;
#include <vector>

int main()
{
    const int SIZE = 10; vector<int> x(SIZE);
    for (int i = 0; i < x.size(); i++)
        x[i] = 2 * i;

    while (true)
    {
        cout << "Enter number: "; int num; cin >> num;
        vector<int>::iterator position =
            find(x.begin(), x.end(), num);

        if (position == x.end())
            cout << "Not found\n";
        else if (++position != x.end())
```

```
            cout << "Found before the item " << *position << '\n';
        else
            cout << "Found as the last element\n";
    }

    return 0;
}
```

The body is **identical** to the array version — only the type of the iterators changed. `++position` after a successful find moves to the next element (or to `end()` if the found element was the last), which is how the program can say what follows the found item.

## 8.5 STL Algorithms

STL algorithms are implemented as **global function templates**. `find()` searches sequentially and stops when an item matches its third argument.

```
template <class Iterator, class T>  /* File: stl-find.cpp */
Iterator find(Iterator first, Iterator last, const T& value)
{
    while (first != last && *first != value)
        ++first;

    return first;
}

#include <iostream>      /* File: find-composer.cpp */
using namespace std;
#include <string>
#include <list>
#include <algorithm>

int main()
{
    list<string> composers;
    composers.push_back("Mozart");
    composers.push_back("Bach");
    composers.push_back("Chopin");
    list<string>::iterator p =
        find(composers.begin(), composers.end(), "Bach");

    if (p == composers.end())
        cout << "Not found." << endl;
    else if (++p != composers.end())
        cout << "Found before: " << *p << endl;
    else
        cout << "Found at the end of the list." << endl;

    return 0;
}
```

Output: `Found before: Chopin`.

**Sub-sequences are iterator ranges**

A sequence or sub-sequence is always specified by **two iterators**: the beginning and one-past-the-end. That is why `find(++p, x.end(), v)` searches only the tail of the

container. The helper used by the following examples is

```
/* File: init.h */
inline int quadratic(int x) { return -x*x + 40*x + 22; }

template <typename T>
void my_initialization(T& x, int num_items)
{
    for (int j = 0; j < num_items; ++j)
        x.push_back( quadratic(j) );
}
```

**Question in the code: rewrite `quadratic(j)` with a lambda**

**Question** (comment `// Can you rewrite using lambda?`). Yes:

```
template <typename T>
void my_initialization(T& x, int num_items)
{
    auto q = [](int j) { return -j*j + 40*j + 22; };
    for (int j = 0; j < num_items; ++j)
        x.push_back( q(j) );   // same values, no named function
}
```

**Answer explained.** The lambda is **captureless**, so it could even be passed where a `int(*)(int)` is expected. A named function is preferable here (it is reused and easier to test), but the point of the exercise is that a lambda can replace a one-off helper without polluting the namespace. The lambda could also be inlined directly: `x.push_back( [](int j) { return -j*j+40*j+22; }(j) );`.

### 8.5.1 Example: the 2nd Occurrence of a Value

```
#include <iostream>   /* File: find-2nd-occurrence.cpp */
using namespace std;
#include <vector>
#include <algorithm>
#include "init.h"

int main()
{
    const int search_value = 341;
    vector<int> x;
    my_initialization(x, 100);

    vector<int>::iterator p =
        find(x.begin(), x.end(), search_value);

    if (p != x.end())   // Value found for the first time!
    {
        p = find(++p, x.end(), search_value);  // Again
        if (p != x.end())
            cout << search_value << " appears after "
                 << *--p << endl;
    }
    return 0;
}
```

**Answer: what is printed?**

$x[j] = -j^2+40j+22 = 341 \iff j^2-40j+319 = 0 \iff (j-11)(j-29) = 0$, so 341 occurs at indices **11** and **29**. The second `find` starts at `++p` (index 12) and returns index 29. Then `*--p` steps back to index **28**, whose value is $-784+1120+22 = 358$.

**Output:** `341 appears after 358`. Note the two idioms: `++p` to skip the found element before searching again, and `--p` to look at the element before the second occurrence. (Since $j < 100$, both occurrences exist; if the value appeared only once, the second `find` would return `x.end()` and nothing would be printed.)

### 8.5.2 `find_if()`: Searching by Condition

```
template <class Iterator, class Predicate>
Iterator find_if(Iterator first, Iterator last,     /* File:
                 Predicate predicate)               stl-find-if.cpp */
{
    while (first != last && !predicate(*first))
        ++first;

    return first;
}
```

`find_if` is more general than `find`: it stops when a **condition** is satisfied. The condition is called a **predicate** — a function (or function object) returning `bool`. This permits partial matching, or matching by key, and is the reason C++ allows a function to be passed as an argument to another function.

```
#include <iostream>       /* File: find-gt350.cpp */
using namespace std;
#include <vector>
#include <algorithm>
#include "init.h"

bool greater_than_350(int value) { return value > 350; }

int main()
{
    vector<int> x;
    my_initialization(x, 100);

    vector<int>::const_iterator p =
        find_if( x.begin(), x.end(), greater_than_350 );

    if (p != x.end())
        cout << "Found element: " << *p << endl;

    return 0;
}
```

**Answer: which element is found?**

$x[j]$ grows until $j = 20$ ($x[20] = 422$) and then decreases. The first value above 350 is at $j = 12$: $x[12] = -144 + 480 + 22 = 358$ ($x[11] = 341$ is not $> 350$). So the output is `Found element: 358`.

### 8.5.3 Function Objects (Functors)

STL **function objects** generalize function pointers. An object that can be called like a function is a function object, functoid, or functor; function pointers and lambdas are two examples. An object can be called if it overloads `operator()`. Function objects are more powerful than function pointers because they can carry **data members**, i.e. information or internal state. A function object returning `bool` is a **predicate**.

```
class Greater_Than          /* File: fo-greater-than.h */
{
private:
    int limit;
public:
    Greater_Than(int a) : limit(a) { }
    bool operator()(int value) { return value > limit; }
};

#include <iostream>   /* File: fo-greater-than.cpp */
using namespace std;
#include <algorithm>
#include <vector>
#include "init.h"
#include "fo-greater-than.h"

int main()
{
    vector<int> x; my_initialization(x, 100);
    int limit = 0;

    while (cin >> limit)
    {
        vector<int>::const_iterator p =
            find_if(x.begin(), x.end(), Greater_Than(limit));

        if (p != x.end())
            cout << "Element found: " << *p << endl;
        else
            cout << "Element not found!" << endl;
    }

    return 0;
}
```

The call `find_if` above, with `Greater_Than(limit)`, is the same as

```
Greater_Than g(limit);   // a temporary function object
p = find_if( x.begin(), x.end(), g );
```

Each time `find_if` examines an item `x[j]` it calls `g(x[j])` — formally, `operator()` of `g`. The limit is stored inside `g` — a plain function pointer could not do that without a global variable.

```
#include <iostream>      /* File: fo-count.cpp */
using namespace std;
#include <vector>
#include <algorithm>
#include "fo-greater-than.h"

int main()
```

```
{
    vector<int> x;
    for (int j = -5; j < 5; ++j)
        x.push_back(j*10);

    // Count how many items are greater than 10
    cout << count_if(x.begin(), x.end(), Greater_Than(10))
         << endl;

    return 0;
}
```

**Answer: how many?**

`x` = $\{-50, -40, -30, -20, -10, 0, 10, 20, 30, 40\}$. Values strictly greater than 10: $20, 30, 40 \Rightarrow$ prints `3`. (`count_if` takes a predicate and counts the items for which it is true; `count` would take a value.)

```
#include <iostream>     /* File: fo-sum.cpp */
using namespace std;
#include <list>
#include <algorithm>

class Sum
{
private:
    int sum;
public:
    Sum() : sum(0) { }
    void operator()(int value) { sum += value; }
    int result() const { return sum; }
};

int main()
{
    list<int> x;
    for (int j = 0; j < 5; ++j) x.push_back(j); // Init x
    Sum sum = for_each( x.begin(), x.end(), Sum() );
    cout << "Sum = " << sum.result() << endl; return 0;
}
```

**The trick in `for_each`**

`for_each` calls `g(*first)` for every element and **returns the function object** `g` by value. The functor accumulates $0 + 1 + 2 + 3 + 4$ internally, so once the loop is over the returned copy carries the result (prints `Sum = 10`). This is how algorithms "report back" through a stateful functor instead of a global variable.

```
template <class Iterator, class Function>  /* stl-foreach.h */
Function for_each(Iterator first, Iterator last, Function g)
{
    for ( ; first != last; ++first )
        g(*first);

    return g; // Returning the input function!
}
```

```
template <class Iterator1, class Iterator2, class Function>
Iterator2 transform(Iterator1 first, Iterator1 last,
                    Iterator2 result, Function g)
{
    for ( ; first != last; ++first, ++result )
        *result = g(*first);

    return result;
}

#include <list>        /* File: fo-add.h */
#include <vector>
#include <algorithm>

class Add
{
private:
    int data;
public:
    Add(int i) : data(i) { }
    int operator()(int value) { return value + data; }
};

class Print
{
private:
    ostream& os;
public:
    Print(ostream& s) : os(s) { }
    void operator()(int value) { os << value << " "; }
};

#include <iostream>        /* File: fo-add10.cpp */
using namespace std;
#include "fo-add.h"

int main()
{
    list<int> x;
    for (int j = 0; j < 5; ++j)  // Initialize x
        x.push_back(j);

    vector<int> y(x.size());
    transform( x.begin(), x.end(), y.begin(), Add(10) );

    for_each( y.begin(), y.end(), Print(cout) );
    cout << endl;

    return 0;
}
```

Output: `10 11 12 13 14`. `transform` writes into a second range, so `y` must already have room; `for_each` with `Print(cout)` holds a reference to the output stream (`ostream&` member) — another thing a function pointer cannot carry.

### 8.5.4 Other Algorithms in the STL

- `min_element` and `max_element`;
- `equal`;
- `generate` (replace elements by applying a function object);
- `remove`, `remove_if` (remove elements);
- `reverse`, `rotate` (rearrange a sequence);
- `random_shuffle`;
- `binary_search`;
- `sort` (using a function object to compare two elements);
- `merge`, `unique`;
- `set_union`, `set_intersection`, `set_difference`.

**When to use what**

| Need | Use |
|---|---|
| sequential scan by value | `find` |
| scan by condition | `find_if` / `count_if` with a predicate |
| apply an action to every element | `for_each` |
| produce a transformed copy | `transform` |
| ordered data, range queries | `map` / `set` (balanced tree) |
| $O(1)$ lookup by key, no order | hash table (see the hashing section) |